Oracle® Video Client

Developer's Guide

Release 3.2 for Windows 95, 98 and NT 4.0

September, 1999

Part No. A53949-04

ORACLE®

Oracle Video Client Developer's Guide, Release 3.2 for Windows 95, 98 and NT 4.0

Part No. A53949-04

Contributors: Richard Bettelheim, Bernie Cohen, John Dowden, Brice Dunwoodie, Gordon Furbush, Dana Izenson, Lesley Kew, Martin McKendrick, Mason Kong, Mason Ng, Dirke Pranke, Matt Prather, Bettina Rafailovic-Dallas, James Steel, Denise Stone, Sean Trabosh, Shiu Wong, John Zussman.

Contents

3 Oracle Video Java Library

4 Oracle Video ActiveX Control

5 Customizing, Deploying and Troubleshooting OVC

A Oracle Video Web Plug-in Reference

B Oracle Video Java Library Reference

C Oracle Video ActiveX Control Reference

D Oracle Video Session URLs

Send Us Your Comments

Oracle Video Client Developer's Guide, Release 3.2 for Windows 95, 98 and NT 4.0

Part No. A53949-04

Oracle Corporation welcomes your comments and suggestions on the quality and usefulness of this publication. Your input is an important part of the information used for revision.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where?
- Are the examples correct? Do you need more examples?
- What features did you like most about this manual?

If you find any errors or have any other suggestions for improvement, please indicate the product version number, book name, chapter, section, and page number (if available). You can send comments to us in the following ways:

- Electronic mail - omsdoc@us.oracle.com
- FAX - (650) 654-6209 Attn: Oracle Interactive Television Documentation Manager
- Postal service:
  Oracle Corporation
  Oracle Interactive Television Documentation Group
  500 Oracle Parkway, Mailstop 6OP5
  Redwood Shores, CA 94065
  USA

If you would like a reply, please give your name, address, and telephone number below.

If you have problems with the software, please contact your local Oracle Support Services.

Preface

The Oracle Video Client software enables you to develop interactive, video-based multimedia applications, such as computer-based training (CBT), interactive kiosks, and movies-on-demand. Oracle Video Client applications can receive and display streamed MPEG (Motion Picture Experts Group) video, MPEG audio, and OSF (Oracle Streaming Format) files from the Oracle Video Server (OVS) system. Other formats can be converted to OSF and streamed also, including WAV and AVI files.

> **Note:** The Oracle Video Client can play both video and audio. For simplicity, this manual usually refers only to video, since this includes both visual and auditory elements, but all information applies equally to audio, unless otherwise noted.

The Oracle Video Server allows many different clients to access media files concurrently. Multiple users can even receive streaming video from different segments of the same media file at the same time. The Oracle Video Client software provides the following tools to help you build media applications:

- The Oracle Video ActiveX Control enables Windows 95, 98, and NT 4.0, and Internet multimedia applications to handle streams from the Oracle Video Server. This control is designed to work with any application that hosts ActiveX controls; it has been tested with:
  - Microsoft Visual Basic 5.0 and 6.0
  - Microsoft Visual C++ 5.0 and 6.0
  - Internet Explorer 3.0 and all later versions
- The Oracle Video Web Plug-in is a Netscape-compatible plug-in that enables Web-based applications to handle streams from the Oracle Video Server. Using

Netscape's LiveConnect interface, you can use JavaScript or Java to dynamically control the Oracle Video Web Plug-in.

- The Oracle Video Java Library is a set of Java 1.1 classes and interfaces that enable Java applications to handle streams from the Oracle Video Server. The library also includes support classes for things like handling player events and querying a video server for a list of available content.

The Oracle Video Client package also includes the Iterated Systems ClearVideo (fractal) video and Voxware MetaSound audio encoder / decoders (also known as *codecs*). These codecs are suitable for low-bitrate streaming, enabling the Oracle Video Client to receive and display video and audio at low bit rates.

The *Oracle Video Client Developer's Guide* provides the information necessary to develop applications with the Oracle Video Client software, specifically:

- **Conceptual information:** Descriptions of the available client interfaces that you can use to create your own Oracle Video Client applications, as well as the infrastructure that enables the client interfaces to access the Oracle Video Server
- **Step-by-step tutorials**: Instructions for developing applications with Microsoft Visual Basic 5.0, Internet applications, and Java applications that stream video locally and from the Oracle Video Server
- **Developer reference:** Descriptions and code examples for the methods and properties associated with the Oracle Video ActiveX Control, the Oracle Video Web Plug-in, and the Oracle Video Java Library

You can find installation and configuration information for the Oracle Video Client, including system requirements and software compatibility requirements, in the booklet that came inserted with your Oracle Video Client installation CD-ROM. This information is also available in electronic form in the file **ovcinstl.pdf** in the **\docs** directory of your installation CD-ROM.

Intended Audience

This manual is for multimedia developers or anyone with the appropriate experience who wants to create stand-alone or Web-based applications that use streaming audio or video from the Oracle Video Server.

How This Manual Is Organized

This manual is divided into the following chapters:

Chapter 1, "Introducing the Oracle Video Client" contains:

- Descriptions of the Oracle Video Client system and its components
- High-level information on the various client interfaces available
- Information on the development and deployment process for creating your own custom video clients

Chapter 2, "Oracle Video Web Plug-in" contains:

- Step-by-step procedures for displaying and controlling media files from the Oracle Video Server in HTML pages, using the Netscape-compatible plug-in client interface
- Sample applications using Javascript and Java to control the plug-in

Chapter 3, "Oracle Video Java Library" contains:

- Information on creating Java applications that can display and control streaming video and audio from the Oracle Video Server
- A sample Java application that loads a media file and controls playback

Chapter 4, "Oracle Video ActiveX Control" provides:

- Procedures for embedding the control in HTML pages for ActiveX-enabled browsers and scripting the control through VBScript and JScript
- Steps for building applications with the Oracle Video ActiveX Control using Microsoft Visual C++ 5.0 and Microsoft Visual Basic 5.0

Chapter 5, "Customizing, Deploying and Troubleshooting OVC" provides:

- Steps for deploying OVC to end users and for customizing OVC for that deployment
- Troubleshooting for deploying OVC

Appendix A, "Oracle Video Web Plug-in Reference" contains reference information for the Oracle Video Web Plug-in interface.

Appendix B, "Oracle Video Java Library Reference" contains reference information for the Oracle Video Java Library interface.

Appendix C, "Oracle Video ActiveX Control Reference" contains reference information for the Oracle Video ActiveX Control interface.

Appendix D, "Oracle Video Session URLs" describes how to request media resources to play on the Oracle Video Client.

Related Publications

See *Getting Started with Oracle Video Server* for related publications.

Conventions Used in This Manual

This guide uses particular notational conventions to clarify syntax, enhance visual access to pages, and otherwise distinguish elements, such as code examples and keystrokes the user should enter from the text provided as background for those examples. These conventions are listed here:

Feature	Example	Explanation
boldface	**mna.h**	Identifies file names.
	timeout	Identifies method arguments.
boldface, trailing parenthesis	**mzsInit()**	Identifies method names.
italics	*container1*	Identifies a place holder in command or function-call syntax; replace with a value or string of your own.
ellipses	n, ...	Indicates that the preceding item can be repeated any number of times.

Conventions for Examples

This guide shows command and code examples in a monospace Courier font:

```
file://C:\orawin\ovc\demo\www\client\test.htm
```

For examples that are longer than a single line, a backslash (\) appears at the end of a line to indicate the line continues on the next:

```
<embed name="Video1" width=400 height=400 type="application/oracle-video"\
mediafile="/mds:/mds/video_archive/oracle2.mpg">
```

Do not enter the backslash if you type out this example. It is used only as a typographic convention.

The installation application installs Oracle files in different areas on different client platforms. For this reason, file paths in this document are relative to the location indicated by the environment variable **ORACLE_HOME**, which represents the directory where you installed the Oracle Video Client software. You can find the default **ORACLE_HOME** directories for various platforms in the table below.

Platform	Directory
Windows 95	C:\ORAWIN95
Windows 98	C:\ORAWIN98
Windows NT 4.0	C:\ORANT

Oracle Support Services

When you or someone in your company acquired this Oracle product, you probably also purchased some level of customer support. Oracle then sent you a package that includes telephone numbers, email addresses, and Web sites you should use to contact customer support.

Oracle provides Web-based support for our Oracle*MetaLink* and Oracle*Mercury* services at **http://www.oracle.com**/**support**. If your company did not purchase customer support, you can still visit **http://www.oracle.com/support** to find out about Oracle Support Services.

1

Introducing the Oracle Video Client

The Oracle Video Client (OVC) provides several ways to incorporate streaming video and audio from the Oracle Video Server (OVS) into your own applications. OVC provides easy-to-use client interfaces that you can embed in your applications to make video and audio available to your users. The interfaces include:

- Oracle Video Web Plug-in, a Netscape-compatible plug-in
- Oracle Video ActiveX Control, an ActiveX control
- Oracle Video Java Library, a set of Java classes

These interfaces handle selecting a file to load, video display, and user control of playback by passing client requests to the video server through a software layer. This software layer handles the technical aspects of communicating with the server, controlling the real-time stream and audio-video playback. By handling the technical aspects of the process and providing many different client interfaces around which you can create your own client applications, the Oracle Video Client provides a flexible and portable client development tool for creating client applications that can run on many different environments.

> **Note:** The Oracle Video Client is a Windows-based application, and it runs only on Windows 95, 98 and Windows NT.

This introduction contains the following sections:

- Client Interfaces discusses the different client interfaces, including their individual advantages and attributes, to help you decide which interface to use for your own client applications
- Client Software Components describes the OVC software architecture

- Developing and Deploying a Client Application discusses the development and deployment process for creating your own video clients

Client Interfaces

Note: It is important to understand the multiple meanings of client as used in the OVC / OVS model:

- The first level of client is that of enterprise client: somewhere on your network, there is a video server that can be accessed by users seated at remote workstations. These workstations are clients of the OVS system and are termed "client machines."
- The client machine contains the Oracle Video Client (OVC) software. This client consists of two parts: the *Oracle Video Interface* (OVI) abstraction layer and the *client interfaces*, such as the Oracle Video Web Plug-in, Oracle Video Java Library, and Oracle Video ActiveX Control.
- Client machines running OVC can be PCs, set-top boxes or video kiosks, but OVC only runs on a machine running either the Windows 95, 98 or NT 4.0 operating system.
- OVI is a client of the OVS system and the client interfaces can themselves be considered clients of OVI.
- You can create your own client applications through a combination of client interfaces, scripting commands, and your own applications that use the OVC client interfaces.

The main components of the Oracle Video Client are the client interfaces. These interfaces use the Oracle Video Interface (OVI), described in "Client Software Components" on page 1-6, to access streaming video from the Oracle Video Server. The separation of the client interfaces from the basic streaming functionality supported by OVI means that you can harness the functionality of the Oracle Video Server from a variety of platforms, without worrying about the underlying mechanisms that handle the low-level tasks.

This section describes the interfaces that OVC provides, and discusses factors that can affect your decision as to which of the client interfaces you want to use to create your own custom video clients.

Interface Descriptions

OVC provides the following client interfaces:

- Oracle Video Web Plug-in
- Oracle Video Java Library
- Oracle Video ActiveX Control

Oracle Video Web Plug-in

Use the Oracle Video Web Plug-in to create HTML documents containing streaming audio and video for Netscape-compatible browsers. You can customize the behavior of the plug-in at load time through the attributes to the **<embed>** tag used to include the video on your Web page. The plug-in also provides a LiveConnect interface that allows you to create controls on your page that can manage the plug-in after load time, through JavaScript or Java.

You can find usage information on the Oracle Video Web Plug-in in Chapter 2, "Oracle Video Web Plug-in". You can find reference information for the Oracle Video Web Plug-in in Appendix A, "Oracle Video Web Plug-in Reference".

Oracle Video Java Library

The Oracle Video Java Library consists of Java classes and interfaces that enable you to create Java applications that can play streaming audio and video, combining audio and video with any of your other needs.

Due to browser security restrictions, browser-embeddable applets created with the Oracle Video Java Library may not function in all Java-enabled browsers. The library includes an embeddable applet, **PlayerApplet**, which has the same restrictions.

You can find usage information on the Oracle Video Java Library in Chapter 3, "Oracle Video Java Library". You can find reference information in Appendix B, "Oracle Video Java Library Reference".

Oracle Video ActiveX Control

You can embed the Oracle Video ActiveX Control into most applications that support ActiveX control embedding. For example, you can use it in an HTML document for Microsoft Internet Explorer or embed it in a Visual Basic application. You can manage the control using scripting languages, such as VBScript and JScript.

You can find usage information on the Oracle Video ActiveX Control in Chapter 4, "Oracle Video ActiveX Control". You can find reference information for the Oracle Video Web Plug-in in Appendix C, "Oracle Video ActiveX Control Reference".

Choosing a Client Interface

To a large extent, the primary target platform and preferred application environment for your client application determines which client interface you should choose. You also need to consider *how* you want to use the video client. There are two main types of clients you can develop:

- Browser-Hosted Client Applications
- Stand-Alone Client Applications

Browser-Hosted Client Applications

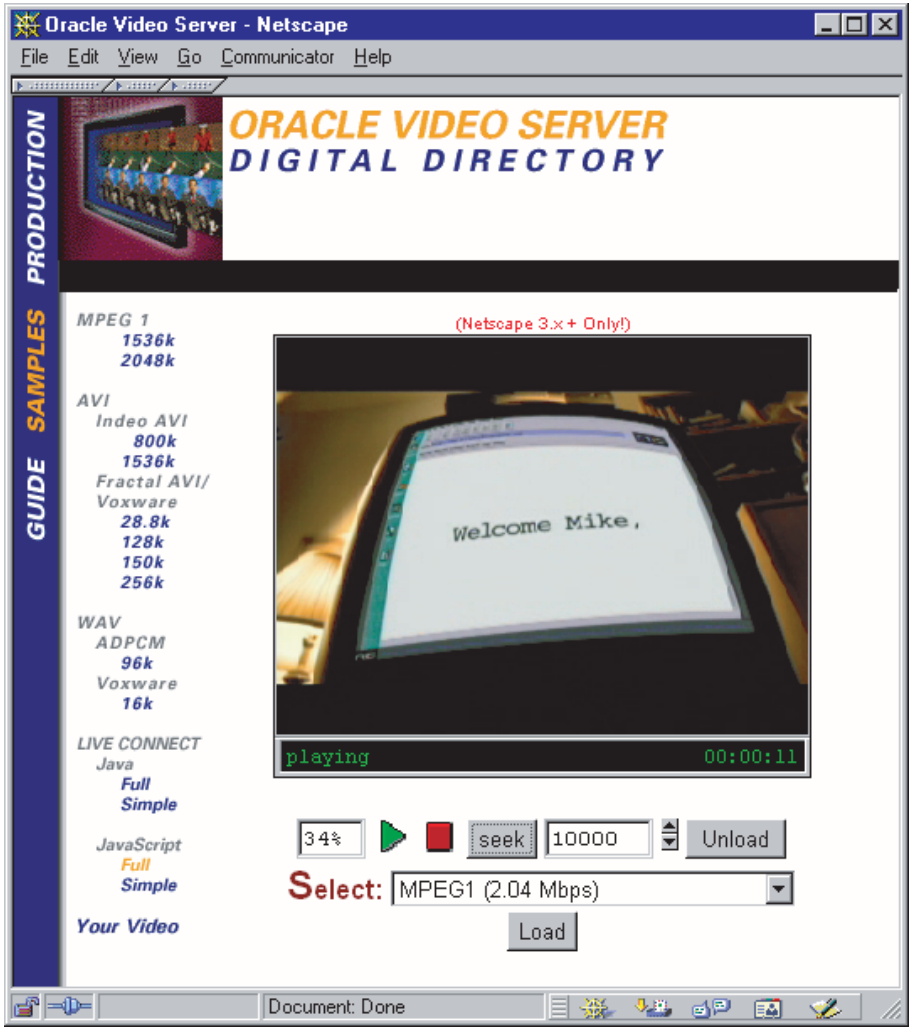
Browser-hosted clients generally consist of an HTML page (static or dynamic) with the plug-in or ActiveX control embedded in the page:

- If your target browser is Netscape Navigator or Communicator, use the Oracle Video Web Plug-in.
- If your target browser is Microsoft Internet Explorer, use the Oracle Video ActiveX Control. Internet Explorer 3.0 and later supports Netscape plug-ins, but not plug-in scripting.

You can also add scripting to an HTML document. In Netscape browsers, use either JavaScript or Java. In Microsoft Internet Explorer, use VBScript or JScript. Scripts can be invoked based on normal events, such as mouse-over, click, and focus events. You can use scripts to create custom interfaces that follow your look-and-feel standards.

Figure 1–1 shows an example of a browser-hosted client.

Figure 1–1 Example of a Browser-Hosted Client



The controls shown in Figure 1–1 use simple Javascript methods to select, load, unload, start, stop and seek within the video.

Stand-Alone Client Applications

There are two ways to create stand-alone applications using OVC:

- Using an ActiveX-enabled development tool, such as Visual Basic, add the Oracle Video ActiveX Control to the development tool's control palette or equivalent. You can then use the client control in your applications the same as any other custom control.
- Use the Oracle Video Java Library in a Java application. The Java classes provide full control over the client and easy programmatic access to all the features of the client.

Your choice depends on the tools you prefer and what you are creating. If you are developing a Visual Basic or Visual C++ application, you must use the OVC ActiveX control. If you are developing for the Internet and know that your end users will be running Netscape browsers, use the Java Libraries to create the application.

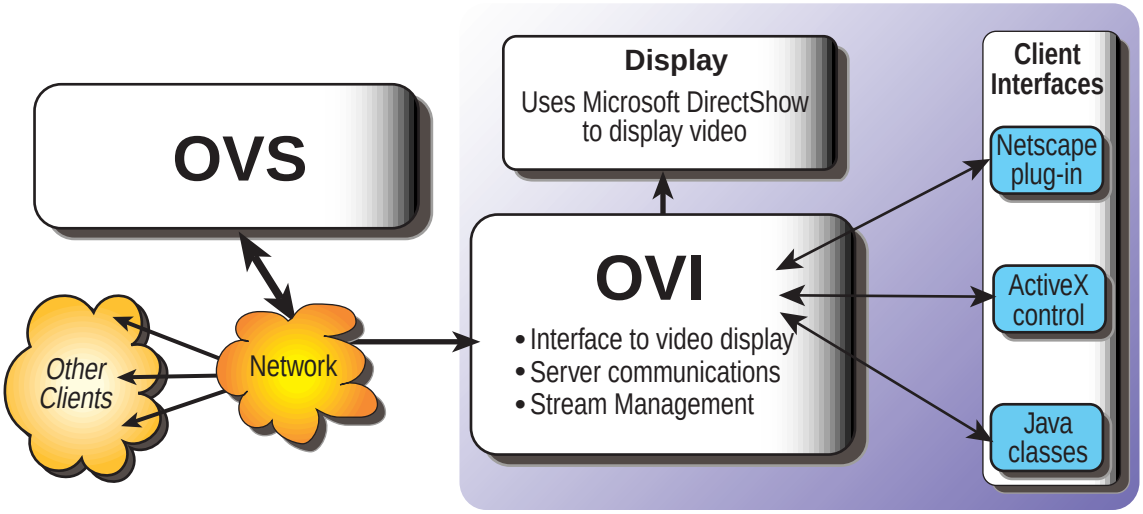
Client Software Components

This section discusses the components that constitute the Oracle Video Server system, from the video server to the viewing client. The system has three parts:

- The Oracle Video Server (OVS) provides streaming video and audio, as well as a number of supporting services, to the client machine.
- The Oracle Video Interface (OVI), running on the client, manages the streams from OVS, handles messages and transactions to and from OVS, and controls the display technology.
- The Oracle Video Client (OVC) client interfaces provide user control of streams, set up the display area, and handle transactions with the client environment.

Figure 1–2 shows the various components in the system.

Figure 1–2 Software components of the Oracle Video Client



OVS is the backbone of this system. It stores, retrieves, and dispatches media content files throughout the system. You can find out more about OVS in *Inside Oracle Video Server*.

OVI is the video client's gateway to the OVS system. It provides the following services:

- communicates with the client interface, passing requests from the interface on to the video server
- handles network communications between the client machine and the video server, including managing video and audio streams as they arrive from the video server
- communicates with the playback and display technology to play back the incoming stream using standard display technologies such as Microsoft DirectShow

Developing and Deploying a Client Application

This section briefly discusses the process of developing your own client application, as well as how to make your new client available to users. For more specific information on deploying your application, see Chapter 5, "Customizing, Deploying and Troubleshooting OVC".

There are a number of things to consider when developing your own client application:

1. Decide which client interface you want to use. See "Choosing a Client Interface" on page 1-4.
2. Decide how much functionality you want to make available to your users. Each client interface provides control over the stream playback, but you can specify how much of the provided functionality to make available to the user.

   For example, suppose you want to use the ActiveX control. This control provides the ability to play, stop, rewind, fast forward, and pause the video. But you're creating a kiosk application, where you don't want your end users to be able to interrupt the video. You can turn the controls off, so that the user can't manipulate the video at all, or add only volume controls so that the user can change the volume, but not interrupt the video.
3. Before your end users can use your new client application, they must install the minimum Oracle Video Client installation. This includes OVI, as well as whatever client interfaces your application requires. Make the OVC installation available to users, for example, through a Web page or shared network resource. You then need to give instructions to your users on how to install OVC, such as which client interfaces they require.

   You can also install OVC as part of the installation process for your client application. To find out whether the proper version of OVC is already installed on the user's system, check the Windows registry for the Oracle Video Client version. This key is located in:

   ```
   HKEY_LOCAL_MACHINE\SOFTWARE\Oracle\Oracle Video Client
   ```

There are further considerations that depend on the client interface you choose:

- If you are creating Web page applications using the Oracle Video Web Plug-in or the Oracle Video ActiveX Control, the "applications" are really HTML documents. The documents embed the plug-in or control, but don't require the user to download the client interface, since it has been installed along with OVC.

The only exception is if you use a Java applet to control the plug-in through the LiveConnect interface. In that case, the user has to download the Java applet. This works just like downloading any other Java applet.

- If you create a stand-alone application using the Oracle Video ActiveX Control or Oracle Video Java Library, you need to make the application available to your users, just as you make the OVC installation available to them.

2

Oracle Video Web Plug-in

This chapter describes the Oracle Video Web Plug-in. The Web Plug-in uses Multimedia Internet Message Extensions (MIME) to associate an file extension with an application; in this case, with the Oracle Video Client. Internet Explorer 4.0 and later and Netscape Navigator/Communicator support MIME types, so this plug-in can be used by Internet Explorer and Netscape Communicator.

The Oracle Video Web plug-in enables you to create Web pages and applications that contain streaming video and audio from Oracle Video Servers. You can use Java or JavaScript to control or script the plug-in in the Netscape browser. These scripting languages enable you to add controls and interfaces and tailor the behavior of OVC.

> **Note:** You cannot use JavaScript to control the plug-in in any version of Internet Explorer.

This chapter contains these sections:

- **Introduction to the Oracle Video Web Plug-in**
- **Requirements**
- **Embedding the Oracle Video Web Plug-in in an HTML Document**
- **Controlling the Plug-in Using JavaScript and Java**

If you install the Typical configuration or choose to install the sample applications in the Custom configuration, your client installation includes a sample that uses the Oracle Video Web Plug-in. Open the file **index.htm** in the directory **ovc\demo\webplugin** in your **ORACLE_HOME**.

You can find reference information about the Oracle Video Web Plug-in in "Oracle Video Web Plug-in Reference" on page A-1, including valid attributes for the **<embed>** HTML tag and methods available on the plug-in itself.

To get the most from this chapter, you should be familiar with HTML, JavaScript, and Java. Specifically, knowledge of the following would be helpful: embedding Netscape-compatible plug-ins in HTML documents, responding to basic events such as mouse clicks, and embedding applets in HTML documents.

Introduction to the Oracle Video Web Plug-in

The Oracle Video Web Plug-in provides a screen for video and audio playback, as well as optional controls and a status line. Using the plug-in is easy: place the Oracle Video Web Plug-in in your HTML document by adding an **<embed>** statement to an HTML document. Within the **<embed>** statement, you can set attributes to modify the appearance and behavior of the plug-in. For example, you can specify whether the controller appears or whether the video starts playing automatically as shown in this example.

Example 2–1 Sample Embed statement for an HTML document

```
<embed type="application/oracle-video" name="video1" width=356  height=244
controls="false" autoStart="true" keepAspectRatio="true"
mediafile="vsi://omsalpha4:5000/mds/video/oracle1.mpi?data_proto=tcp">
```

You can even make the plug-in completely invisible if you're using the plug-in for audio playback only (using the **hidden** attribute).

Within Netscape Navigator (version 3.0 or later) you can customize the plug-in's functionality by adding JavaScript or Java graphical user interfaces to dynamically control the plug-in. The LiveConnect interface provides access to the control from either language. Starting and stopping video, modifying the volume or stream position, and changing the current media resource are all possible.

After installing the Oracle Video Client, each time a user loads a page that uses the plug-in, the **<embed>** statement loads the Oracle Video Web Plug-in automatically. The plug-in appears in the browser as a video screen (unless hidden), with some optional components

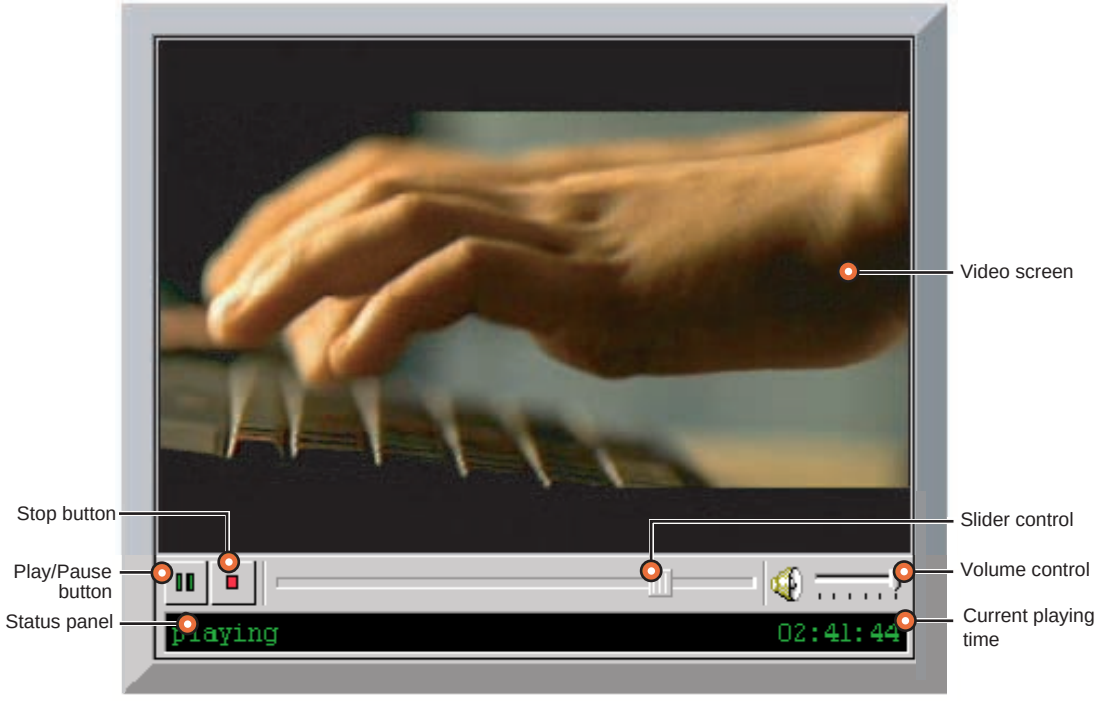
- Controller with:
  - Play / Pause button
  - Stream position slider

  - Volume slider
- Status line

Controls that you design using JavaScript or Java appear wherever you place them.

Depending on the attributes set in the **<embed>** statement, the plug-in can connect to the video server immediately upon being loaded and play a predetermined video. Or you can make the plug-in load, but not play until the user clicks the Play button. If you are using the built-in controls, the Play button turns into the Pause button when playback begins, and becomes the Play button again when playback stops. When the page is closed, the video stream is automatically deallocated and the plug-in is unloaded by the browser. You can also provide a graphical widget to select a video to play, using Javascript, Java or an ActiveX control.

Figure 2–1 Plug-in with controls and status line visible



The plug-in also offers other means of control:

- Click the video screen to start the video. To pause, click the video screen again.

- Right-click the plug-in to display a pop-up menu with the following options: **Play**, **Pause**, **Rewind to Start of Movie**, **Forward to End of Movie**, and more. Choose **Play** from the pop-up menu to start the video. Choose **Pause** to pause it.
- The LiveConnect interface lets you call stream control methods on the plug-in through JavaScript or Java.

Requirements

The Oracle Video Client must be installed on all machines on which you want to use the Oracle Video Web Plug-in. In addition to the standard client requirements, the Oracle Video Web Plug-in requires a browser that supports Netscape plug-ins, such as Netscape Navigator 3.0 or greater, or Microsoft Internet Explorer 3.0 or greater.

To control the plug-in with JavaScript or Java, the Oracle Video Web Plug-in requires a browser that supports the Netscape LiveConnect interface, such as Netscape Navigator 3.0 or greater.

Installing the Oracle Video Web Plug-in

To install the Oracle Video Web Plug-in from the OVC CD-ROM, run **Setup.exe** (located in the root directory), and:

- Select the **Typical** or **Compact** installation configuration, or
- Select the **Custom** installation configuration, making sure you select the Oracle Video Web Plug-in check box, when prompted.

When you elect to install the Oracle Video Web Plug-in, the installer automatically installs it if your browser is Netscape Navigator/Communicator or Internet Explorer 3.0 or later — browsers that support Netscape plug-ins. The plug-in file **npovc.dll** is installed in the default plug-in directory for each browser. When you restart your browser, the plug-in is configured and ready to use. If your browser was running during the installation, you need to close and re-open it.

You can also install the OVC Web-plug-in by running **\runtimes\ovces.exe** from the OVC CD-ROM and responding to the dialog boxes. This is the quickest way in install the plug-in.

You can find the complete installation instructions for the Oracle Video Client in the CD insert that came with your installation CD-ROM. This information is also available in electronic form in the file **ovcinstl.pdf** in the **\docs** directory of your installation CD-ROM and, if you selected the **Typical** installation configuration or

selected the **Docs** option in the **Custom** configuration, in the **ovc\docs** directory of your **ORACLE_HOME**.

Associating Oracle Video Files with the Oracle Video Web Plug-in

When you download a data file in a network application, such as your browser or e-mail program, you want to be able to handle the data properly without having to save it to your hard drive and manually start another application. For example, if you download a sound file, you don't want to have to save the file, start your sound player, load the file, then finally play the sound. You just want to hear the sound played over your speakers.

The problem is that your browser or e-mail program doesn't necessarily know what application to launch for a particular type of data. This is handled through MIME types (for Multimedia Internet Message Extensions), which provide a way of associating a particular type of data with a particular application.

There are three parts to a MIME type:

- **MIME type name.** This comes in the form *content-type*/*subtype*. *content-type* is a broad classification of the data, such as *image*, *application*, or *video*. *subtype* defines the type more specifically. For example, MIDI files are commonly given the MIME type **audio/midi**.
- **Associated file extensions.** In order to identify a file as having a particular MIME type, you either have to know that the file has that MIME type, such as through data packet headers, or find some way of extracting this from the information that you have about the file. Most browsers and HTTP servers use the file extension for this. Thus, each MIME type has one or more file extensions associated with it.
- **Associated application.** This specifies the application that can handle data of a particular MIME type.

Oracle video files are accessed through tag files, which have the extension **.mpi** (for more information on Oracle video files and Oracle video tag files, see the *Oracle Video Server Content Administrator's Guide*). The Oracle Video Web Plug-in requires that Oracle video files be associated with the plug-in through the MIME type **application/oracle-video**. When this MIME type is properly configured, you can access data files with the extension **.mpi**, which are associated with the Oracle Video Web Plug-in, through the MIME type.

There are two places you need to configure the Oracle Video MIME type:

- **Client MIME Configuration**

- **Server MIME Configuration**

Client MIME Configuration

Under normal circumstances, your browser automatically configures which MIME types it supports when it starts up. Netscape's plug-in specification requires that browsers check their plug-in directory to see which plug-ins are installed. The browser then queries each plug-in to find which MIME type or types the plug-in supports. Then, when the user tries to access data with a supported MIME type, the browser uses the appropriate plug-in to handle the data.

When you embed an Oracle video file in an HTML document, you need to specify the file's MIME type so the browser knows to load the Oracle Video Web Plug-in. There are two ways to do this:

- The **mediafile** and **src** attributes; **mediafile** indicates to the plug-in which media resource to play, while the **src** attribute, recognized by the browser, indicates a dummy file with the **.mpi** extension. The dummy file's name causes the browser to properly set the MIME type for the data and load the plug-in, while the plug-in finds the actual video session URL to load through the **mediafile** attribute.
- The **mediafile** and **type** attributes; **mediafile** indicates to the plug-in which file to open, while **type** explicitly specifies the MIME type of the embedded object.

> **Note:** The **type** attribute works in Netscape 3.0 or greater and Internet Explorer 4.0 or greater. For other browsers, use the **src** and **mediafile** attributes; see the discussions of these attributes starting on page 2-9.

Although only the extension **.mpi** is associated with the Oracle Video Client through the MIME type **application/oracle-video**, OVC can also play Oracle Stream Format (extension **.osf**) and regular MPEG files (extension **.mpg**). You can embed these files in HTML documents by using the **src** or **type** attribute to specify the MIME type, while the **mediafile** attribute tells the plug-in which video session URL to load. The extension of the media file does not affect the loading of the plug-in. See "Specifying the Video Session URL and MIME Type" on page 2-9 for an example of how to do this.

Server MIME Configuration

If you plan to serve video through HTML pages that embed the Oracle Video Web Plug-in, you need to configure your HTTP server to handle the

application/oracle-video MIME type. The HTTP server recognizes the MIME type of requested data by the file extension. It then returns the data with the appropriate MIME type specified in the HTTP packet header. The browser maintains its own list of MIME types, which includes the appropriate application or plug-in to load for supported MIME types. The browser loads the appropriate plug-in based on that information.

Configure your HTTP server to associate the extension **.mpi** with the **application/oracle-video** MIME type and the **.ovi** extension with the **application/oracle-video-registry** MIME type. Consult the documentation for your HTTP server for information on configuring MIME types.

Embedding the Oracle Video Web Plug-in in an HTML Document

To embed the plug-in into an HTML document, insert an **<embed>** statement with the appropriate attributes in your HTML code. The **<embed>** statement uses attributes to determine which video plays (the **mediafile** attribute), whether the video starts as soon as the plug-in loads (the **autoStart** attribute), whether the video replays every time it ends (the **loop** attribute), and more. See "<Embed> Attributes" on page A-1 for definitions of all of the available **<embed>** attributes.

This section contains the following topics:

- **Creating the <embed> Tag**
- **Specifying the Video Session URL and MIME Type**
- **Specifying Plug-in Characteristics**
- **Playing Audio-Only Streams**

Creating the <embed> Tag

The example below shows a simple HTML document with an **<embed>** tag.

```
<html>
<head>
<title>Oracle Video Web Plug-in Example</Title>
</head>

<body>

<embed type="application/oracle-video" name="video1" width=356 height=244
mediafile="vsi://omsalpha4:5000/mds:/mds/video/oracle1.mpi?data_proto=tcp">
```

```
</body>
</html>
```

where:

type="application/oracle-video"
Specifies the MIME type. In this case, it invokes the plug-in and prepares it to play an Oracle video file.

> **Note:** The **type** attribute only works in Netscape 3.0 or greater and Internet Explorer 4.0 or greater. For other browsers, use the **src** and **mediafile** attributes; see the discussions of these attributes starting on page 2-9.

name="video1"
Provides a name for the plug-in for JavaScript and Java calls. In this case, the plug-in is named **video1**. This enables you to call methods on the plug-in using the syntax **document.video1.*method*()**.

width=356
Specifies the plug-in width of 356 pixels. This value includes both the plug-in itself and the border which is always 2 on each side. Since the video content was encoded as 352 pixels wide, the width of the OVC Web Plug-in is 356.

height=244
Specifies the plug-in height of 240 pixels plus the 4-pixel border. The value for height includes both the plug-in itself, the border, and the optional controller and status line. In this case, no controller or statusline are displayed, so this represents the actual height of the plug-in plus the border. If the controller is included, add 25 pixels. If the status line is included, add 24 pixels.

Note: It is important to set the height and width properly for the OVC Web Plug-in. If the height and width are not set correctly, the video is size has to be recalculated on playback, and that seriously affects performance. A video that normally plays at 30 frames per second will probably only playback at 16 frames per second if the height and width are not set correctly.

mediafile="vsi://omsalpha4:5000/mds:/mds/video/oracle1.mpi?data_proto=tcp" Tells the plug-in to go to port 5000 on the server **omsalpha4** and return the tag file **oracle1.mpi** located in the MDS volume **video**. The resulting stream uses the TCP protocol. For a complete description of the syntax, see the **mediafile** attribute discussion in Appendix D, "Oracle Video Session URLs".

Specifying the Video Session URL and MIME Type

To embed an Oracle Video Server file into an HTML document, specify the actual video session URL to be loaded and played using the **mediafile** attribute of the **<embed>** tag. Browsers themselves don't recognize the **mediafile** attribute: they just pass it on to the plug-in, which uses it to locate the media resource. The browser still needs to identify and recognize the MIME type of the embedded item to load the proper plug-in for playback. Use the following methods to identify the desired video session URL and the MIME type of the file:

- **Type and Mediafile Attributes**
- **Src and Mediafile Attributes**

Type and Mediafile Attributes

The preferred way to specify the MIME type of the embedded media is to use the **type** attribute. The browser checks its list of registered MIME types and loads the appropriate plug-in for that type. The plug-in uses the **mediafile** attribute to load the appropriate video session URL.

For Oracle Video Server video session URLs, you should always specify the MIME type **application/oracle-video**. When you install the plug-in, the installer associates files with a MIME type of **application/oracle-video** with the Oracle Video Web Plug-in.

Example: This sample **<embed>** tag shows how to embed a video session URL from the server **omsalpha4** with the appropriate MIME type.

```
<embed type="application/oracle-video" width=356 height=244
```

```
mediafile="vsi://omsalpha4:5000/mds:/mds/video/oracle1.mpi?data_proto=tcp">
```

Note: The **type** attribute only works in Netscape 3.0 or greater and Internet Explorer 4.0 or greater. For other browsers, use the **src** and **mediafile** attributes; see the discussions of these attributes starting on page 2-9.

Src and Mediafile Attributes

The **src** attribute should be used for all browsers that don't support the **type** attribute (any browser other than Netscape Navigator 3.0 or greater or Internet Explorer 4.0 or greater). Browsers inspect the **src** attribute to determine the MIME type of the embedded file from the file's extension. This works like the **type** attribute: the browser associates **.mpi** files with the MIME type **application/oracle-video**, which is associated with the Oracle Video Web Plug-in.

However, there are a couple of things to be aware of when you're using the **src** attribute:

- The **src** attribute actually specifies a file name, instead of just a plain MIME type like the **type** attribute. This file does not have to be the same as the file specified by the **mediafile** attribute. It must, however, be a real file, specified by a valid URL or path, with the extension **.mpi**.

  The Oracle Video Web Plug-in sample pages handle this by setting the **src** attribute to point to the file **oracle.mpi**, which is installed in the same directory as the sample HTML pages.

- Some browsers can't handle "empty" or "dummy" files (files that are zero-length and contain no data) specified by the **src** attribute. This means that you should specify a file that contains some data of any type.

  Again, the Oracle Video Web Plug-in sample pages handle this with the **oracle.mpi** file, which contains 1K of data.

Example: This sample shows how to embed a video session URL using the **src** attribute to indicate the MIME type of the media file.

```
<embed src="muse.mpi" mediafile="c:\tmp\muse.mpg" width=356 height=244>
```

Notice that the **src** and **mediafile** attributes specify files with different names and even different extensions. The **.mpi** extension in the **src** attribute instructs the browser to load the Oracle Video Web Plug-in, while the **mediafile** attribute instructs the plug-in to load the file **c:\tmp\muse.mpg**.

Specifying Plug-in Characteristics

The Oracle Video Web Plug-in also recognizes a number of other attributes from the **<embed>** tag. Table 2–1 summarizes these attributes and their effects on the plug-in's look-and-feel and play characteristics. You can find more detailed descriptions of each of these attributes in Appendix A, "Oracle Video Web Plug-in Reference".

Table 2–1 <embed> tag attributes

Attribute	Values (default in bold)	Description	Req'd?
autoStart	true | **false**	Specifies whether video starts playing as soon as the plug-in is loaded.	
background	"*file*"	Specifies a background image.	
controls	true | **false**	Specifies whether the plug-in appears with or without controls.	
controlMask	**controller** [+statusline]	Selects which controls appear in the plug-in; requires the added statement **controls=true**.	
height	*nnnn*	Specifies the height of the plug-in in pixels or percentage. If you specify percentage, you must include the percent sign (%).	✔
hidden	true | **false**	Specifies whether the plug-in is displayed. Overrides **height** and **width** settings.	
keepAspectRatio	**true** | false	Specifies whether the plug-in will maintain the aspect ratio (the height and width ratio) for the content to be played, even if the Client is resized.	
leftClick	**true** | false	Specifies whether left clicking in the video screen toggles Play and Pause.	
loop	true | **false**	Specifies whether the stream plays continually, returning to the **playFrom** position when it reaches the **playTo** position and continuing playback.	

Table 2–1 <embed> tag attributes (Cont.)

Attribute	Values (default in bold)	Description	Req'd?
mediafile	*mediafile_url*	Specifies the protocol, server, and video session URL to play. **Note:** For information on how the **mediafile**, **src**, and **type** attributes work together, see "Specifying the Video Session URL and MIME Type" on page 2-9. For information on video session URLs, see Appendix D, "Oracle Video Session URLs".	✔
name	*plugin_name*	Specifies a local name for the plug-in. **Note:** This parameter is required if you call the plug-in from a JavaScript function or Java applet embedded in the same HTML page as the plug-in.	✔ (See note)
playFrom	**default** | beginning | **end** | *wallclock* | | *hh:mm:ss:cc* | *millisecss*	Starts play at the specified stream position. If the **loop** attribute is **true**, then playback loops to the **playFrom** point. **Note:** "cc" is hundreths of a second.	
playTo	beginning | **end** | *wallclock* | | *hh:mm:ss:cc* | *millisecs*	Stops play at the specified stream position. If the **loop** attribute is **true**, then playback loops when it reaches the **playTo** position. **Note:** "cc" is hundreths of a second.	
popupMenu	**true** | false	Specifies whether the pop-up menu appears when the right mouse button is clicked on the plug-in. This menu allows basic operations like play and pause, rewind, forward, and close.	
skipIncrement	1 to 100 (percent) **10**	Specifies the amount the client skips backward or forward when you press the left or right arrow keys when the client is in full-screen mode. The option, skipIncrement, can be set as a percentage of the length of the video clip. Valid values are **0** to **100**. The default value for SkipIncrement is **10**.	

Table 2–1 <embed> tag attributes (Cont.)

Attribute	Values (default in bold)	Description	Req'd?
sliderRate	*nnnn*	Specifies the increment in milliseconds by which you can adjust the plug-in's seek slider. The default is 1000.	
src	*file.extension*	Specifies the source file and MIME type of the requested media file. Should be used in conjunction with **mediafile**. **Note:** If you don't use **src**, you must use **type**. See "Specifying the Video Session URL and MIME Type" on page 2-9 for more information on **type**, **src**, and **mediafile**.	✔ (See note)
toolTips	true | **false**	Specifies whether tool tips show up when the user moves the mouse over the plug-in.	
type	application / oracle-video	Specifies the video session URL MIME type. The only value you should specify for **type** attribute is **application/oracle-video**, as shown. Should be used in conjunction with **mediafile**. **Note:** If you don't use **type**, you must use src. See "Specifying the Video Session URL and MIME Type" on page 2-9 for more information on **mediafile**, **src**, and **type**.	✔ (See note)
volume	*vol*	Specifies a volume level from 0 (inaudible) to 100 as a percentage.	
width	*nnnn*	Specifies the plug-in's width in pixels or percent, just like height.	✔

Playing Audio-Only Streams

You can run the plug-in using audio streams without video, with or without controls:

- If you don't need the controller or status line, you can hide the plug-in by specifying the **hidden** plug-in tag or by specifying a width and height of 0. In this case, you can only control the stream through JavaScript or Java calls.

- To display the controller, add 25 pixels to the **height** attribute. The **width** attribute should be set to at least 144 pixels to allow some slider range. Using the default controls, the stream is controlled by a Play / Pause button and the seek slider. The volume is set by the pop-up volume slider or by setting the **volume** attribute.
- If you need the status line, add an additional 24 pixels to the **height** attribute.

Controlling the Plug-in Using JavaScript and Java

You can dynamically control the Oracle Video Web Plug-in using JavaScript or Java through the Netscape LiveConnect interface. For example, you can have buttons or icons that play the current stream, pause, seek, pop-up lists of available movies, and any other type of custom control or function that you want to develop. You can see an example of this in Figure 2–2.

Note: Because you need to use Netscape's LiveConnect interface to control plug-ins through Java or JavaScript, you must use a browser that supports LiveConnect. This means Netscape Navigator version 3.0 or later. To script with Internet Explorer, use the Oracle Video Client ActiveX Control.

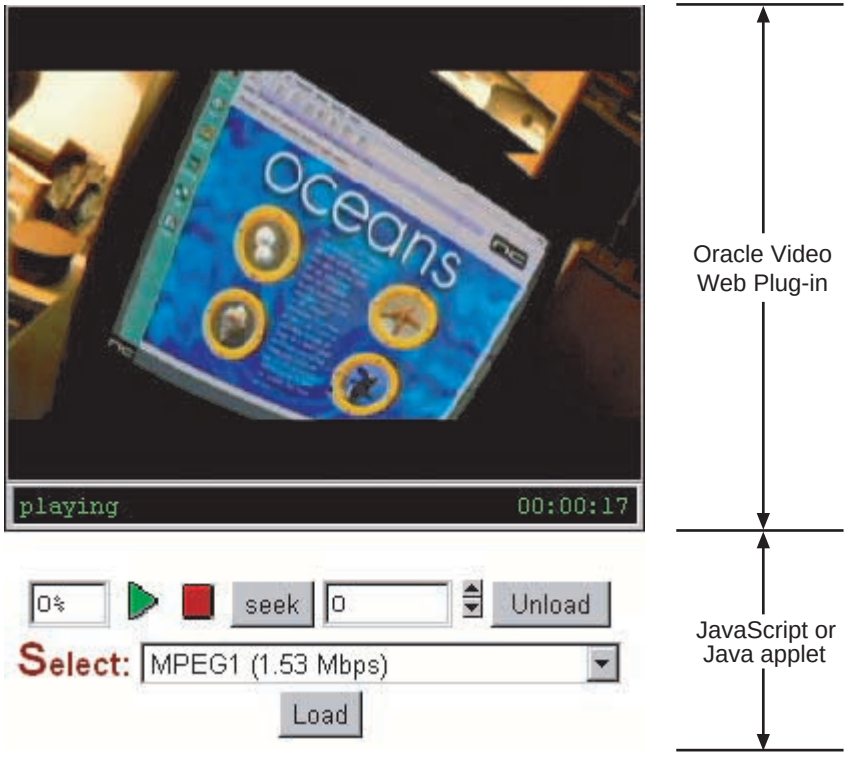
This section contains the following topics:

You can also examine the sample code files that were installed along with the Oracle Video Client. The directories are in your **ORACLE_HOME**:

- JavaScript: **ovc\demo\webplugin\samples\liveconnect\javascript**
- Java: **ovc\demo\webplugin\samples\liveconnect\java**

Run these sample applets and applications to better understand how plug-ins work. Feel free to modify and re-use these code examples in your Oracle Video Client custom applets and applications. You can find a reference to all of the methods available on the Oracle Video Web Plug-in in "JavaScript Methods" on page A-10.

Figure 2–2 One way to customize a plug-in using JavaScript or Java



Controlling the Plug-in with JavaScript

This section describes a number of techniques you can use to control and customize the Oracle Video Web Plug-in, including:

- **Naming an Embedded Plug-in**
- **Accessing Plug-in Methods and Properties**
- **Controlling the Plug-in with Form Buttons**
- **Using Graphical Controls**
- **Controlling the Plug-in with Dynamic Parameters**
- **Creating a Pop-up List**
- **Using an Image Map**

- **Other Things You Can Do with JavaScript**
- **Sample Code for JavaScript-Controlled Plug-in**

Naming an Embedded Plug-in

To give JavaScript a handle on the plug-in, specify a value for the **name** attribute in the **<embed>** statement:

```
<embed name="video1" width=356 height=244 type="application/oracle-video"
mediafile="vsi:///mds/video/oracle1.mpi">
```

The name you specify should be unique within the HTML document. This includes other named items, such as images, tables, forms, and so on. This is the name you'll use whenever you want to address a property or method of the plug-in.

Once you've named your plug-in, you can access it through the **document** object from anywhere within your document by that name. For example, if you want to call the **play()** method for the plug-in named **video1**, use something like this:

```
document.video1.play();
```

Accessing Plug-in Methods and Properties

You can access the Oracle Video Web Plug-in in several ways using JavaScript. The easiest is to load the HTML file that contains the **<embed>** statement, enter a JavaScript statement as a URL in the **Location** box of Netscape Navigator (located right above the main browser window), then press **Enter**.

To start playing the video specified in the example above, you would enter the following statement in the **Location** box:

```
javascript:document.video1.play()
```

To pause the video using the **Location** box, enter:

```
javascript:document.video1.pause()
```

Controlling the Plug-in with Form Buttons

Entering commands in the **Location** box has limited value. Using JavaScript calls to form elements—such as buttons, selection lists, or edit boxes—is more useful.

For example, you can create a form button in your HTML page that calls the **play()** method. The following example creates a button labelled **Play**. When you click this

button, the browser searches for an item in the document whose **name** attribute value is **video1**. Once it finds the video, it calls the plug-in's **play()** method, which instructs the plug-in to play the movie specified in the **mediafile** attribute of the **<embed>** statement.

```
<embed name="video1" width=356 height=244 type="application/oracle-video"
 mediafile="vsi:///mds/video/oracle1.mpi">

 <form name="form1">
 <input type="button" value="play" onClick="document.video1.play()">
 </form>
```

Using Graphical Controls

Play icon

You can let users control the plug-in using simple form buttons, but it's more "Web-like" to use a graphical icon to control the video. This example plays the video when the user clicks the **/images/play.gif** image.

```
<embed name="video1"
         width=356
         height=244
         type="application/oracle-video"
         mediafile="vsi:///mds:/mds/video/oracle1.mpi">

 <a href="javascript:document.video1.play();">
 <img src="/images/play.gif" width=24 height=24 border=0></A>
```

Controlling the Plug-in with Dynamic Parameters

You can extend the concepts introduced in the previous sections and create an input field, which can contain numeric or textual data, in the form. In this example, the user can type a number representing a position in the stream into the input field. When the user clicks the button, the **setPos()** method sets the position in the movie to the value entered in the input field.

```
<embed name="video1" width=356 height=244 type="application/oracle-video"
 mediafile="vsi:///mds/video/oracle1.mpi">

 <form name="form1">
 <input name="editSeek" type="text" value=0 size=10>
 <input type="button" value="Seek"
    onClick="document.video1.setPos(parseInt(form.editSeek.value))">
 </form>
```

Be sure to validate the user's input. For the sake of clarity, this example contains no validation code. In a deployment environment, however, you should make sure that the values that users input are valid.

Creating a Pop-up List

When creating custom controls or interfaces using JavaScript and the Oracle Video Web Plug-in, you can use all of the techniques that are normally available to you. The following code allows the user to select a movie from a selection list. When the user selects a new movie from the list, the plug-in loads the movie and starts playing it. This also creates a **Close** button, allowing the user to turn off a currently playing movie.

```
<html><head>
 <title>Pop-up Test</title>

 <script language="JavaScript">
 function playSel()
 {
    // get a handle on the pop-up list
    var mlist = document.forms[0].movielist;

    // get the name of the movie to load
    var movie = mlist.options[mlist.selectedIndex].value;

    if (movie != "")         // if movie is not empty
    {
       document.video1.load(movie); // load the selected movie
       document.video1.play(); // play the selected movie
    }
 }
 </script>
 </head>

 <body>
 <embed name="video1" width=356 height=244 type="application/oracle-video"
 mediafile="vsi:///mds:/mds/video/oracle1.mpi">

 <form name="form1">
 <select name="movielist" onChange="playSel()">
    <option value="">Default
    <option value="/mds/video/oracle.mpi">Demo
    <option value="/mds/video/oracle1.mpi">Demo 1
    <option value="/mds/video/oracle2.mpi">Demo 2
    <option value="/mds/video/oracle3.mpi">Demo 3
```

```
</select>
<input type=button value="Close" onClick="document.video1.unload()">
</form>
</body></html>
```

Using an Image Map

Using client-side image maps can greatly improve the appearance and functionality of a Web page. Image maps give the greatest amount of flexibility possible in designing your client interface. They allow you to control the appearance, overall interface metaphor, and user environment. By associating hot spots in the image map to methods and properties of the Oracle Video Web Plug-in, you can produce a sleek, professional interface that still offers the full functionality of the Oracle Video Client.

Spin control

This example shows how to create a spinner that increases or decreases the value in the **editSeek** field in one-second increments.

```
<script language="JavaScript">
 function spinUp()
 {
    // get the current value of the editSeek field
    var seekVal = parseInt(document.forms[0].editSeek.value);

    // add one second (1000 ms) to the value of the editSeek field
    document.forms[0].editSeek.value = seekVal + 1000;
 }

 function spinDown()
 {
    // get the current value of the editSeek field
    var seekVal = parseInt(document.forms[0].editSeek.value);

    // make sure you don't create a negative number (1000 ms = 1 second)
    if (seekVal < 1000) seekVal = 1000;

    // subtract one second (1000 ms) from the value of the editSeek field
    document.forms[0].editSeek.value = seekVal - 1000;
 }
 </script>

 <!-- Create the image map -->
 <img border=0 src=/images/spin.gif usemap="#spinmap">
 <map name="spinmap">
```

```
<!-- Associate the up arrow in the image with spinUp() -->
<area shape="rect" coords="0,0,11,10" href="javascript:spinUp()">

<!-- Associate the down arrow in the image with spinDown() -->
<area shape="rect" coords="0,10,11,20" href="javascript:spinDown()">
</map>
```

Other Things You Can Do with JavaScript

The examples above are just a few of the things you can do with JavaScript. Some other ideas:

- Create a single image that contains all of the typical VCR controls. Use a client-side image map to correlate mouse click locations with **play()**, **pause()**, and other plug-in methods.
- Create a client-side image map with various hot spots. Map each hot spot to a different seek point (or even a completely different stream) so that when the user clicks on a location, the video changes. For instance, you could create a map of the United States displaying the hometowns of each of the teams in the National Football League. Users click a city to play a movie about that city's home team.
- Create a custom video stream position scrollbar. Get an image you would like to use as a scrollbar. Use a client-side image map to split it up into many sub-regions, each of which seeks to a slightly different position in the stream.
- Create a function that checks stream position and updates a text field or graphic, or changes a document in a frame at various key points in the stream.

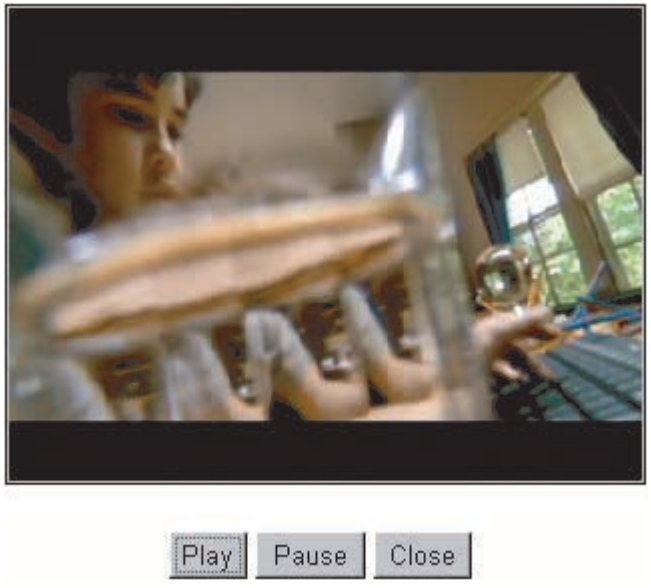
Sample Code for JavaScript-Controlled Plug-in

This section shows a very simple sample HTML file that uses JavaScript to let the user control the plug-in. The form buttons below the plug-in let the user play and pause the currently loaded video, or unload the video entirely.

You can find the source code for this example in the file **defvid.htm** in your **ORACLE_HOME** in the directory:

ovc\demo\webplugin\samples\liveconnect\javascript\simple\sample1

Figure 2–3 SamplePlug-in using JavaScript




```
<html><head>
 <title>Oracle Video Web Plug-in Simple JavaScript Demo</title>
 <script language="JavaScript">
 function play() {
    // If player is currently in state realized --> call play
    if (document.OVSplugin.getState() == 4) {
        if (document.OVSplugin.play())
       return true;
    }
    // if player is in any other state --> call resume
    else {
       if (document.OVSplugin.resume())
       return true;
    }
    return false;
 }
 </script>
 </head>
 <body bgcolor="#FFFFFF">
 <center>
 <font size="1" face="Arial" color="Red">
```

```
(Netscape 3.0 or greater only!)
</font><p>

<table border=5 cellpadding=0 cellspacing=0 align=center valign=center>
<tr align="center" valign="middle">
   <td align="center" valign="middle">
   <!-- Embed the Oracle Video Client Web Plug-in  -->
   <embed
      name="OVSplugin"
      width=244
      height=356
      type="application/oracle-video"
      autoStart="false"
      controls="false"
      loop=true
      leftClick=true
      toolTips=false
      popupMenu=true
      volume=100
      mediafile="vsi:///mds/video/ovs_mpg1_2048k.mpi?data_proto=tcp">
   </embed>

   <center>
   <form name=form1>
      <input type=button onclick="play()" value=play>
      <input type=button onclick="OVSplugin.pause()" value=pause>
      <input type=button onclick="document.OVSplugin.unload()" value=close>
   </form>

   </td>
</tr>
</table>
</body>
</html>
```

Controlling the Plug-in with Java

OVC provides two Java classes, **OviPlayer** and **OviObserver**, to allow interaction between the plug-in and your Java applets.

This section covers:

- **OviPlayer and OviObserver**
- **Retrieving the OviPlayer Object**

OviPlayer and OviObserver

OVC provides a Java class and an interface that let you control and observe the Oracle Video Web Plug-in from your applet:

- The **OviPlayer** class declares methods to access and control video. This class allows basic operations on a video like **load()**, **unload()**, **play()**, **pause()**, **getLength()**, and **getPos()**. The real implementation of these methods is contained in a section of native code contained in the plug-in.
- The **OviObserver** interface lets you take advantage of "observer" methods. These methods provide a callback mechanism that notifies you when the stream has advanced or reached the end of stream. You can then respond with some action that you define.

Table 2–2 and Table 2–3 give short descriptions of the methods available in the **OviPlayer** and **OviObserver** classes. You can find more detailed references of these classes in "OviPlayer" on page A-11 and "OviObserver" on page A-19.

Table 2–2 OviPlayer methods

Method	Description
advise()	Specifies the **OviObserver** object for this **OviPlayer**.
forward()	Forwards the stream to the end of the movie or the position indicated by the **playTo** property.
getCreateTime()	Returns the data and time the stream was created as an epoch.
getLength()	Gets the total length of the stream (in milliseconds).
getMaxPos()	Returns the maximum stream position (in milliseconds).
getMinPos()	Returns the minimum stream position (in milliseconds).
getObserver()	Returns the **OviObserver** object associated with this object, if any.
getPos()	Gets the current stream position (in milliseconds).

Table 2–2 OviPlayer methods (Cont.)

Method	Description
getState()	Returns the current state of the player.
getVol()	Gets the play volume (0 to 100).
load()	Loads the stream indicated by the video session URL contained in the **String** parameter.
pause()	Pauses the stream; stays at current position.
play()	Plays the currently loaded stream. There are two versions: ▪ **boolean play()** ▪ **boolean play(String playFrom, String playTo)**
resume()	Resume playing from the current stream position.
rewind()	Rewind the stream to the beginning of the movie or the position indicated by the **playFrom** property.
setAutoStart()	Tell the movie to automatically play when loaded. Takes a **boolean** parameter.
setFullScreen()	Puts the plug-in into full-screen mode.
setLoop()	If true, loop from beginning of stream (or **playFrom** if specified) when the end of stream (or **playTo**, if specified) is reached. Takes a **boolean** parameter.
setPopupMenu()	Activate or deactivate the right mouse button pop-up menu. Takes a **boolean** parameter.
setPos()	Seek to a specified stream position (in milliseconds).
setVol()	Set the play volume (range from 0 to 100).
stop()	Stop playing the stream and rewind to the beginning of the stream or the position indicated by the **playFrom** property.
unload()	Unload the current stream.

Table 2–3 OviObserver methods

Method	Description
onPositionChange()	Notify the applet when the movie changes position by an increment is set by the **sliderRate** attribute. If **sliderRate** isn't set, the default increment is 1,000 milliseconds (one second).
onStop()	Notify applet when end of stream is reached.

Retrieving the OviPlayer Object

You cannot simply create an **OviPlayer** object in your Java code and invoke the methods on the object, because the Java run-time engine can't resolve the references to the native code in the plug-in and produces unresolved link errors. Instead, retrieve the instance of the **OviPlayer** object used internally by the plug-in. To do this, you must import the **OviPlayer** class, as well as the **JSObject** and the **JSException** classes from the Netscape LiveConnect/Plug-in SDK. You can then follow these steps to get a valid **OviObject** instance that references the plug-in:

1. Retrieve the JavaScript context of the current HTML page using the **JSObject.getWindow()** method. The **getWindow()** method takes a single parameter, the window containing the applet.

   The **JSObject.getWindow()** method is static, so you don't actually need to create a **JSObject** object. The return value is a **JSObject**.

2. Call the **getMember()** method on the **JSObject** object returned in the last step.

   This method takes a **String** parameter, where the **String** specifies the name of the member you want. Use "document" as the parameter. This tells the **getMember()** method to retrieve the document object for the current page. The **getMember()** method returns a **JSObject**.

3. Call the **getMember()** method on the **JSObject** object returned in the last step, passing the plug-in name (from the **name** attribute in the **<embed>** tag) as the parameter to **getMember()**. Cast the return value to **OviPlayer**.

Once you've retrieved the **OviPlayer** object, you can use it just as you'd expect: you can load video session URLs with the **load()** method, play the current media file by calling **play()**, and so on.

```
// The following code gets a handle on the window in which
 // the applet (and plug-in) appear
 JSObject jsWindow = JSObject.getWindow(this);

 // the next line of code gets a handle on the HTML document
 // (web page) in which the applet (and plug-in) appear
 JSObject jsContext = jsWindow.getMember("document");

 OviPlayer playerInstance = (OviPlayer) jsContext.getMember("plugin_name")

 playerInstance.load("vsi:///mds/video/oracle1.mpi?data_proto=tcp");
 playerInstance.play();
```

Using OviObserver

The **OviObserver** interface is a Java interface that you implement in one of your own classes. To use **OviObserver** in your applet:

1. Implement the interface in one of your applet's classes.

   Here's a simple example:

   ```
   public class myApplet extends Applet implements OviObserver {
       // The rest of the class implementation goes here...
   }
   ```

2. Implement the **onStop()** and **onPositionChange()** methods of the **OviObserver** interface. Take whatever action you want when these events happen.

   Here's a simple example of an implementing class:

   ```
   class myApplet extends Applet implements OviObserver {
       // notify applet when end of stream reached and do something
       public void onStop() {
          System.out.println ("End of stream reached");
       }

    // notify applet when the stream position advances by one second
    // and do something, like keep track of were you are in the stream
       public void onPositionChange() {
          System.out.println ("The stream just advanced one second");
       }
    }
   ```

3. Retrieve an instance of the **OviPlayer** object, as described in "Retrieving the OviPlayer Object" on page 2-26.

4. Call the **OviPlayer**.**advise()** method, passing as a parameter the object that implements the **OviObserver** interface:

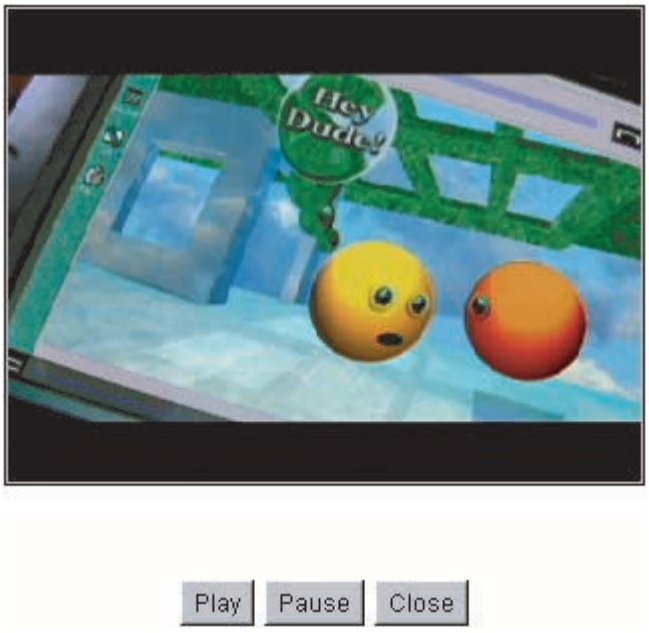
   ```
   // "this" is the object that implements the OviObserver interface.
    playerInstance.advise(this);
   ```

Once you've registered the observer with the **OviPlayer**, the **onStop()** and **onPositionChange()** methods are called whenever the appropriate events occur.

Simple Plug-in Example using Java

The following code example consists of two parts, the HTML code for the Web page and the code for the Java applet. The file containing the Java code, **simpleJava.java**, becomes **simpleJava.class** after compiling it.

Figure 2–4 Sample Plug-in using Java



HTML Code for Simple Plug-in Java Applet This shows the HTML code for a document that contains the Oracle Video Web Plug-in and a Java applet that controls it. You can find the sample source in the file **defvid.htm** in your **ORACLE_HOME** in the directory:

ovc\demo\webplugin\samples\liveconnect\java\simple\sample1

```
<html>
 <head>
 <title>Simple Java Applet Example of Plug-in Controls</title>
 </head>
 <body bgcolor="#FFFFFF">
 <center>
 <table border=1 cellspacing=2 cellpadding=2>
 <tr><td align="center" valign="middle" bgcolor="Gray">
```

```
<center>
 

<!-- Embed the Oracle Video Client Web Plug-in  -->
<embed
   name="OVSplugin"
   width=244
   height=356
   type="application/oracle-video"
   controls=false
   loop=false
   leftClick=true
   toolTips=false
   popupMenu=true
   volume=100
   mediafile="vsi:///mds/video/ovs_mpg1_2048k.mpi?data_proto=tcp">
</embed>
<p> 

<!--  ##############################################
                  IMPORTANT!
      ##############################################
-   The 'MAYSCRIPT="true"' flag in the applet tag is critical. This tells the
      browser's security manager to allow communication between the applet and
      the plug-in.
   -   The value of the applet parameter, pluginName, must agree with the
      'name' attribute of the embed object (the plug-in). In this case the
      plug-in name attribute has the value 'OVSplugin', and the below applet
      parameter, pluginName, also has the value 'OVSplugin'. This agreement of
      attribute / parameter values is imperative for communication between the
      plug-in and the Java applet. -->

<!--  Embed the Java Applet Controls  -->
<applet
   code=simplejava.class
   name=ovc_applet
   mayscript="true"
   width=320
   height=30>
      <param name=pluginname value=OVSplugin>
</applet>
</center>
</td></tr>
</table></center>
</body></html>
```

Java Applet Code for Simple Plug-in Example This shows the code for the Java applet that controls the Oracle Video Web Plug-in. You can find the sample source in the file **simpleJava.java** in your **ORACLE_HOME** in the directory:

ovc\demo\webplugin\samples\liveconnect\java\simple\sample1

```
//*****************************************************************************
// simpleJava.java:   Applet
//*****************************************************************************
 import java.applet.*;
 import java.awt.*;
 import netscape.javascript.JSObject;    //provides access to objects
 import netscape.javascript.JSException; //provides access to objects

 import OviObserver;                 //plug-in listener (not implemented)
 import OviPlayer;                   //OviPlayer interface

//===========================================================================
// Main Class for applet simpleJava
//===========================================================================
 public class simpleJava extends Applet
 {
    // Members for applet parameters
    // <type>       <MemberVar>    = <Default Value>
    //---------------------------------------------------------------------------
    private String m_plgname    = "";

    // GUI Objects
    //---------------------------------------------------------------------------

    Button btnPlay = null;
    Button btnPause = null;
    Button btnClose = null;
    JSObject document, window;
    OviPlayer ovi = null;

   // The init() method is called by the AWT when an applet is first loaded or
   // reloaded.  Override this method to perform whatever initialization your
   // applet needs, such as initializing data structures, loading images or
   // fonts, creating frame windows, setting the layout manager, or adding UI
   // components.
   //---------------------------------------------------------------------------
    public void init()
```

```
{

   // PARAMETER SUPPORT
   // The following code retrieves the value of the applet parameter
   //----------------------------------------------------------------------
   String param;

   // param: Parameter description
   //----------------------------------------------------------------------
   param = getParameter("pluginName");
   if (param != null)
      m_plgname = param;

   resize(320, 30);
   setBackground(Color.gray);

   btnPlay = new Button ("Play");
   btnPause = new Button ("Pause");
   btnClose = new Button ("Close");

   add(btnPlay);
   add(btnPause);
   add(btnClose);

   // GET JAVASCRIPT OBJECT (document)
   // Retrieve the the document object from the Browser environment
   //----------------------------------------------------------------------
}

public void start()
{
}

public void stop()
{
}

public void destroy()
{
   ovi = null;
   document = null;
   window = null;
   System.gc();
}
```

```
// MOUSE SUPPORT:
// The mouseUp() method is called if the mouse button is released
// while the mouse cursor is over the applet's portion of the screen.
//--------------------------------------------------------------------------
public boolean mouseUp(Event evt, int x, int y) {

   // GRAB PLUG-IN OBJECT
   // Retrieve the plug-in object handle from the browser context
   //--------------------------------------------------------------------------
   ovi = getPlugin();

   boolean cond = false;
   if(evt.target == btnPlay) {
      if (ovi.getState() == ovi.ST_REALIZED)
         ovi.play();
      else
         ovi.resume();
      cond = true;
   }

   if (evt.target == btnPause) {
      ovi.pause();
      cond = true;
   }

   if (evt.target == btnClose) {
      ovi.unload();
      cond = true;
   }

   return cond;
}
```

```
    // GET JS OBJECT HANDLES AND POINTER TO PLUGIN
    // Retrieves JavaScript Object contexts from Browswer.
    // This function depends upon the Netscape supplied Java package,
    // netscape.javascript. This function will NOT work under Internet
    // Explorer.
    //---------------------------------------------------------------------
    public OviPlayer getPlugin() {
       OviPlayer plugin = null;
       ovi = null;
       document = null;
       window = null;
       System.gc();

       try {
          window = JSObject.getWindow(this); //grab window obj
       }
       catch(NullPointerException e) {
          System.out.println("getWindow() failed in getNavWinDoc()...");
       }

       if (window != null)
          try {
             //grab document obj
             document = (JSObject) window.getMember("document");
          }
          catch(NullPointerException e) {
             System.out.println("getMember() failed in getNavWinDoc()...");
          }

       if (document != null) {
          try {
             plugin = (OviPlayer) document.getMember(m_plgname);
          }
          catch(NullPointerException e) {
             System.out.println("unable to grab plugin");
          }
       }
       else {
          System.out.println("Document null in getPlugin()...\n");
       }

       return plugin;

    } //end getPlugin
}
```

3

Oracle Video Java Library

This chapter describes the Oracle Video Java Library, which enables Java applications to play streaming video and audio from the Oracle Video Server. This chapter contains these sections:

- **Introduction to the Oracle Video Java Library**
- **Requirements**
- **Programming with the Oracle Video Java Library**
- **Using the PlayerApplet Class**
- **Quick Start: A Sample Java Application**

You can also find the API reference for the Oracle Video Java Library in Appendix B, “Oracle Video Java Library Reference”.

Note: Java applets created with the Oracle Video Java Library can be run from the command line, in an applet viewer, or in browsers that permit the execution of unsigned applets. To play video in other browsers, you need to use one of the following:

- The Oracle Video Web Plug-in, as discussed in Chapter 2
- The Oracle Video ActiveX Control, as discussed in Chapter 4

Introduction to the Oracle Video Java Library

The Oracle Video Java Library enables you to create Java applications that play video and audio from an Oracle Video Server. It provides a number of public classes and interfaces that allow you to playback and control video and audio streams, find out information about the current stream, and query the video server for available content titles. This section describes the main groups of public classes and interfaces and how they work together, including:

- **Player Classes**
- **Stream Information Classes**
- **Content Query Classes**

There is also an applet class in the Oracle Video Java Library called **PlayerApplet**. You can use **PlayerApplet** in Java-enabled browsers that allow the use of unsigned applets and as an embedded applet in a Java application. In effect, this means that you can use **PlayerApplet** in Sun's HotJava browser or the JDK **appletviewer** utility, but not in Netscape Navigator or Microsoft Internet Explorer. For more information on **PlayerApplet**, see "Using the PlayerApplet Class" on page 3-31.

This section provides an introduction and description of the Oracle Video Java Library classes and interfaces. For complete reference information on all of the classes and interfaces in the Oracle Video Java Library, see Appendix B, "Oracle Video Java Library Reference".

Player Classes

This section describes the primary classes and interfaces in the Oracle Video Java Library. These are:

- **Player**, which contains all of the functionality needed to directly control video session URL loading and playback
- **PlayerFactory**, which creates new **Player** objects
- The **PlayerListener** interface, which allows your application to be notified of events affecting the **Player** object
- **PlayerException**, which supports notification of raised exceptions specific to the Oracle Video Java Library

Player

The primary object in the Oracle Video Java Library is the **Player** object. The public methods of **Player** can be loosely grouped into three categories:

- **User-interface methods** allow you to control the appearance and behavior of the user interface components, including the video screen, playback controls, and status bar.
- **Media control methods** give you VCR-like control over stream playback, including play / pause functionality, stop, rewind, and forward.
- **Service methods** enable you to control the actual **Player** object by monitoring the player's state, adding a listener, or terminating the player.

User-interface methods The Java video player provides three separate interface components: the video screen, the playback controls, and the status bar. You can retrieve these controls from the **Player** object as a single object, contained in a standard Java **Component** object. This simplifies the layout of each component when the host window is resized. However, you can also get the video screen, the playback controls, or the status bar as separate components for maximum control. Each of these is also returned as a standard Java **Component** object.

You can also implement your own custom controls in place of the default controls or status bar. This is possible since **Player** offers methods that provide VCR-like control over the stream. The only caveat is that you can't override the player's video screen. But you can play audio-only streams without a video screen.

You can find out more about working with the player's interface components in "Retrieving Player Interface Components" on page 3-21.

Media control methods The second group of **Player** methods allows full VCR-like control over the playback of the stream. This includes methods that can pause and resume, play, start and stop, set the position within the stream, and so on. You can use this functionality to provide your own custom user controls or provide hidden control of a stream in the background.

You can find out more about controlling the player in "Loading and Unloading Streams" on page 3-24 and "Controlling Playback" on page 3-26.

Service methods This group of methods provides basic object functionality for the **Player** object. This includes such operations as terminating the **Player** object, retrieving an object's state, and registering an object listener, which provides event-handling methods for the **Player** object.

You can find out more about managing **Player** objects in "Getting and Setting Player Properties" on page 3-11, "Terminating a Player" on page 3-11, and "Handling Player Events" on page 3-18.

PlayerFactory

The **PlayerFactory** class instantiates a **Player** object. Calling the static method **PlayerFactory.getPlayer()** returns a **Player** object.

```
static Player player=null; // create a Player object variable
player = PlayerFactory.getPlayer(); // instantiate the player
```

Note that a **Player** object is not tied to a particular stream. The same **Player** can be reused for many different streams of completely different content types such as MPEG or OSF.

You can find out more about creating a new **Player** object in "Creating a Player" on page 3-10.

PlayerListener

The **PlayerListener** interface specifies a number of methods that a **Player** object may call when specific events occur: errors, end of stream, and changes in the player state. To receive these events, you need to create a class that implements the **PlayerListener** interface. This example shows a simple implementation:

```
public class myApp implements PlayerListener {

   public void stateChange(int newState) {
      System.out.println("newState: " + newState);
   }

   public void error(int code, String msg){
      System.out.println("OVC- " + code + ": " + msg);
   }

   public void endOfStream() {
      System.out.println("end of stream reached");
   }
```

Register the class that implements the listener interface by calling the method **Player.addListener()**, passing the object that implements the **PlayerListener** interface. When the appropriate events occur, the **Player** object calls the listener methods on the implementing object.

You can find out more about implementing the **PlayerListener** interface and handling **Player** events in "Handling Player Events" on page 3-18.

PlayerException

Methods in the Oracle Video Java Library throw a **PlayerException** object when an exception is raised. **PlayerException** can indicate a number of conditions:

- The requested operation is not implemented.
- The **Player** object is in the incorrect state to execute the requested operation.
- An invalid parameter was passed to the method.
- An internal error occurred.
- A general error occurred.
- An untranslated error occurred.

PlayerException also contains a numeric code, which provides specific information on the exception that occurred, and a description, which provides a textual explanation of the problem. It also contains a **toString()** method that returns all of the information contained in the exception object.

You can find out more about handling Oracle Video Java Library exceptions in "Handling Player Exceptions" on page 3-30.

Stream Information Classes

This section describes the classes provided by the Oracle Video Java Library to report on the current stream, including:

- **StmName**, which contains the logical content asset cookie name, the name, and transport protocol of the stream, and the IP address or DNS name of the video server, and the video session URL
- **StmPos**, which contains information on the stream position

- **StmInfo**, which contains stream information, such as the name, transport protocol, description, frame rate, and so on
- **StmStats**, which contains stream statistics, such as the number of data packets received, number of data packets dropped, average frames per second, and so on

Player methods deal with stream position through the **StmPos** class. **StmPos** gives you flexibility over how you specify the stream position, whether you're querying to find out the current stream position or to set the stream position to a new point. You can set the position as an absolute time from the beginning of the stream in either milliseconds or *hh*:*mm*:*ss*:*cc* format. You can also use the current frame number or set it to the beginning or end of the stream.

Player methods use the **StmInfo** class to pass static stream information back and forth. This class contains information such as the name of the stream, the transport protocol, the video session URL for the stream, and a text description of the stream. You can get stream information for a player by calling the **Player**.**getInfo()**, which returns a **StmInfo** object. Then call **StmInfo.toString()**, which returns a formatted string containing all of the above information and more.

You can find the stream's statistics through the **StmStats** class. This class contains network and technical information about the stream, such as the number of data packets received and dropped, consumer (client) and producer (server) playback state, and average frames and bits per second. You can get stream information for a player by calling the **Player**.**getStats()**, which returns a **StmStats** object. Then call **StmStats.toString()**, which returns a formatted string containing all of the above information and more.

You can find out more about getting stream information in "Getting and Setting Player Properties" on page 3-11.

Content Query Classes

> **Note:** The Content query classes and interfaces are deprecated. These query routines return error ovc-46. For information on the supported way to query content, see Appendix C in the *Oracle Video Server Schema and PL/SQL Procedures Guide*.

The Oracle Video Java Library provides a set of classes that you can use to query the Oracle Video Server for a list of available content files. The classes are:

- **Content**, which performs the actual content query
- **ContentIter**, which you use to iterate through the returned content entries
- **ContentException**, which is thrown when an exception occurs during the query operation

You can find information on the use of these classes in "Querying Available Content Titles" on page 3-26.

Requirements

There are some basic requirements for creating and running Java applications that use the Oracle Video Java Library, including:

- **Installing and Configuring the Oracle Video Java Library**
- **Run-time Requirements**
- **Version Requirements**

While some Java tutorial information is presented in this chapter, you should be familiar with Java programming concepts such as objects, classes, interfaces, methods, and compiling and running Java applications.

Installing and Configuring the Oracle Video Java Library

To install the Oracle Video Java Library, either:

- Select the Typical option during OVC installation.
- Select the Custom option and select the Oracle Video Java Library check box.

If you want to develop and compile Java applications or applets that use the Oracle Video Java Library, you need the Java Development Kit (JDK) version 1.1.5 or a later version.

The Oracle Video Java Library is contained in the Java archive file **ovc.jar**. The installer installs this file in **ORACLE_HOME\jbin**.

To run or compile Java classes that use the Oracle Video Java Library, you need to add the **ovc.jar** file to your Java class path. Because this is a JAR archive and not a collection of **.class** files, you explicitly need to indicate the file itself and not just the path to the file. So your CLASSPATH should look something like this:

```
CLASSPATH=C:\JDK1.2\Lib;C:\OraWin95\jbin\ovc.jar;.
```

Run-time Requirements

To run Java applications or applets that use the Oracle Video Java Library, you need the following components:

- The Oracle Video Client, with the Oracle Video Java Library option installed
- The Java Runtime Environment (JRE) version 1.1.5 or later; the JRE is available at **http://www.javasoft.com**.

Version Requirements

The Oracle Video Java Library is compliant with Java 1.1 and later versions. It will not run in any applications that support only Java 1.0, such as Netscape Navigator or Microsoft Internet Explorer, before version 4.0 of either browser. There is an update for Netscape Communicator 4.0 that updates its Java support to Java 1.1 or later.

Programming with the Oracle Video Java Library

This section describes a number of common programming tasks that you can perform when using the Oracle Video Java Library, including:

- **Creating a Player**
- **Terminating a Player**
- **Getting and Setting Player Properties**
- **Handling Player Events**

- **Displaying and Customizing the Player Interface**
- **Loading and Unloading Streams**
- **Controlling Playback**
- **Querying Available Content Titles**
- **Synchronizing Calls to Player Methods**
- **Handling Player Exceptions**

If you need to find the exact syntax for a specific method referenced in this section, refer to Appendix B, "Oracle Video Java Library Reference".

> **Note:** Many of the methods in the Oracle Video Java Library can throw **PlayerException** objects. You should always check whether methods you call throw exceptions and handle them appropriately. See "Handling Player Exceptions" on page 3-30 for information on how to handle **PlayerException**.

Importing the Oracle Video Java Library Package

The Oracle Video Java Library is stored in a Java archive (JAR) file named **ovc.jar**. By default, the installer places this file in the **jbin** directory below your **ORACLE_HOME** directory. This file needs to be placed in your CLASSPATH. See "Installing and Configuring the Oracle Video Java Library" on page 3-7 for more information on configuring your CLASSPATH.

Once you've set the CLASSPATH, you need to import the Oracle Video Java Library package into your source file. The package name is **oracle.ovc**. You need to import all of the classes and interfaces used in your application.

For example, if your application uses **Player**, **PlayerFactory**, and **PlayerException**, you need to import these:

```
import oracle.ovc.PlayerFactory;
import oracle.ovc.PlayerException;
import oracle.ovc.Player;
```

If you want to import all of the classes and interfaces contained in this package, use the * wildcard in your import statement:

```
import oracle.ovc.*;
```

Creating a Player

A Java application can use the Oracle Video Java Library to create and embed a **Player** in three basic steps:

1. Get a new **Player** object by calling the **PlayerFactory.getPlayer()** method:

   ```
   Player m_player = PlayerFactory.getPlayer();
   ```

   Note that you don't need to create a **PlayerFactory** object before making this call. The **PlayerFactory.getPlayer()** method is static, meaning that you can call it through the **PlayerFactory** class instead of a specific instance of that class. When called, **getPlayer()** checks whether a **PlayerFactory** object has already been initialized. The **PlayerFactory** class stores a central **PlayerFactory** object in a static member. If this member hasn't been initialized, **getPlayer()** initializes it automatically. If it has already been initialized, **getPlayer()** uses the already created **PlayerFactory** object to create the new **Player** object.

2. Call **getPlayerUI()** to get the interface for the player. This method returns a standard Java **Component** object. You can pass this **Component** to the window containing the player just as you would a regular Java component, such as a button or text field.

   You can also specify which components of the player interface you want to appear, including the video screen, control panel, and status panel. This example shows how to retrieve the default configuration of the player, which displays all three components:

   ```
   Component m_playerUI = m_player.getPlayerUI();
   ```

   You can find out how to change the configuration of the player's user interface in "Retrieving Player Interface Components" on page 3-21.

3. Add the interface component to the containing window. In this example, it's added to a **Frame** window:

   ```
   Frame m_window = new Frame();
   m_window.add(m_playerUI);
   ```

4. Show the containing window. This example continues from **Step 3**:

   ```
   m_window.show();
   ```

Once you've created a player and displayed it in a window, you can load a stream and begin playback, as described in "Loading and Unloading Streams" on page 3-24.

Terminating a Player

You can terminate your player—including unloading the stream, shutting down the player window, and deallocating all resources—by calling the **Player.term()** method:

```
m_player.term();
```

This method takes no parameters and returns **void**. **term()** automatically unloads the stream before terminating the player. See "Loading and Unloading Streams" on page 3-24 for more information. Once you've terminated a player, you can no longer use it to load other media files.

Getting and Setting Player Properties

A **Player** object has a number of attributes that you can examine and, in some cases, modify. The basic categories are:

- **Stream Position**
- **Volume Settings**
- **Stream and Player State**

Stream Position

The stream position indicates the current playback position. You can both query the player to find its current stream position and set the current stream position to wherever you want.

Getting the stream position You can retrieve the current stream position by calling the **Player.getPos()** method. This method takes an **int** parameter that specifies what format you want for the return value. This returns a **StmPos** object containing the current position in its **m_val** member.

StmPos has two public data members that you can examine and modify:

- The **int m_fmt** indicates the format in which **StmPos** returns the current position. **m_fmt** can have the values shown in Table 3–1:

Table 3–1 Stream Position Formats

Value	Returns stream position as...
StmPos.POSFMT_TIME	Number of milliseconds from beginning of stream
StmPos.POSFMT_FRAMES	Number of frames from beginning of stream

- The **long m_val** contains the actual position of the stream. This is stored as the number of milliseconds or frames from the beginning of the stream (zero for static content, or the creation date of the content for a rolling or terminated feed), but is converted by **toString()** to the units indicated by **m_fmt**.

You can get the stream position in a printable form from the returned **StmPos** object by calling the **StmPos.toString()** method. This returns the current position in the current format, as indicated by **m_fmt**. To change the format of the current position returned by **toString()**, change the value of **m_fmt**.

The following example shows how to get the current position from a **Player** object and display it as a time-formatted string.

```
// Get the StmPos object with the current position
StmPos pos = m_player.getPos(StmPos.POSFMT_TIME);
System.out.println(pos.toString());
```

Setting the stream position You can set the current stream to a new position by calling the **Player.setPos()** method, which takes a **StmPos** parameter and returns **void**. There are a number of steps to setting the player to a new position. These steps assume you have already created a **Player** object and loaded a stream. In the example code shown here, the **Player** object is called **m_player**.

1. Create a new **StmPos** object. There are three different ways to do this:
   a. Call the **StmPos(int fmt, long val)** constructor. The **fmt** parameter indicates the format for the new **StmPos** object, while **val** indicates the position you want to set.
   b. The **StmPos(int fmt)** constructor is just like the first constructor, except that you only set the format; the value is assumed to be 0. Set the **StmPos** object's **m_val** to the desired value if you want some other position.
   c. Call the **StmPos.fromString()** method. This method is static, which means you can call it through the **StmPos** class instead of a specific instance of the class. This method works like a constructor, returning a **StmPos** object that you can use as your new instance of the **StmPos** class. **fromString()** takes a single **String** parameter, which it uses this **String** to set the format of the new **StmPos** object. The values recognized for this parameter are shown in Table 3–2. Note that **fromString()** is not case sensitive.

Table 3–2 Meaning of StmPos.fromString() parameters

Parameter	Format
beginning	Sets **m_fmt** to **StmPos.POSFMT_BEGINNING**; sets the stream position to the beginning
end	Sets **m_fmt** to **StmPos.POSFMT_END**; sets the stream position to the end
current	Sets **m_fmt** to **StmPos.POSFMT_CURRENT**; sets the stream position to its current value
default	Sets **m_fmt** to **StmPos.POSFMT_DEFAULT**; sets the stream position to its default position: ■ For all streams except for unbounded streams (streams with no predetermined end, such as live video), the default position is the beginning of the stream ■ For unbounded streams, the default position is the end of the stream
string	Sets **m_fmt** to **StmPos.POSFMT_TIME** and **m_val** to the time contained in *string*; *string* should be of the form *hh*:*mm*:*ss*:*cc* or in wallclock time (see "playFrom" on page A-6 for more information),

2. Call the **Player.setPos()** method, passing the **StmPos** object you created in **Step 1** to the method.

Once this has been successfully completed, the stream position changes to the set value. Playing the stream causes playback to begin at that position.

See "Loading and Unloading Streams" on page 3-24 for information on setting the stream position at load time.

Volume Settings

Getting and setting the volume is fairly straightforward. Volume is represented as an integer from 0 to 100, with 0 meaning no volume and 100 meaning full volume.

Getting the volume You can get the current volume setting of a **Player** object by calling the **getVol()** method. This method takes no parameters and returns an **int** that indicates the current volume. The volume setting goes from 0 (off) to 100 (full volume).

Setting the volume You can change the volume setting of a **Player** object by calling the **setVol()** method. This method takes an **int** that indicates the new volume. The volume setting can be from 0 (off) to 100 (full volume).

You can also set the volume when you load a new stream:

1. Create a new **StmOpts** object.
2. Set the **StmOpts.m_volume** member to the desired volume setting.
3. Pass the **StmOpts** object to the **Player.load()** method.

Stream and Player State

There are three different types of player states that you can query:

- The state of the **Player** itself, such as whether it's initialized or not, loading or unloading, playing, and so on
- Static information about the stream, such as its name, description, or length
- Statistics about the stream, such as network performance, average frames or bits per second, or playback state

Player state To find the state of the player, call the **Player.getState()** method. This method takes no parameters and returns an **int**. This return value can have a number of possible values, as shown in Table 3–3.

Table 3–3 Player States

Value	Indicates player is...
Player.ST_UNINIT	Uninitialized
Player.ST_INIT	Initialized but does not have a stream loaded
Player.ST_REALIZED	Ready to play the currently loaded stream
Player.ST_PLAYING	Playing the currently loaded stream
Player.ST_PAUSED	Paused during playback
Player.ST_EOS	At the end of the currently loaded stream
Player.ST_ERROR	In an error state

The sample code below shows how you might use this information.

```
// Create a new player and load a media file
Player m_player = new Player();
m_player.load("/mds/video/oracle.mpi");

while(m_player.getState() == Player.ST_UNINIT) {
   // Just loop while the player's still loading...
   }

// Now that we know it's loaded, we can start playing
m_player.play();
```

Stream Information You can get information about the current stream by calling the **Player.getInfo()** method. This method takes no parameters and returns a **StmInfo** object.

Table 3–4 StmInfo Data Members

Name	Type	Description
m_aspect	int	Aspect ratio * 1000
m_asset	String	Asset cookie in video session URL
m_bitrate	int	Total bit rate in bits per second
m_bytes	long	File size

Table 3–4 StmInfo Data Members (Cont.)

Name	Type	Description
m_contStat	int	Content status; this can have the following values: ■ **StmInfo.CSTAT_DISK** indicates the current stream is stored on disk on the video server ■ **StmInfo.CSTAT_FEED** indicates the current stream is a one-step encode file from the video server ■ **StmInfo.CSTAT_LOCALFILE** indicates the current stream is a local file ■ **StmInfo.CSTAT_NETWORK** indicates the current stream is a broadcast or multicast stream ■ **StmInfo.CSTAT_ROLLING** indicates the current stream is a rolling feed ■ **StmInfo.CSTAT_TAPE** indicates the current stream is stored in the Hierarchical Storage Manager (HSM) on the video server ■ **StmInfo.CSTAT_UNKNOWN** indicates unknown status You can get this information as a **String** value by calling the **StmInfo.contStatToString()** method.
m_contType	String	Container type, such as MPEG, OSF, WAV, and so on
m_createTime	Date	Date and time the content file was created (GMT)
m_desc	String	Description
m_fps	int	Frame rate, expressed as frames per second * 1000
m_msecs	int	Total duration of the stream (in milliseconds)
m_name	String	Printable name of stream
m_proto	String	Transport protocol
m_size	Dimension	Source input size (in pixels)
m_url	String	video session URL for stream

You can also get all of this information in a formatted **String** by calling the **StmInfo.toString()** method.

Stream statistics You can get information about the current stream by calling the **Player.getStats()** method. This method takes no parameters and returns a **StmStats** object. The public data members for **StmStats** are shown in Table 3–5.

Table 3–5 StmStats Data Members

Name	Type	Description
m_bps	int	Average bits per second
m_cnsState	int	Consumer playback state: ■ **StmStats.STM_CONTROL** indicates that the stream is in the middle of a network transaction. ■ **StmStats.STM_ENDED** indicates that the end of stream has been reached. ■ **StmStats.STM_IDLE** indicates that the stream is idle. ■ **StmStats.STM_PAUSED** indicates that the stream is paused. ■ **StmStats.STM_PLAYING** indicates that the stream is playing. ■ **StmStats.STM_STALLED** indicates that the stream has stalled.
m_curFrame	long	Current frame in stream
m_curTime	long	Current time in stream
m_drops	int	Number of data packets dropped
m_fBytes	int	Number of free bytes in cache
m_fps	int	Average frames per second
m_maxTime	long	Last seekable position in stream
m_minTime	long	Earliest seekable position in stream
m_pkts	int	Number of data packets received
m_prdState	int	Producer playback state; possible values are the same as for **m_cnsState**
m_rBytes	int	Number of ready bytes in cache

You can also get all of this information in a formatted **String** by calling the **StmStats.toString()** method.

Handling Player Events

To handle events from a **Player** object, you need to implement the **PlayerListener** interface. This section describes the methods in **PlayerListener** you need to implement and also describes how to implement the listener interface and add it to a **Player**'s list of listeners.

> **Note:** You should be familiar with the Java 1.1 event model, as well as the concept of interfaces, before reading this section.

PlayerListener Methods

PlayerListener handles a number of **Player** events, including:

- **Change of player state**
- **End of stream**
- **Errors**

The methods required to handle these events are described below.

Change of player state A change of player state indicates that the player has undergone a change from one state to another. This is often the point at which you may wish to take some sort of action. For example, whenever a user starts playing a stream, you want to change an icon on the interface to indicate the play status. Because playing is one of the state changes handled by the **PlayerListener** interface, you can watch for this event.

The method used to handle changes of state is called **stateChange()**. This method takes a single parameter, an **int**. This parameter indicates the new state of the **Player** object and can have one of the values shown in Table 3–3.

End of stream Reaching the end of a stream means that the **Player** has run out of data to play. In short, the movie's over. You may want to take some sort of action whenever a stream finishes. For example, you may want to begin loading the next stream.

The method used to handle the end of a stream is called **endOfStream()**. This method takes no parameters.

Errors You need to do something when an error occurs in your application. Most recoverable errors are handled through the use of exceptions. If an error occurs that isn't handled by an exception, you can assume that it was a fatal error. The **error()**

method gives you a chance to do clean up and damage control, including destroying the **Player** object.

The **error()** method takes two parameters:

- An **int** that contains an error code. This code may be used by Oracle technical support in case diagnostic help is required.
- A **String** that contains a text message.

Implementing and Registering a PlayerListener

To implement a **PlayerListener** to handle **Player** events:

1. Create a new class that implements the **PlayerListener** interface.

   You can create any class to implement this interface, although you may find it easiest to implement in whichever class you create the **Player** object. The example below shows the syntax for adding **PlayerListener** to another class:

   ```
   public class myClass implements PlayerListener {
      // Class implementation goes here...
      }
   ```

2. Add the **PlayerListener** methods to the class in which you implemented the **PlayerListener** interface.

   ```
   public class myClass implements PlayerListener {
      // Class implementation goes here...

      // PlayerListener methods
      public void stateChange(int newState) {
         // Take whatever actions you want for this event
         }
      public void error(int code, String msg) {
         // Take whatever actions you want for this event
         }
      public void endOfStream() {
         // Take whatever actions you want for this event
         }
      }
   ```

3. Create an instance of your new class.

   ```
   myClass m_inst = new myClass();
   ```

4. Create a new **Player** object.

   ```
   Player m_player = PlayerFactory.getPlayer();
   ```

5. Call the **addListener()** method on the **Player** object you created in the last step, passing the object you created in **Step 3**. The **addListener()** method takes a single parameter, an object that implements the **PlayerListener** interface.

   ```
   m_player.addListener(m_inst);
   ```

Once you've registered an implementation of **PlayerListener** with a **Player** through the **addListener()** method, the **Player** object calls the appropriate methods on the listener whenever an event occurs.

The sample code below shows a simple example of how to implement and register a player listener.

```
public class myClass implements PlayerListener {
   Player m_player = null;
   myClass app = null;

   public static void main(String args[]) {
      app = new myClass();

      try {
         m_player = PlayerFactory.getPlayer();
         }
      catch(PlayerException e) {
         System.out.println("Failed to getPlayer()");
         }

      try {
         m_player.addListener(app);
         }
      catch(PlayerException e) {
         System.out.println("Failed to addListener()");
         }

      // Get player interface, add to frame, load stream, and so on...
      }

   // PlayerListener methods
   public void stateChange(int newState) {
      System.out.println("Player state change to: " + newState);
      }
```

```
public void error(int code, String msg) {
   System.err.println("Error code: " + code);
   System.err.println("Error message: " + msg);
   }

public void endOfStream() {
   System.out.println("End of stream reached!");
   }
}
```

Displaying and Customizing the Player Interface

The Oracle Video Java Library player provides three Java **Component**-based interface objects that you can use to provide a video display, status bar, and playback controls. You can also combine any combination of the three interfaces into a single **Component**, simplifying the process of adding the player to your interface.

Because these objects are based on the standard Java **Component**, you can use them in windows, frames, and containers just as you would any standard Java component, such as buttons, list boxes, and icons.

This section covers the following topics:

- **Retrieving Player Interface Components**
- **Customizing Interface Components**
- **Setting Full-Screen Interface**

Retrieving Player Interface Components

There are two ways you can get object references for the interfaces:

- Call individual methods to retrieve each separate component. There are three methods provided by **Player**, one for each component:
  - **getControlComp()**
  - **getStatusComp()**
  - **getVisualComp()**

  Each of these methods returns a **Component** object representing the requested component.

- You can also retrieve one or more of the components combined into a single **Component** object by calling the **Player.getPlayerUI()** method. There are three versions of this method:
  - The first version takes no parameters. This creates the default player interface, combining a video screen, controls, and status bar.

    This example uses the first version of **getPlayerUI()** to specify the default configuration—all three components displayed:

    ```
    Component m_UI = m_player.getPlayerUI();
    ```

  - The second version takes three **boolean** parameters. This allows you to specify which parts of the default interface—video screen, controls, and status bar, respectively—you want displayed. Specifying **true** for any component means that component should be included in the returned **Component**.

    This example uses the second version of **getPlayerUI()** to create the player with only the video screen showing, with no controls or status bar:

    ```
    Component m_UI = m_player.getPlayerUI(true, false, false);
    ```

  - The third version takes three **boolean** and two **int** parameters. The **boolean** parameters function the same as in the second version, turning various interface components on and off. The **int** parameters let you specify the width and height of the returned **Component** object; that is, the overall size of the player interface.

    This example uses the third version of **getPlayerUI()** to create the player with the video screen and controls showing, with no status bar, and a size of 320 by 240 pixels:

    ```
    Component m_UI = m_player.getPlayerUI(true, true, false, 320, 240);
    ```

Once you've retrieved the **Component** containing the player interface, you can add it to a frame or window just as you would any other **Component**. The code below shows a simple example using the default player interface:

```
public class Spud extends Frame {
   public static void main(String argv[]) {
      Component myUI = null;
      Spud spud = new Spud();
      try {
         Player m_player = PlayerFactory.getPlayer();
         myUI = m_player.getPlayerUI(); // Get the default UI component
```

```
        }
      catch(PlayerException e) {
         System.out.println("Exception raised: " + e.toString());
         }

      spud.add(myUI); // Add the UI to the enclosing frame
      spud.show(); // Show the enclosing frame
      }
   }
```

Customizing Interface Components

Because you can request the player's interface components separately, you can mix and match the elements that you use. You can also substitute your own interface components for the default components provided by the player, with the exception of the video screen: you must use the default screen if you want to show video.

To create your own interface component, simply design and lay the component or components out as you would normal Java controls. Through the event handlers for these components, call **Player**'s playback control methods in response to user actions. See "Controlling Playback" on page 3-26 for more information on the playback control methods.

Setting Full-Screen Interface

In addition to displaying the player interface in a container window, you can also change the video window to full-screen mode. To do this, call the method **Player.setFullScreen()**. This method returns **void** and takes a single parameter, a **boolean**. Passing **true** as the parameter puts the player in full-screen mode: the controls, status bar, and other operating system desktop items disappear and the video screen takes up the available real estate.

Note that you don't necessarily have to call this method yourself to provide this functionality to your users. If the pop-up menu is enabled, the user can select full-screen mode through the pop-up menu.

The user can get out of full-screen mode by pressing the Escape key.

Loading and Unloading Streams

A **Player** object is not tied to a particular stream. In fact, you can't create a **Player** with a default stream. In order to use a **Player** to play a stream, you need to explicitly load the stream you want to play by calling the **Player.load()** method.

load() returns **void** and takes two parameters:

- A **String** containing the video session URL for the stream you want to load. For information on video session URLs, see Appendix D, "Oracle Video Session URLs".
- A **StmOpts** object. **StmOpts** wraps a number of start-up options for the player in a single object. It consists of a number of public data members that you can modify to specify your own start-up options. Table 3–6 shows the available data members in **StmOpts** and the default values given to these members when you call the default **StmOpts** constructor.

Table 3–6 StmOpts Data Members

Name	Type	Default	Description
m_autoStart	boolean	false	When **true**, playback starts at the beginning of the stream or the position specified by **m_playFrom** once the player loads the video session URL; otherwise, waits for a play command.
m_img	String	""	Specifies a background image to be displayed when the player is inactive.
m_leftClick	boolean	true	Specifies whether a left mouse click on the video screen can play and pause playback.
m_loop	boolean	false	Specifies whether playback loops when the stream position reaches the end or the position specified by **m_playTo**, if specified.
m_playFrom	StmPos	default	Specifies the position from which to begin playback; see "Stream Position" on page 3-11 for more information on the **StmPos** class.
m_playTo	StmPos	end	Specifies the position at which to end playback; see "Stream Position" on page 3-11 for more information on the **StmPos** class.
m_popup	boolean	true	Specifies whether a pop-up menu appears when the user right-clicks the video screen.
m_volume	int		Specifies the master volume.

You can call the **load()** method as soon as you've created a valid **Player** object. If you want to load the stream with options other than the default, you first need to create a **StmPos** object, change the desired options, then call **load()** with the modified **StmPos** object.

This example shows how to load a stream using the default options. Notice that the call to **load()** uses **null** in place of a **StmPos** object.

```
public class Spud extends Frame {
   public static void main(String argv[]) {
      Spud app = new Spud("Load a file");
      Player player = null;

      try {
         player = PlayerFactory.getPlayer();
         app.add(player.getPlayerUI());
         app.show();
         player.load("/mds/video/intro.mpi", null);
         }
      catch (PlayerException e) {
         System.out.println("Exception: " + e.m_msg);
         }
      }

   public Spud(String title) {
      super(title);
      }
   }
```

You can also unload video session URLs from the player by calling the **unload()** method. This method returns **void** and takes no parameters. If you've already loaded a stream, call **unload()** before trying to load another. You should also call **unload()** before terminating a stream. See "Terminating a Player" on page 3-11 for more information on terminating **Player** objects. The example below shows how you might use the **unload()** method.

```
// Stop playing and get rid of the player
m_player.unload();
m_pleyer.term();
```

Controlling Playback

Player provides a number of methods that provide VCR-like control over video session URL playback. These methods, which all return **void**, include:

- **play()** starts stream playback; where playback depends on which **play()** method you call, **play()** or **play(StmPos from, StmPos to)**:
  - **play()** starts from the beginning of the stream, regardless of the current stream position, as long as you didn't specify a **playFrom** position with the **load()** method. If so, playback starts from that point.
  - **play(StmPos from, StmPos to)** plays from the position indicated by **from** until it reaches the position indicated by **to**.

  You can only call **play()** when the stream is stopped. To start a paused stream, call **resume()**.
- **stop()** stops playback, resetting the position back to the beginning of the stream. If you specified a starting position, **stop()** resets that to the beginning of the stream also.
- **pause()** stops playback, but, unlike **stop()**, doesn't set the current position back to the beginning.
- **resume()** resumes playback at the current position. You can only call **resume()** when the stream is paused. To start a stopped stream, call **play()**.

Querying Available Content Titles

Note: The Content query classes and interfaces are deprecated. These query routines return error ovc-46. For information on the supported way to query content, see Appendix C in the *Oracle Video Server Schema and PL/SQL Procedures Guide*.

The **Content** class, along with **ContentIter** and **ContentException**, enables you to request a list of available content titles from an Oracle Video Server. This section contains the following topics:

- **Content Classes**
- **Performing a Query**

You can find a sample using the content query classes in your OVC installation. Look in the directory **ovc\demo\java** in your **ORACLE_HOME**. The file

CtntList.java contains a demonstration of using the content query classes to get a list of available titles from the Oracle Video Server.

Content Classes

There are three classes used for retrieving lists of available content titles from the Oracle Video Server:

Content The **Content** class contains only one method, **query()**, and no data members. Since **query()** is static, you never need to actually create a **Content** object, but instead can just call **query()** through the class.

Content.query() takes three parameters:

- A **String** containing the address (in the form of a video session URL) of the server you want to query. If you pass **null** for this parameter, **query()** checks the default server. If you don't have a default server specified, or if that server can't be contacted, **query()** raises a **ContentException**.
- A **String** that contains a file specifier. **query()** uses this as a filter to determine which files are retrieved on the query. You can use UNIX-style wildcards to build this. For example, the string "ora*" would retrieve only titles whose names begin with "ora".
- A **ContentIter** object. This object works with **query()** so that you can retrieve the available titles in discreet chunks.

query() returns an array of **StmInfo** objects. Each of these **StmInfo** objects represents a content title available on the specified server that matches the wildcard you passed to **query()**. The number of objects in the returned array depends on the values you passed to the **ContentIter** constructor and the number of titles available. See the description of **ContentIter** for more information on how **query()** uses the **ContentIter** object.

ContentIter The **ContentIter** class works with the **Content.query()** method to retrieve lists of available content from an Oracle Video Server. The **ContentIter** constructor takes two parameters:

- Position to start querying, stored in the **int** data member **m_pos**. For example, you may specify that you want to start at position 30. This means that the first 30 items that meet the file specification passed to the **query()** method are disregarded. Your returned list begins with the 31st.
- Number of titles to retrieve with each query. This is stored in the **int** data member **m_num**. Specify 10 titles and **query()** returns 10 titles each time you

call it; at least, **query()** returns that many as long as there are that many available.

Because the number of available titles is potentially quite high, you do the query iteratively; that is, you perform the same steps repeatedly, each time retrieving another bunch of titles until you've exhausted the available ones. You can also specify which position you want to start on, meaning that if you already retrieved some titles, you don't have to start back at the beginning.

The **query()** method updates **m_pos** after each query operation. For example, suppose you start with **m_pos** set to 0 and **m_num** set to 10. After the first call to **query()** (assuming it's successful), **query()** sets **m_pos** to 10. After the second call, **query()** sets **m_pos** to 20, and so on.

Once you've exhausted the available files, **query()** makes some changes to your **ContentIter** object:

- **m_pos** is set to -1
- **m_num** is set to the number of titles successfully retrieved

Continuing from the example above, you call **query()**, passing your **ContentIter** object with a 10-title capacity for each query. If there are 55 titles available on your server, you can call **query()** six times: the first five calls give you ten titles, with **m_pos** becoming progressively larger (0, 10, 20, and so on) and **m_num** staying the same, 10. But when you make the sixth call, **query()** realizes that there aren't enough titles left to retrieve ten titles for you. It sets **m_pos** to -1 and sets **m_num** to 5, since, after retrieving five chunks of ten titles, there are five titles left in the list.

ContentException The **Content.query()** method can raise a **ContentException** if it encounters an exceptional situation. Possible causes are poorly formed server addresses or a down or inactive server.

ContentException is nearly identical to the **PlayerException** object in usage, methods, and data members. See "Handling Player Exceptions" on page 3-30.

Performing a Query

To find a list of available titles:

1. Declare a **StmInfo** array. Don't declare the size of the array. This array is used to store the list of available titles. Your array declaration should look something like this:

   ```
   StmInfo[] contentList;
   ```

2. Create a **ContentIter** object.

   For the **ContentIter** constructor, pass two parameters, the position at which you want to start in the list of available titles and the number of titles you want to retrieve with each query.

   The example below shows an iterator that starts at the first available title and gets 10 titles at a time:

   ```
   ContentIter iter = new ContentIter(0, 10);
   ```

3. Call the **Content.query()** method. Because this method is static, you can call it through the class without creating an instance of the **Content** class.

   query() takes three parameters:

   - A server address (as a **String**)
   - A file specifier (as a **String**)
   - The **ContentIter** object you created in **Step 2**

   Assign the return value of the **Content.query()** call to the **StmInfo** array you created in **Step 1**.

   This example returns all available titles from the default server:

   ```
   contentList = Content.query(null, "*", iter);
   ```

4. Check the value of the **m_pos** member of your **ContentIter** object:

   - If **m_pos** is *not* -1, perform whatever operations you want with the list of titles. After that, you can go to **Step 3** and get another chunk of titles.
   - If **m_pos** is -1, the last call to **query()** returned the last available titles. Check the value of **m_num** to find out how many titles were returned.

This example shows how to create a **StmInfo** array and a **ContentIter** object, retrieve all of the available titles from the default video server in chunks of 10, and print out their names and descriptions.

```
int i;
ContentIter iter = ContentIter(0, 10);
StmInfo[] contentList;

try {
   while (iter.m_pos!=-1) {
      contentList = Content.query(null,"*",iter);
```

```
      for(i=0; i<iter.m_num;i++) {
         System.out.println("Name: " + contentList[i].m_name);
         System.out.println("Description: " + contentList[i].m_desc);
         }
      }
   }
catch(ContentException e) {
   System.out.println("ContentException raised: " + e.toString());
}
```

Synchronizing Calls to Player Methods

All **Player** methods are synchronous in the sense that, when the routine returns, the calling thread can assume the routine has completed. For example, when **Player.load()** returns, you can assume that the **Player** object has completed loading a stream. The methods are not, however, necessarily synchronized in the context of a multi-threaded Java programming environment. In certain cases, it is possible that you may call **Player** methods concurrently or in an inappropriate order. For instance, in the case of a graphical user interface, by default Java spawns a new thread for every mouse event that occurs. It's possible for different threads to call **Player** methods, resulting in improper or contradictory sequences of calls.

Therefore, when writing applications that are based on the Oracle Video Java Library, you must take into account the multi-threaded environment of Java. The potential for concurrent or unordered calls to **Player** methods can be managed by carefully tracking the **Player** object's states. The relationships between **Player** states and **Player** methods are discussed in detail in "Media Control Methods" on page B-12. See the description of the **Player.getState()** method in "Service methods" on page 3-4 and the **PlayerListener** class reference on page B-21 for detailed information on tracking **Player** object states.

Handling Player Exceptions

Most **Player** methods throw the **PlayerException** exception object when they encounter errors or other exceptional situations. **PlayerException** provides information on the exception, so that you can recover from the error or clean up and exit the application if you can't recover from the error.

PlayerException has three public members. The first is an **int** named **m_type**. **m_type** can have one of the following values:

- **PlayerException.EX_BADPARAM** indicates that an invalid parameter was passed to the method.
- **PlayerException.EX_BADSTATE** indicates that the player is in the incorrect state to execute the requested operation.
- **PlayerException.EX_ERROR** indicates a general error state.
- **PlayerException.EX_INTERNAL** indicates an internal error.
- **PlayerException.EX_NOTIMPL** indicates that the requested operation is not implemented.
- **PlayerException.EX_UNTRANS** indicates an untranslated error. Call **PlayerException**.**toString()** for more information on the exception.

PlayerException also contains an **int** member **m_code** that contains a specific numeric code and a **String** member **m_msg** that provides a more detailed explanation of the problem. You can get all of this information in one string by calling the **toString()** method.

Using the PlayerApplet Class

The **PlayerApplet** is a simple applet that supports the same syntax and features as the Oracle Video Web Plug-in. You can embed this applet in stand-alone Java applications.

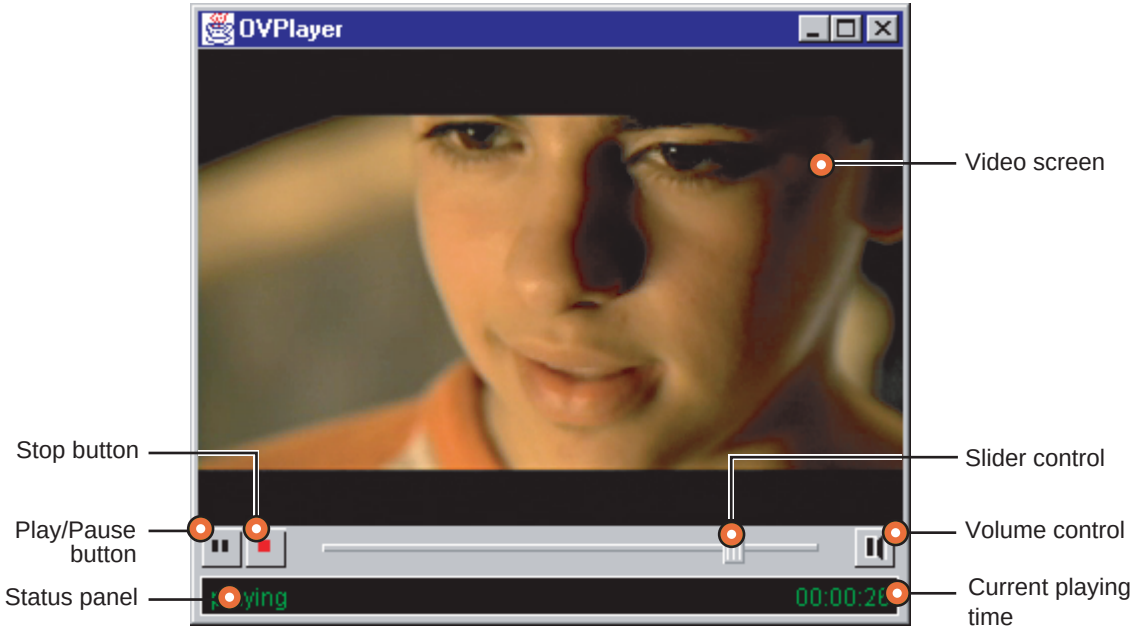
You can also use it to embed video into an HTML document just as you would with the Oracle Video Web Plug-in, except that **PlayerApplet** works only in Java-enabled browsers that allow the use of unsigned applets. In effect, this means that you can use **PlayerApplet** in Sun's HotJava browser or the JDK **appletviewer** utility, but not in Netscape Navigator or Microsoft Internet Explorer. For more information on **PlayerApplet**, see "Using the PlayerApplet Class" on page 3-31.

Quick Start: A Sample Java Application

This section describes how to use the Oracle Video Java Library to create a simple Java application that displays video in a window. A step-by-step tutorial presents each section of code, along with explanations of what's happening. The compiled source code for this example (and other more complex examples) are installed when the Oracle Video Java Library is installed. You can find the sample code in the file **Simple.java**, located in the directory **ovc\demo\java** in your **ORACLE_HOME**.

Figure 3–1 shows how this simple Java application appears and shows the features of the application's interface.

Figure 3–1 Simple.bat runs Simple.class, the compiled version of Simple.java



Step-by-Step Tutorial of Simple.java

The following is a step-by-step tutorial of the code in the sample application, **Simple.java**. Keep in mind that this is the minimum code required to create a Java client application. Applications using the Oracle Video Java Library can be implemented in many different ways.

1. Import the basic Java packages required: **java.awt** and **java.awt.events**.

   This provides support for windowing and interface event handling.

   ```
   // Sun JDK Imports
   import java.awt.*;
   import java.awt.event.*;
   ```

2. Import the Oracle Video Java Library classes.

   ```
   // Oracle Video Client Imports
   import oracle.ovc.PlayerFactory;
   import oracle.ovc.PlayerException;
   import oracle.ovc.Player;
   ```

3. Create an application class. This is the class that contains the **main()** method, data members, and methods. In the sample application, this class is called **Simple**.

   ```
   public class Simple
   {
      // Rest of the class, including member variables and methods, go here...
   }
   ```

4. In your **main()** method, create an application object so that you can call the methods of that object. In the sample application, the application object is called **app**.

   ```
   Simple app = new Simple ();
   ```

5. Create a window to hold the video player.

   In **Simple.java**, the window is created as a subclass of the Java AWT **Frame** class. The subclass is named **PlayerFrame**. The instantiation of **PlayerFrame** is relatively straightforward:

   - The title parameter is passed on to the superclass
   - A window adapter (**PFAdapter**, a subclass of **WindowAdapter**) that handles only the window closing event is created and added as a window listener

- The size of the window is set to the size of the height and width parameters passed to the **PlayerFrame** constructor

```
playerframe = new PlayerFrame("OVPlayer", m_Width, m_Height);
```

Later, after you create the **Player** object (**Step 6**) and its user interface (**Step 7**), this frame window hosts the player's interface, which is "added" to the frame window using the **add()** method.

6. Create a **Player** object by calling the **PlayerFactory**.**getPlayer()** method. This method takes no parameters and returns a **Player** object reference.

   In the sample, this is handled in the **addPlayer()** method of the **Simple** object. You'll note that, in this example, the **PlayerFactory** object is instantiated dynamically: since it's needed only once here, there's no need to keep it around.

   Because **Player** and **PlayerFactory** methods can throw exceptions, you must enclose method calls on these objects with a **try-catch** block. There are two specific exceptions you may encounter:

   - **UnsatisfiedLinkError** is a standard Java exception thrown when the virtual machine can't satisfy all of the links in a class that it has loaded.
   - **PlayerException** is thrown by all members of the **Player** class when an error is encountered. To find out more about exception handling in the Oracle Video Java Library, see "Handling Player Exceptions" on page 3-30.

```
//  Create a player object.
try {
   player = PlayerFactory.getPlayer();
   }
catch (UnsatisfiedLinkError e) {
   System.out.println(e.getMessage());
   }
catch (PlayerException e) {
      System.out.println("Unable to create a Player object.");
   }
```

7. Create and install the player's user interface. This consists of three steps:

   a. Get a user interface object from the **Player** object by calling the **getPlayerUI()** method. This method returns a **Component** object that contains the various elements of the video player, including the video screen, a control panel, and a status line. You can actually specify which of these elements you want in your interface by specifying different

parameters to the **getPlayerUI()** method. See User Interface Methods for more information.

b. Add the user interface to the application's frame window by calling the window's **add()** method, passing as a parameter the object returned by **getPlayerUI()**.

c. Finally, display the frame window by calling the window's **show()** method.

```
try {
   playerUI = player.getPlayerUI();
}
catch (PlayerException e) {
   // Handle the exception here...
}
//  Add the visual component of the player object
//  to the frame before invoking init().
playerframe.add(playerUI);
playerframe.show();
```

The player object is now ready to load and display video.

8. To load a video file, call the **Player.load()** method. This method takes two parameters:

- A **String** parameter containing a valid video session URL. Video session URLs describe the file to be loaded. You can find the format for video session URLs in "Oracle Video Session URLs" on page D-1.
- A **StmOpts** object. **StmOpts** objects allow you to specify options for the video stream, such as whether the video should start automatically, where to start and stop within the stream, and more. You can find out more about the **StmOpts** class on page B-29. Specifying **null** for this parameter indicates that the stream should load with the default options: no automatic start, no looping, full volume, start at the beginning, and finish at the end.

```
player.load(m_MediaFile, null);
```

In **Simple.java**, the **m_Mediafile** points to one of the sample content files installed with the Oracle Video Server. You could change this so that:

- the Java application queries the video server for a list of available content (see "Querying Available Content Titles" on page 3-26)
- the user can select from a preset list of videos. For example, you may want to limit the user to a set of training videos. You could present the titles in a drop-down list and load the appropriate video when one is selected.

- the user can browse locally stored files. For example, create a dialog box that contains a list of files in a particular directory or provides the ability to browse the local hard drive.

Now the media resource should be loaded and you can perform whatever operations you want: play the video, change the volume, change the position, and so on.

4

Oracle Video ActiveX Control

The Oracle Video ActiveX Control defines methods, properties, and events that you can incorporate into any web page (however, the page will only be viewable using Internet Explorer). The ActiveX Control can also be incorporated into any Microsoft Visual Basic, Microsoft Visual C++ or any other ActiveX-enabled application. The ActiveX (formerly OCX and OLE) standard lets you create modular applications that can run on the Internet, embed into ActiveX container applications, or work in your own applications.

This chapter is intended for developers who want to add video capabilities to their ActiveX-compliant applications. It shows you how to load the Oracle Video ActiveX Control in Visual Basic and HTML documents, and guides you through creating a simple application.

This chapter contains these sections:

- **Introduction to the Oracle Video ActiveX Control**
- **Requirements**
- **Using the Oracle Video ActiveX Control in HTML Documents**
- **Creating Applications with the Oracle Video ActiveX Control**

You can also find reference information about the Oracle Video ActiveX Control in Appendix C, “Oracle Video ActiveX Control Reference”, including methods, properties, and events associated with the control.

Introduction to the Oracle Video ActiveX Control

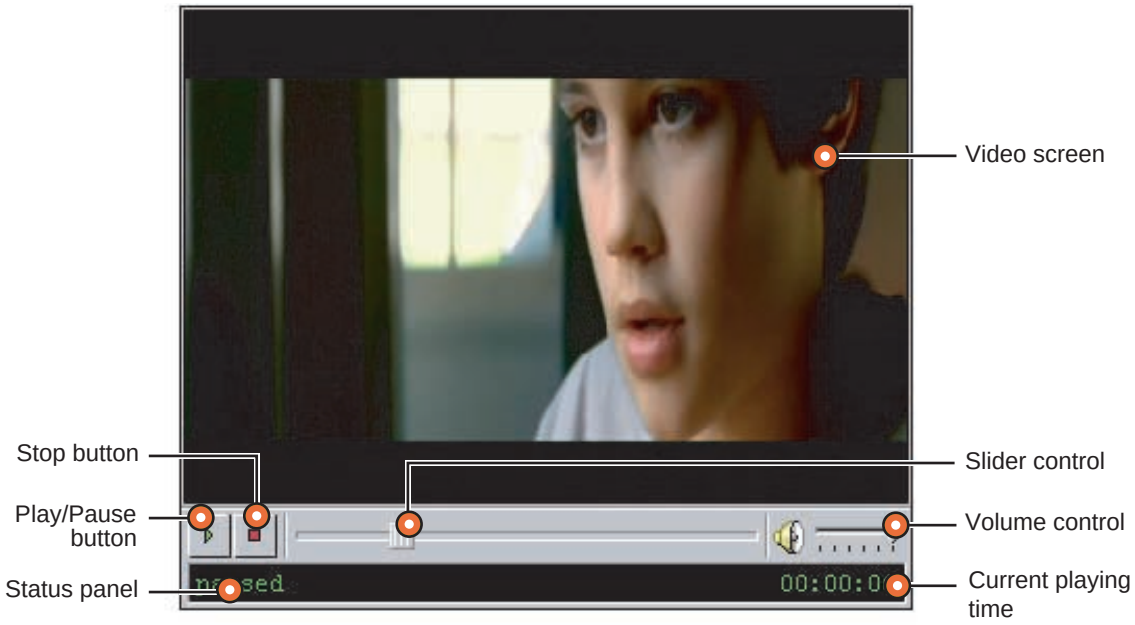
This section provides a basic introduction to the Oracle Video ActiveX Control:

- **Becoming Familiar with the Oracle Video ActiveX Control**
- **Installing the Oracle Video ActiveX Control**
- **Controlling the Oracle Video ActiveX Control**

Becoming Familiar with the Oracle Video ActiveX Control

Figure 4–1 shows the Oracle Video ActiveX Control as it appears on-screen.

Figure 4–1 The Oracle Video ActiveX Control



The controls located at the bottom of the Oracle Video ActiveX Control enable users to play, pause, seek, and stop the video and adjust the volume.

Installing the Oracle Video ActiveX Control

This is the developer installation. The end user installation can be set up as described in Chapter 5.

To install the Oracle Video ActiveX Control, make sure you select either the Typical or Compact installation configuration or select the Oracle Video ActiveX Control checkbox if you choose the Custom installation configuration.

The installer installs Oracle files in different areas on different Windows platforms. For this reason, file paths are given relative to **ORACLE_HOME**, which represents the directory where you installed the client.

- On Windows 95 clients, the default **ORACLE_HOME** is **C:\ORAWIN95**
- On Windows 98 clients, the default **ORACLE_HOME** is **C:\ORAWIN98**
- On Windows NT 4.0 clients, the default **ORACLE_HOME** is **C:\ORANT**

The installer registers the Oracle Video ActiveX Control in the Windows registry during installation.

You can find the complete installation instructions for the Oracle Video Client in the CD insert that came with your installation CD-ROM. This information is also available in electronic form in the file **ovcinstl.pdf** in the **\docs** directory of your installation CD-ROM and, if you selected the Typical installation configuration or selected the Docs option in the Custom configuration, in the directory **ovc\docs** in your **ORACLE_HOME**.

Using the Oracle Video ActiveX Control

Because it is an ActiveX control, the Oracle Video ActiveX Control is very versatile. It can be used as:

- An embedded control in an HTML document. Microsoft's Internet Explorer supports ActiveX controls in its Windows-based browsers. To find out how to use the control in an HTML document, see "Using the Oracle Video ActiveX Control in HTML Documents" on page 4-5.
- A custom component in ActiveX-enabled development environments, such as Visual Basic. You can use this capability to add OVS video to your own stand-alone applications. To find out how to use the control in Visual Basic, see "Loading the Oracle Video ActiveX Control" on page 4-11.

The Oracle Video ActiveX Control has been tested and found to be compatible with recent versions of:

- Microsoft Visual Basic
- Microsoft Visual C++

- Microsoft Internet Explorer using JScript and VBScript
- Macromedia Authorware

See the *Oracle Video Client Release Notes* for the most up-to-date compatibility information. The control will function properly in any application that fully supports ActiveX controls, but we cannot guarantee its performance in untested applications or development tools. Oracle strongly recommends that you test the control in any uncertified applications before using it for development.

Controlling the Oracle Video ActiveX Control

You can use a number of techniques to script and control the Oracle Video ActiveX Control. What you use depends on where you are using the control:

- **In HTML Documents**
- **In Application Development Tools**

In HTML Documents

You can control the Oracle Video ActiveX Control in an HTML document using JScript and VBScript.

- You can get the highest level of control in Microsoft's Internet Explorer using VBScript. VBScript allows you to call the control's methods, handle any events, and modify and check the properties of the control using a subset of Visual Basic specific to Internet Explorer.
- Internet Explorer also provides JScript, which is similar to Netscape's JavaScript. Both of these are related to (but not identical to) the Java language.

You can find information on controlling the Oracle Video ActiveX Control in an HTML document using VBScript in "Using the Oracle Video ActiveX Control in HTML Documents" on page 4-5.

In Application Development Tools

Embedding the Oracle Video ActiveX Control in your own application is the same as with any ActiveX control. For example, in Visual Basic, you add the control as a component, where it appears as an icon. Either double-click the icon or select the control's icon on the toolbar and click and drag on the form. Once you've added it to your application, you can select it to make its property sheet appear. There, you can modify the control properties, add event handlers, and so on.

You can also reference the control's properties and methods from outside the control. Simply reference the control and method name, passing the appropriate parameters.

For more information on using the Oracle Video ActiveX Control in Visual Basic, see "Loading the Oracle Video ActiveX Control" on page 4-11 and "Programming with the Oracle Video ActiveX Control" on page 4-12.

Requirements

There are some basic requirements for creating client applications using the Oracle Video ActiveX Control:

- You need to be working in a development environment that supports ActiveX controls. This means Windows 95, 98 or Windows NT 4.0 or later.
- If you are creating a stand-alone application, you need an appropriate development tool such as Microsoft Visual Basic or Microsoft Visual C++.
- If you are creating an HTML document, you need an ActiveX-enabled browser, usually Microsoft Internet Explorer 3.0 or later, for testing and development purposes.

Using the Oracle Video ActiveX Control in HTML Documents

This section discusses how to insert the Oracle Video ActiveX Control into an HTML document, including:

- **Embedding the Oracle Video ActiveX Control**
- **Setting Properties for an Embedded Oracle Video ActiveX Control**
- **Security Requirements in Internet Explorer**
- **Sample ActiveX Control in HTML**

Embedding the Oracle Video ActiveX Control

To embed the Oracle Video ActiveX Control into an HTML document, you must use the **<object>** tag. This tag is recognized by Microsoft Internet Explorer and indicates that you want to use an ActiveX control to display media or somehow interact with the user.

To load the control, the main attributes to the **<object>** tag that you need to specify are:

- **ID**, which you need to specify only if you want to script the control, since you refer to the control in your code by this name
- **ClassID**, which specifies the control you want to load
- **Width** and **Height**

A typical object statement looks something like this:

```
<object ID="ovcax1" Width=356 Height=244
   ClassID="CLSID:547A04EF-4F00-11D1-9559-0020AFF4F597">
```

By default, a two-pixel border is included on each edge of the ActiveX Control video screen (but it can be turned off by setting BorderStyle=0), and the width and height values include both the control itself and the border. In this case, a border was specified, so this represents the actual width and height of the control plus the border. If you don't turn off the border, you need to add double the border width to the desired size of the control to get the actual width you need to set.

Also, if you include the optional control panel and status line, you need to take these into account when setting the height of the control.

- If you include the control panel (which includes the play / pause and stop buttons, the slider control and volume control), add 26 pixels to the height.
- If you include the status line, add 24 pixels to the height.

So, if you include both the control panel and the status line and turn on the border, add 54 pixels to the height.

You control the appearance of the control panel and status line by setting control properties, as discussed in the next section.

Note: It is important to set the height and width properly for the OVC ActiveX Control. If the height and width are not set correctly, the video is size has to be recalculated on playback, and that seriously affects performance. A video that normally plays at 30 frames per second will probably only playback at 16 frames per second if the height and width are not set correctly.

Setting Properties for an Embedded Oracle Video ActiveX Control

The **<object>** statement tells the browser which control you want to embed in the HTML document. The Oracle Video ActiveX Control doesn't take any of its start-up information from it, though. Instead it gets it from the **<param>** tags placed within the **<object>** block, that is, between the **<object>** and **</object>** tags. You need to specify information such as:

- Media file to load
- Whether to start playback automatically
- How to display the control's VCR controls

These elements are controlled using properties on the Oracle Video ActiveX Control. You can set any Oracle Video ActiveX Control property (except for those that are read-only) when embedding the Oracle Video ActiveX Control into an HTML document using **<param>** tags within the **<object>** block.

<param> tags have three elements:

- The **<param>** tag itself
- The property name, such as **AutoStart** or **Mediafile**:

  `name="`*`propName`*`"`

- The property value; for string or numeric properties, this should just be the string or value, while for boolean properties, you should specify **1** for true or on and **0** for false or off

  `value="`*`value`*`"`

For example, to set the video session URL, the **<param>** tag would look something like this:

```
<param name="Mediafile" value="vsi://server:5000/mds:/mds/video/video.mpi?data_
proto=tcp">
```

Any properties that you don't set through the use of **<param>** tags use the default value for that property.

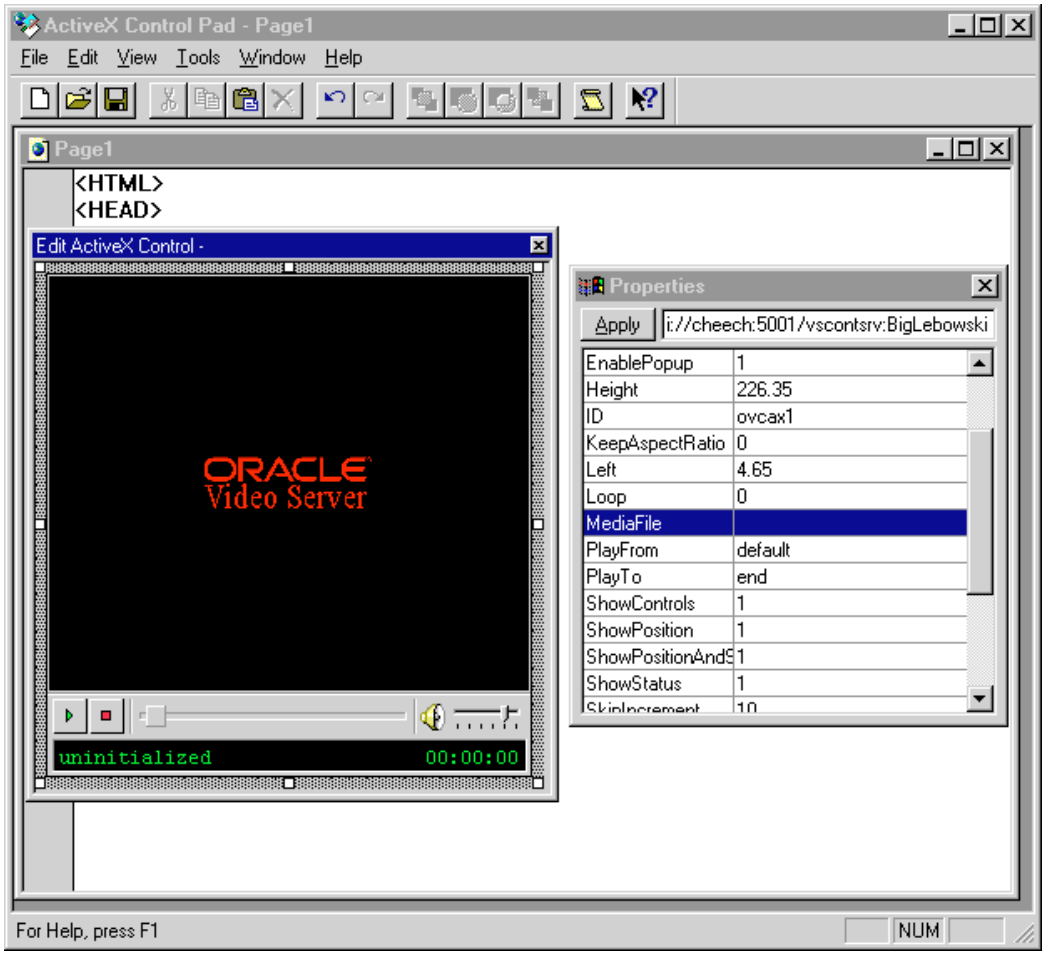
Using an HTML Tool, such as Microsoft FrontPage or ActiveX Control Pad

If you prefer using an HTML editor to create HTML pages, the editor may enable you to work with ActiveX Controls through the Graphical User Interface (GUI). Tools such as Microsoft FrontPage and Active Control Pad

(**http://www.microsoft.com/workshop/misc/cpad/default.asp**) enable you to manipulate the properties of the Oracle Video Client ActiveX Control.

First, insert the Oracle Video ActiveX Control, then set properties in a properties dialog box, as shown.

Figure 4–2 Editing the Oracle Video Client ActiveX Control Properties in ActiveX Control Pad



After setting the ActiveX Control properties, the tool generates the HTML code for the page that includes the control.

Security Requirements in Internet Explorer

Microsoft Internet Explorer enables local control of how the browser deals with different types content, such as Active content, Java, Javascript, and ActiveX scripts and ActiveX controls and plug-ins through "security zone safety level" options.

The zones for which you can set the safety levels are:

- Local Intranet sites
- Trusted sites
- Internet sites (all sites not listed as a Local, Trusted or Restricted site)
- Restricted sites

The security levels you can set for each zone are:

- **High** – potentially unsafe content is avoided. Unsigned controls are not allowed.
- **Medium** – you are notified before potentially unsafe content is downloaded. You must explicitly approve the use of unsigned controls.
- **Low** – all content is allowed without warning.
- **Custom** – fine tune security settings.

To enable the Oracle Video Client to play in Internet Explorer, the security settings for active content safety level to **Medium**, **Low**, or **Custom** (where the settings enable or prompt to allow ActiveX controls and signed or unsigned content):

- In Internet Explorer 3.0:
  1. Select **Options** on the **View** menu.
  2. Switch to the **Security** tab.
  3. Make sure the **Allow downloading of active content** and **Enable ActiveX controls and plug-ins** check boxes are checked.
  4. Click the **Safety Level** button.
  5. In the **Safety Level** dialog box, select either **Medium** or **Low**.

     Medium prompts you before downloading unsigned ActiveX controls.

     Low downloads the control without prompting.

     Which you select depends on your environment. If your users work exclusively in enterprise, intranet, or other closed environment situations,

you may just want to set the security level to **Low** on the assumption that users are safe from unsafe controls within the firewall. If there's a possibility that users may also go onto the Internet or some other uncontrolled area, you may want to set the security level to **Medium** so that users know when a site attempts to pass active content on to them.

6. Click **OK**, then **OK** again.

- In Internet Explorer 4.0:

  1. Select **Internet Options** on the **View** menu.
  2. Switch to the **Security** tab.
  3. Select the zone containing the Web server or server with HTML documents with the Oracle Video ActiveX Control embedded.

     This will probably be either **Local Intranet Zone** or the **Trusted Sites Zone**. For both of these zones, you can also explicitly specify a particular site.

  4. Select either **Medium** or **Low**.

     Medium prompts you before downloading unsigned ActiveX controls.

     Low downloads the control without prompting.

     If you trust an entire site, you should set this to **Low** to avoid having to approve the download each time you access a page containing the Oracle Video ActiveX Control.

Sample ActiveX Control in HTML

This sample shows a simple example of embedding the Oracle Video ActiveX Control into your HTML documents and controlling it through VBScript routines.

```
<html>
<head>
<title>New Page</title>
</head>

<body>
   <object ID="ovcax1" Width=352 Height=240
       ClassID="CLSID:547A04EF-4F00-11D1-9559-0020AFF4F597">
      <param name="Mediafile" value="vsi://server:5000/mds/video/oracle1.mpi?
        data_proto=tcp">
      <param name="ShowControls" value="1">
      <param name="AutoStart" value="0">
      <param name="BorderStyle" value="1">
```

```
      <param name="Loop" value="0">
      <param name="EnablePopup" value="1">
      <param name="EnableLeftClick" value="1">
      <param name="PlayFrom" value="beginning">
      <param name="PlayTo" value="end">
      <param name="ShowPositionAndStatus" value="1">
      <param name="TimerFrequency" value="500">
   </object>

   <form name="Button1">
      <input type=Button value="Play" onClick="ovcax1.Play()">
   </form>
</body>
</html>
```

Automatic Upgrade/Installation Detection

The OVC ActiveX Control supports automatic upgrade/installation detection in Internet Explorer via the Internet Component Download Model.

For more information on the mechanism for automatically installing or upgrading OVC, see “Deploying OVC to End Users via the Web” on page 5-8.

Creating Applications with the Oracle Video ActiveX Control

This section discusses how to create stand-alone applications in Visual Basic, using the Oracle Video ActiveX Control as a component to provide streaming media capabilities. There are three main topics in this section:

- **Loading the Oracle Video ActiveX Control**
- **Programming with the Oracle Video ActiveX Control**
- **A Simple Application**

Loading the Oracle Video ActiveX Control

Before you can use the Oracle Video ActiveX Control, it must be available to your application. To see if the control is available, start Microsoft Visual Basic and verify that the Movie Tool icon (shown here) is visible. Look in the Toolbox (if you can’t find the Toolbox, choose the Toolbox command on the View menu).

If you can’t see the movie icon, follow these steps to add the icon.

Microsoft Visual Basic

When you begin a Visual Basic project, the movie icon is not visible in the Toolbox. You must load the custom control for each new project, as follows:

1. Choose **Components** from the **Project** menu.
2. Find the Oracle Video ActiveX Control in the list box on the **Controls** tab.
3. Check the box next to the Oracle Video ActiveX Control entry.
4. Click the **OK** button.

Programming with the Oracle Video ActiveX Control

Once you have loaded the control, you can use the Oracle Video ActiveX Control as you would any other item in the Visual Basic Toolbox — select the Movie Tool icon and add the Oracle Video ActiveX Control to your application.

> **Note:** You can have only one active Oracle Video ActiveX Control in any one application.

- Set the properties of the control dynamically at design time using the Oracle Video ActiveX Control's property sheet.
- Add code to handle any of the events defined for the control just as you would with any other ActiveX control.
- Create buttons to execute any of the control's methods. For example, create your own control set, such as play and pause, instead of the built-in controls.

A Simple Application

This tutorial is designed to guide you through creating a simple Oracle Video Client application in Visual Basic. The finished application lets users view, select, and play media files from a local hard disk or from a video server on the network. Use the finished application, shown in Figure 4–3, as a starting point for developing your own applications.

Figure 4–3 The Finished Application

1. Add the Oracle Video ActiveX Control object to your application.

   For Visual Basic:

   a. Double click the Movie Tool icon in the Toolbox to add it to your project. The default name of an Oracle Video ActiveX Control object is **ovcax1**.

   b. Alternatively you can click the Movie Tool icon, then drag in your form. This allows you to easily customize the size of the control.

   Note: When you first open the Oracle Video ActiveX Control, you might not be able to see the controls if the initial size of the object is too small. If necessary, click on the control and drag the corners to resize it in the application window.

   ---

2. If you can't see the VCR controls on the Oracle Video ActiveX Control, set the **ShowControls** property to true (1).

3. Add a button labeled **Local Videos**. This example uses the **ImportFileAs()** method to bring up a list of videos available from the hard disk. For more information on this method, see "ImportFileAs()" on page C-9.

For Visual Basic:

a. Double-click on the **CommandButton** icon in the Toolbox. A command button appears on the form. If the command button becomes deselected, click this button to reselect it.

b. Change the Caption property to **Local Videos**.

c. Double-click on the **Local Videos** button. This opens the script window and places the cursor in the **Command1_Click()** method.

d. Type the following in the script window:

```
ovcax1.ImportFileAs
```

The **ImportFileAs()** method opens a dialog box that lets the user browse for a media file stored locally.

e. Close the script window.

4. Add a button labeled **Server Videos**. When clicked, this button calls the **ImportStreamAs()** method, which opens a dialog box that lets the user select from a list of videos available on the video server. For more information on this, see "ImportStreamAs()" on page C-10.

For Visual Basic:

a. Add a button to the form, as described in **Step 3**.

b. Change the **Caption** property to **Server Videos**.

c. Double-click on the **Server Videos** button. This opens the script window and places the cursor in the **Command2_Click()** method.

d. Type the following in the script window:

```
ovcax1.ImportStreamAs("")
```

e. Close the script window

f. In the Windows Registry, set the OVM.query_proto preference to **omn**. This step is necessary for the procedure to work because ImportStreamAs is disabled by default. For information about setting OVC preferences, see "OVC Preferences" on page 5-5.

5. Finally, add a **Quit** button to the form.

For Visual Basic:

a. Add a command button to the form.

b. Change the **Caption** property to **Quit**.

c. Double-click on the **Quit** button. This opens the script window and places the cursor in the **Command3_Click()** method.

d. Type the following in the script window:

```
End
```

e. Close the script window.

6. Run the application.

   - In Visual Basic, from the Run menu, choose **Start.**

You can now select and play videos from a local hard disk or a networked video server, and can use the play, pause, stop, controls when viewing the video.

You can also use the slider control to seek to any location in the video. If the slider control is used while viewing a video content file, the video frame is not updated until play is resumed.

This is all you need to do to use video in your application. You can also use the features of the Oracle Video ActiveX Control to create much richer applications. For example, you can use the **Play()**, **Pause()**, **Resume()**, **SetPos()**, and **Stop()** methods to create your own start, pause, seek, and stop buttons.

5

Customizing, Deploying and Troubleshooting OVC

This chapter is intended for the administrators and developers responsible for customizing, deploying OVC to end users, and troubleshooting the OVC automatic installation or upgrade mechanism. This chapter covers:

- Customizing client attributes, such as local error messages
- Managing the built-in support and automatic upgrade facilities provided with OVC
- Configuring the end-user installation of the Oracle Video Client
- Troubleshooting problems with the customization and automatic upgrade facilities

This chapter contains these sections:

- **Customizing OVC using the OVC Configuration and Installation Plug-in**
- **Deploying OVC to End Users via the Web**
- **Troubleshooting**

Customizing OVC using the OVC Configuration and Installation Plug-in

Note: The OVC Configuration and Installation Plug-in works only if it is present on the end user's machine, and it is present only if OVC version 3.0.5 or later is installed on that machine.

Background

The OVC Configuration and Installation Plug-in (**npovci.dll**) enables you to define persistent entries in the Windows Registry for OVC using the HTML **<EMBED>** tag in a Web page. This is useful for managing unusual features of the Client, such as for defining local error messages, which are error messages specific to your organization.

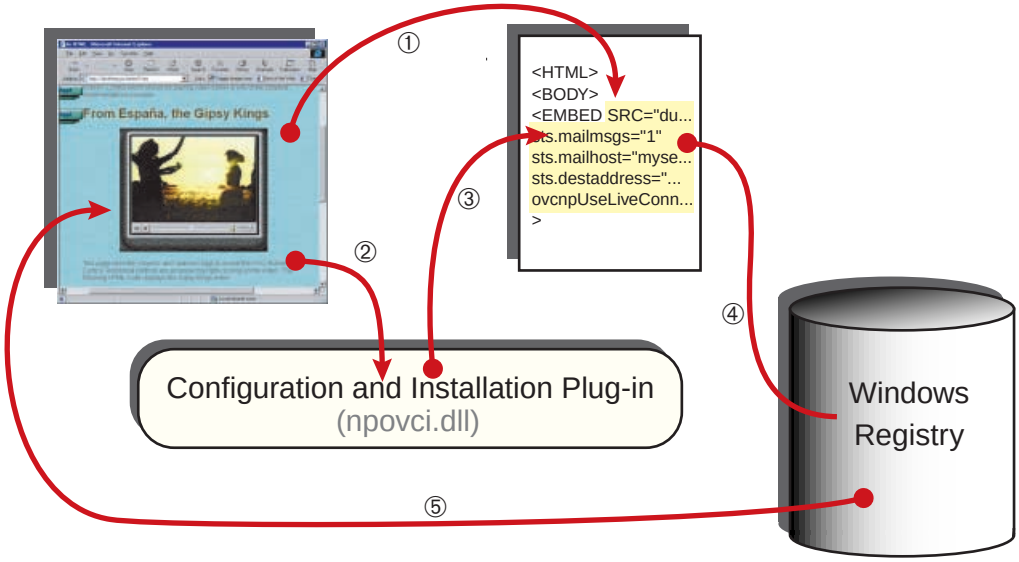
Persistent vs. Per-Instance

Customizing OVC via the OVC Configuration and Installation Plug-in is called *persistent* customization. The entries added to the Windows Registry will remain there unless they are overwritten with different values, or removed manually. Persistent customization differs from the normal *per-instance* method of customizing OVC, via the parameters to the HTML **<EMBED>** tag for the OVC Web Plug-in, and the **<OBJECT>** tag for the OVC ActiveX Control. (The OVC ActiveX Control can also be customized using the control properties within applications that support ActiveX Controls, such Visual Basic and Visual C++.) Per-instance customization is covered in Chapter 2, "Oracle Video Web Plug-in" and Chapter 4, "Oracle Video ActiveX Control".

Overview

The way the OVC Configuration and Installation works is summarized in the following picture and described after the picture.

Figure 5–1 The OVC Configuration and Installation Plug-in Updating the Registry



① If an HTML **<EMBED>** tag includes **TYPE="application/x-oracle-video-registry"** (for Netscape 3.0+ browsers) or **SRC="*dummy*.ovi"** (for Netscape and Microsoft browsers, version 3.0+), or both, ② the browser launches the Configuration and Installation Plug-in. If the SRC tag is used, the Configuration and Installation Plug-in launches because it is associated with files with the extension **.ovi**. If the TYPE tag is used, Netscape browsers recognize the TYPE definition as a MIME type. See "Specifying the Video Session URL and MIME Type" on page 2-9 in for more information on these tags.

③ & ④ After launching, the Configuration and Installation Plug-in updates Registry settings from the parameter lists that are also included in the HTML **<EMBED>** tag. ⑤ Then, when OVC runs, it utilizes settings in the Registry as appropriate.

Example 5–1 Disabling LiveConnect with the Configuration and Installation Plug-in

For example, to disable LiveConnect, the **<EMBED>** string might look like this:

```
<EMBED type="application/x-oracle-video-registry" ovcnp.useLiveConnect=0>
```

When a user goes to a Web page that includes either the TYPE tags or the SRC tag, the Configuration and Installation plug-in sets the Registry key **HKEY_CURRENT_USER\Software\Oracle\OVC\ovcnp\useLiveConnect** to the

value **0** or *off*. (For security reasons, only Registry entries under **HKEY_CURRENT_USER\Software\Oracle\OVC** can be set using this plug-in.)

The parameter in the **<EMBED>** statement after the SRC or TYPE definition (in this example, **ovcnp.useLiveConnect**) is an *OVC Preference*, which the Configuration and Installation Plug-in maps to a Registry key, converting the periods to backslashes.

Note: The ***dummy*.ovi** file should be on the web server, and the MIME type should also be defined on the web server. The name of the "dummy" file can be anything, as long as the extension is **.ovi**.

The MIME type for **.ovi** files should be set to **application/x-oracle-video-registry**.

The MIME type for **.mpi** files should be set to **application/oracle-video**.

Note: Some browsers can't handle empty or "dummy" files specified by the SRC attribute that contain no data (the file size is 0 bytes). Include some data in the file referenced by the SRC attribute. The contents of the file do not matter, as long as it contains something.

Persistent But NOT Immediate

Custom values added to the Registry are persistent, but they are not immediate. If you create a web page that contains an OVC Web Plug-in configured to play a video and containing updates to the Registry, do not expect those Registry values to be applied to the plug-in on the current page.

To be certain that the Registry entries are used for a specified instance of the OVC Web Plug-in when playing video, place them on a page that must be accessed *before* the page that contains the OVC Web Plug-in.

Also, every time OVC reinstalls, all previous custom Registry settings are erased. So, Oracle recommends that you create one page that users access to update OVC. After a reboot, users should be directed to a page that updates the Registry. Then they can be go to pages containing content, or streaming video data.

OVC Preferences

The following table lists OVC preferences.

Table 5–1 OVC Configuration and Installation Plug-in Preferences

OVC Preference	Description	Value (default in bold)
ovcnp.useLiveConnect	Disables / enables LiveConnect. Disabling LiveConnect avoids the cost of initializing the Netscape Java Virtual Machine and LiveConnect, resulting in reduced OVC start-up time. If you are using Javascript or Java to control the OVC Web Plug-in, LiveConnect must be enabled.	0 (off) or **1**
ovi.realtime	Disables / enables OVC to run as a high priority process, or in real-time mode. Disabling real-time mode frees CPU resources so end users can use other applications while still watching streaming video on OVC.	0 or **1**
ovc.reportconfig	[One time event] If all other required error message reporting preferences are set, setting this to true generates an error report titled OVC-315 even when there is no error, if: ▪ the notice hasn't been generated previously. ▪ it is not during the install.	**0** or 1
ovc.reportnotice	If all other required error message reporting preferences are set, setting this to true generates an error report titled OVC-316 every time the web page with this Preference set to true is loaded.	**0** or 1
sts.genericmsgs	Turns on / off the utilization of the generic messages in the local message file. The generic messages are messages 250 through 0255.	**0** or 1
sts.ovc-*xxxx*	The replacement or overriding message for a specified error, where *xxxx* is the message number to override.	"*call %s failed, contact support at 555-9872"* or whatever the new message should be.
sts.mailmsgs	Turns on / off sending errors as email messages.	**0** or 1
sts.mailhost	The hostname or IP address of the outgoing mail (SMTP) server.	"*server.domain.com"*

Table 5–1 OVC Configuration and Installation Plug-in Preferences (Cont.)

OVC Preference	Description	Value (default in bold)
sts.smtpport	The IP port to contact the SMTP mail daemon on.	**25**
sts.destaddr	The address of the recipient of the error messages to be sent by email.	"*support@server.com*"
sts.mailexpiresecs	Number of seconds to store messages. If messages are older than this, they are deleted. (96400*3 = 3 days of seconds = 289200)	**289200**
sts.mailqueuelength	Number of messages to spool in the mail queue.	**10**
sts.tracing	Enable the trace file mechanism for all current logging.	0 or **1**
sts.mailtrace	Include trace file data in email error reports.	**0** or 1
sts.ovccdbg	Include Config Checker debug information in email error reports.	**0** or 1
sts.ovccsys	Include Config Checker system variables in email error reports.	0 or **1**
sts.cardinfo	Include hardware decoder card information in email error reports.	0 or **1**
sts.dbwin	Enable debug console output for dbwin32 and other debugging consoles.	**0** or 1
ovasrc.trace	Set the log level for source filter events.	0 - 4, where 4 = debug
ovm.log-filter	Log filter specification to use. Notation: "maxsev #", for example **maxsev 6**	0 - 7, where 7 = debug
ovi.loglevel	Log level for OVI events. The default is **6**.	0 - 7, where 7 = debug
ovm.linequal	Defines the overfeeding rate, where 10000 = none and 9700 = 3%.	**9700** - *xxxxx*
ovm.inittimeout	Number of milliseconds to wait for initialization data. Error message 55.	**60000** -*xxxx*
ovm.networktimeout	Number of milliseconds to wait before displaying the "Stand By" network congestion dialog.	**5000** - *xxxx*
ovm.usecustomaddress	Toggles the use of a custom client address set via **ovccfg**.	**0** or 1

Table 5–1 OVC Configuration and Installation Plug-in Preferences (Cont.)

OVC Preference	Description	Value (default in bold)
ovm.client	The client address to use for DSM. This should be used only for testing or with the ATM protocol.	"*address*[:*port*]"
ovm.query_proto	Specifies the communication protocol to query the server for content. Note that for the 3.2 release of OVC, using the default value of rtsp will return error 46.	**rtsp** or omn
ovm.comm_proto	Specifies the communication protocol that Oracle Video Client uses with the server.	**rtsp**, omn, resolve
ovm.data_proto	Specifies the network protocol used to transmit the video stream.	**udp**, tcp, aal5
ovm.startrate	Defines the bursting rate of playback at startup, where 10000 = none and 900 0= 11%.	9000 - *xxxx*
ovm.droptimeout	Number of msecs to wait during a network dropout. Error message 54.	60000 - *xxxx*
ovm.usetimeout	Defines whether to time out after no data has been received for a long time (ovm.inittimeout / ovm.droptimeout).	**0** for multicasts **1** for interactive sessions

For example, to enable email messages for errors, **sts.mailmsgs**, **sts.mailhost**, and **sts.destaddr** must all be set to valid values. The email messages sent to the value in **sts.destaddr** will contain a specific error code, even if generic messages are displayed on the Client.

Example 5–2 Enabling Email Messages using OVC Preferences

For example, using the Configuration and Installation Plug-in

```
<EMBED SRC="dummy.ovi" sts.mailmsgs=1
 sts.destaddr="support@mycompany.com"
 sts.mailhost="mycomputer.mycompany.com">
```

turns on OVC error mail messages, sending them to ***support@mycompany*.com** via the mail server ***mycomputer.mycompany*.com**.

> **Note:** To reduce dependencies on the Domain Name Service (DNS) and blocking DNS time-outs on computers, machines should be referred to by IP address, not hostname, where possible.

Deploying OVC to End Users via the Web

Overview

Developers responsible for deploying the Oracle Video Client to end users can create a link or web page to install or upgrade the Oracle Video Client. Currently, Oracle supports four methods for installing OVC on end users' machines via a web page. Those four methods are:

- Using the OVC Configuration and Installation Plug-in
- Using the Internet Component Download Model
- Using SmartUpdate with Netscape Communicator
- Using a link to download the installation executable (manual method)

Each of these methods are described in detail in "The Supported Web-based Installation / Upgrade Methods" on page 5-11.

In each of these methods, you need to create a web page for either installing or upgrading OVC on end users' machines. In the first three methods, OVC is automatically upgraded — there is little end user interaction, and no active decisions need to be made (just agreeing to various dialogs: certificate acceptance, license agreements, etc.). In the second and third method, OVC can also be automatically installed, if it was not installed previously. The last method can be used as a backup in case none of the other methods are applicable to the configuration. The first method utilizes the OVC Configuration and Installation Plug-in and works with all supported browsers. However, it also *requires* that the OVC 3.0.5+ is already installed on the end user's machine.

The following table describes the four methods for automatically installing or upgrading OVC supported by Oracle.

Table 5–2 Four End User Web-based Installation Methods

	OVC Configuration and Installation Plug-in	Internet Component Download	Netscape's SmartUpdate	Manual
Which browser?	Any IE3+ / Netscape 3+ browser	Internet Explorer 3+	Only Netscape Communicator 4+	All
Which HTML statement?	**<EMBED ...**	**<OBJECT ...**	**<SCRIPT ...** (Javascript)	**<A HREF ...** (link) or **<INPUT TYPE="SUBMIT" ...** (various methods)
Which download file?[1]	**ovces.exe**	**ovce.exe**	**ovce.jar**	**ovce.exe**
Security Certificate?	Yes	Yes	Yes	Certificate not checked
Pros	■ Digital Security Certificate ■ Automatically installs if OVC previously installed ■ Works with all supported browsers (IE3+ / Netscape 3+) ■ Oracle can enhance this update method	■ Digital Security Certificate ■ Integration with OVC ActiveX Control ■ Automatically installs OVC ■ OVC does not have to be previously installed	■ Digital Security Certificate ■ Automatically installs OVC ■ OVC does not have to be previously installed	■ Works for all supported browsers ■ OVC does not have to be previously installed
Cons	■ OVC 3.0.5+ must be previously installed	■ Only works with IE3+	■ Only works with Netscape 4+ ■ Javascript and JAR Information Manager	■ Not Automatic ■ No Security ■ End user must locate the downloaded file and run it
Why use this method?	■ OVC is previously installed, then this works for all supported browsers ■ Oracle Preferred Method	■ Using Internet Explorer and OVC ActiveX Control, this is integrated ■ Not certain if OVC is already installed	■ Using Netscape Communicator 4+ ■ Not certain if OVC is already installed	■ If not certain what browser end users will use ■ This can always be a fall back when other methods fail

[1] **These are the default file names as delivered on the CD-ROM. You can rename the prefix part of the name to whatever you want.**

The next two sections describe the installation files included on the OVC CD-ROM and give an overview of how to specify the OVC version.

The Installation Files

Three installation programs are included on the OVC CD-ROM (in the **runtimes** directory):

- **ovces.exe**, which performs the OVC end user installation and is intended for use with the OVC Configuration and Installation Plug-in method of automatic upgrading as described in "Upgrading OVC Using the OVC Configuration and Installation Plug-in" on page 5-11. This installation program is called from within an HTML **<EMBED>** tag. It can be used with all supported browsers.
- **ovce.exe**, which performs the OVC end user installation for the Component Download Model as described in "Upgrading/Installing OVC Using the Internet Component Download Mechanism" on page 5-13 (it contains a digital certificate for use with the Component Download Model). This installation program should be called from within an HTML **<OBJECT>** tag.
- **ovce.jar**, which is used with Netscape's SmartUpdate feature, as described in "Upgrading/Installing OVC Using SmartUpdate with Netscape Communicator" on page 5-14.

All these files include an Oracle digital certificate, which is a security feature that gives you reassurance that the software you are installing is an Oracle product.

The filenames are the names used by Oracle. You can rename them to whatever you want, just keep the extensions the same.

Either of the two executable files can be used for manually installing OVC.

The Version String

Oracle uses a five-digit, *period-separated* version number with all of its products, including OVC. So, version 3.2 of OVC is really 3.2.0.0.0.

However, the SmartUpdate and Internet Component Download automatic upgrade/installation methods use a four-digit, *comma-separated* version number. So, for those two mechanisms, the five-digit OVC version number must be converted to a four-digit number. Do this by combining the fourth and fifth digits into one, thus:

The fourth digit of the four-digit notation is the sum of the fourth digit times 100 plus the fifth digit.

$$\textit{fourth digit of four-digit notation} = \textit{digit4} \times 100 + \textit{digit5}$$

So, if the OVC version number was 3.2.0.5.2, the four-digit notation would be 3,2,0,502.

The Oracle Configuration and Installation Plug-in method supports either the four-digit string or the five-digit string.

The Supported Web-based Installation/Upgrade Methods

> **Note:** Each of the mechanisms described below will upgrade or install OVC as described. When OVC is installed or updated, previous custom OVC Registry entries are erased. Make sure that you direct end users to a page that updates the Registry after their machines reboot, and before they can view streaming video.

The following sections discuss the supported web-based methods for installing and/or automatically upgrading OVC on end user machines:

- Upgrading OVC Using the OVC Configuration and Installation Plug-in
- Upgrading/Installing OVC Using the Internet Component Download Mechanism
- Upgrading/Installing OVC Using SmartUpdate with Netscape Communicator
- Upgrading/Installing OVC Manually

Upgrading OVC Using the OVC Configuration and Installation Plug-in

"Customizing OVC using the OVC Configuration and Installation Plug-in" on page 5-2 describes one purpose for the OVC Configuration and Installation Plug-in — namely, its "configuration" capabilities. As the name of the plug-in implies, it also supports an automatic mechanism for upgrading OVC on end user machines — its "installation" capabilities. This section describes the method of using this plug-in to upgrade OVC to the latest version available.

This installation method requires that the Configuration and Installation Plug-in is already installed on the end user's machine, and that is true if and only if OVC version 3.0.5 or later is currently installed.

In the **<BODY>** of an HTML page, include an **<EMBED>** statement, along with the **SRC=*dummy*.ovi** string, to invoke the Configuration and Installation Plug-in.

```
<EMBED SRC="dummy.ovi" ...
```

When the browser loads the web page containing this **<EMBED>** statement, it launches the Configuration and Installation Plug-in, which then checks the parameters in the **<EMBED>** statement. If an OVC.CODEBASE URL parameter is

included, it then checks the version number specified after the number sign (**#**) and compares that number to the version number of the currently installed version of OVC.

Example 5–3 Updating OVC with the <EMBED> Tag and OVC CODEBASE Parameter

```
<EMBED SRC="dummy.ovi"
 OVC.CODEBASE="http://server/directorypath/ovces.exe#3,2,0,0">
```

If the currently installed version of the Plug-in does not match, then the Configuration and Installation Plug-in forces the browser to download the new version of OVC from the server specified in the OVC.CODEBASE URL and install it.

Note: This mechanism only works if the Oracle Configuration and Installation Plug-in has been installed previously. It does NOT upgrade from previous versions of OVC older than 3.0.5.0.3. Such installations require a different mechanism.

Note: Some browsers can't handle empty or "dummy" files of zero length (files that contain no data) specified by the SRC attribute. Include some data in the file reference by the SRC attribute.

To load a new URL before launching the installation executable, insert the OVC.UPDATEURL parameter in the **<EMBED>** tag with a valid URL to a new HTML page, as shown in the following example.

Example 5–4 The OVC UPDATEURL and CODEBASE Parameters

```
<EMBED SRC="dummy.ovi"
 OVC.CODEBASE="http://server/directorypath/ovces.exe#3,2,0,0"
 OVC.UPDATEURL="http://server/directorypath/newpage.html">
```

When the update URL is specified, the Plug-in instructs the browser to go to the new page, and then it launches the installation executable. When the browser goes

to a new page that does not contain OVC components, it (the browser) unloads the OVC Plug-in.

> **Note:** Microsoft DirectShow 6.0 must be installed for the Oracle Video Client to work. You can obtain the DirectShow 6.0 from Microsoft at **http://www.microsoft.com/directx/download.asp**.

Upgrading/Installing OVC Using the Internet Component Download Mechanism

The Internet Component Download mechanism works with the Internet Explorer browser version 3.0+, so to use this method to automatically install or upgrade OVC, be certain that end users will be using the Internet Explorer browser.

For more information on the Internet Component Download mechanism, see **http://www.microsoft.com/workshop/delivery/download/icd.asp.**

The Oracle Video Client ActiveX Control can be embedded in an HTML using the HTML **<OBJECT>** tag (see "Using the Oracle Video ActiveX Control in HTML Documents" in Chapter 4). Even if you insert the OVC Active X Control using a GUI-based tool, such as Microsoft FrontPage, the HTML generated by that tool simply generates the appropriate HTML **<OBJECT>** tag.

The following example is an excerpt of HTML code that shows an embedded OVC ActiveX Control.

Example 5–5 The OVC ActiveX Control <OBJECT> Tag

```
<body>
   <object ID="ovcax1" Width=352 Height=240
       ClassID="CLSID:547A04EF-4F00-11D1-9559-0020AFF4F597">
      <param name="Mediafile"
       value="vsi://server:5000/mds/video/oracle1.mpi?data_proto=tcp">
       etc.
   </object>
```

The Internet Component Download mechanism supports a CODEBASE URL parameter which is used to check the current version of the ActiveX Control against a version specified, and if the version specified is newer, the Internet Explorer browser launches the executable file specified as part of the CODEBASE URL.

To enable automatic update detection, insert the CODEBASE URL parameter with the **version** string in the object tag, as shown is the following example.

Example 5–6 The OVC ActiveX Control <OBJECT> Tag with the CODEBASE URL

```
<body>
   <object ID="ovcax1" Width=352 Height=240
      ClassID="CLSID:547A04EF-4F00-11D1-9559-0020AFF4F597"
      CodeBase="http://server/directorypath/ovce.exe#version=3,2,0,0">
      <param name="Mediafile" value="vstcp://server:5000/mds/video/oracle1.mpi">
      etc.
   </object>
```

The browser compares the version of the OVC installed on the local machine with the version listed in the **version** string (in this case, **3,2,0,0**). If the installed version is older, the installation program listed in the CODEBASE URL (in this case, **http://*server/directorypath*/ovce.exe**) installs the newer version of OVC. If OVC is not already installed, it is installed automatically. To always force the installation, the version number to use is -1,-1,-1,-1.

Upgrading/Installing OVC Using SmartUpdate with Netscape Communicator

The Netscape SmartUpdate mechanism works with the Netscape Communicator version 4.0+ to enable the automatic installation or upgrade of a software plug-in, in this case, the Oracle Video Client.

SmartUpdate is a Netscape name for the process of what happens when a plug-in is "automatically updated," but the means for performing the automatic upgrade/installation is embedded Javascript on a web page that references a Java Archive, or JAR file. Netscape calls this mechanism the JAR Information Manager. For more information, see **http://developer.netscape.com/docs/manuals/communicator/jarforcd/index.htm** or **http://developer.netscape.com/docs/manuals/communicator/jarman/index.htm**.

In this installation/upgrade method, Oracle supplies a new plug-in installation executable and a signed security certificate bundled into a JAR file. Store the JAR file on a Web server, and create an HTML page that references the OVC Web JAR file and tests the version number within a JavaScript trigger program.

The following is an example of a short HTML page that uses JavaScript and SmartUpdate/the JAR Information Manager to determine whether to update the currently installed version of the OVC Web Plug-in.

Example 5–7 A JAR Information Manager "Trigger" Script to Update/Install OVC

```
<HTML>
<BODY>
<SCRIPT LANGUAGE="JavaScript">
```

```
var myPlugin = navigator.plugins["Oracle Video Client"]

if ( navigator.javaEnabled() ) {

 vi = new netscape.softupdate.VersionInfo(3,2,0,0);
 trigger = netscape.softupdate.Trigger;

  if ( trigger.UpdateEnabled() ) {
     if (myPlugin)
           trigger.ConditionalSoftwareUpdate(
           "http://server/directorypath/ovce.jar",
           "plugins/Oracle Video Client", vi, trigger.DEFAULT_MODE);
     else
            trigger.StartSoftwareUpdate(
           "http://server/directorypath/ovce.jar",
            trigger.DEFAULT_MODE);
    }
  else document.write("Enable SmartUpdate before running this script.");
  }
 else document.write("Enable Java before running this script.");
</SCRIPT>
</BODY>
</HTML>
```

Upgrading/Installing OVC Manually

You may opt to provide another method for installing the Oracle Video Client on end user machines, in addition to or instead of the methods described on the previous pages. For example, you might create a more robust solution, such as a Web page which tests to determine which browser is accessing your page, and whether OVC is already installed, and proceed accordingly.

Since there is no best upgrade/installation solution for installing OVC, and each method described so far has drawbacks, you may decide to provide an executable file as a link on a page so that end users can download the file and run the installation manually.

There are many ways to create such a page. If you need help implementing an upgrade/installation mechanism, contact Oracle Support Services as described in “Oracle Support Services” on page xvii.

Troubleshooting

If the Registry customization feature or the automatic upgrade/installation web page or pages are not working, check to see if the following things are correct.

MIME Type

The MIME type must be specified on the web server. Two important MIME types must be set:

- For **.mpi** files the MIME type should be **application/x-oracle-video**
- For **.ovi** files the MIME type should be **application/x-oracle-video-registry**

SRC and TYPE Parameters

Use either or both of these parameters within an HTML **<EMBED>** tag to invoke the OVC Configuration and Installation Plug-in.

SRC The SRC parameter must be set to equal a file with the extension **.ovi**, which should be a file of zero bytes that exists on the web server. To keep track of this file, place it in the directory or folder with the installation executable and/or jar files.

The SRC parameter should be set to a URL, such as **SRC="http://*myserver.mycompany*.com/OVC/*dummy*.ovi"**.

The SRC parameter works with all supported browsers.

TYPE To invoke the OVC Configuration and Installation Plug-in, the TYPE parameter should be set to **application/x-oracle-video-registry**, thus **TYPE="application/x-oracle-video-registry"**.

The TYPE parameter works with Netscape 3+ browsers and Internet Explorer 4+ browsers.

See “Specifying the Video Session URL and MIME Type” on page 2-9 for more information.

“Dummy” File

When using the SRC parameter with the <EMBED> tag, create a file with the extension **.ovi** on the web server. The file should be greater than zero bytes in size (it should contain something). The location specified within the <EMBED> tag should be correct. Set it to a URL, such as **SRC="http://*myserver.mycompany*.com/OVC/*dummy*.ovi"**.

Version String

The version string should be a five-digit, *period-separated* version number, such as 3.2.0.0.0, or a a four-digit, *comma-separated* version number, such as 3,2,0,0.

The five-digit version number and the four-digit version number both work with the OVC Configuration and Installation Plug-in. The version number is listed within an HTML **<EMBED>** tag at the end of the OVC CODEBASE URL, after a **#** sign, such as **OVC.CODEBASE="http://server/directorypath/ovces.exe#3,2,0,0"** .

The four-digit number works with the SmartUpdate and Internet Component Download automatic upgrade / installation methods. The version number is listed within an HTML **<OBJECT>** tag at the end of the CODEBASE URL, after **#version=**, such as **CodeBase="http://*server*/*directorypath*/ovce.exe#version=3,2,0,0"**.

You can force an install every time by using a version number that is much higher than the real version number, or by using -1 for each digit, thus -1,-1,-1,-1.

So, make sure version number, the version number syntax, and the CODEBASE syntax are correct.

See “The Version String” on page 5-10 for more information on version strings.

Executable Files

Make sure you use the correct executable installation file. You can rename them, but as delivered by Oracle, the executable **ovces.exe** is for use with the OVC Configuration and Installation Plug-in and **ovce.exe** is for the Internet Component Download model.

See “The Installation Files” on page 5-10 for more information on the executable files.

For support contact information, see “Oracle Support Services” on page xvii.

A

Oracle Video Web Plug-in Reference

This appendix contains the reference information for the Oracle Video Web Plug-in. This plug-in allows you to embed the Oracle Video Client in applications that support the Netscape plug-in standard. For details on how to embed the plug-in in an HTML page and other tasks, see Chapter 2, "Oracle Video Web Plug-in".

This appendix contains the following information:

- **<Embed> Attributes**
- **JavaScript Methods**
- **Java Classes**

<Embed> Attributes

This section lists all of the available **<embed>** statement attributes recognized by the Oracle Video Web Plug-in. The following attributes are available:

autoStart	**leftClick**	**skipIncrement**
background	**loop**	**sliderRate**
controls	**mediafile**	**src**
controlMask	**name**	**toolTips**
height	**playFrom**	**type**
hidden	**playTo**	**volume**
keepAspectRatio	**popupMenu**	**width**

autoStart

Syntax: autoStart="true | **false**"

Specifies whether the plug-in begins playing as soon as the player has completed loading the stream.

If true, the stream starts as soon as the page is opened. The default value is **false**.

background

Syntax: background="*#rrggbb*"

Specifies a background color to be displayed behind the plug-in area. This uses the standard HTML format for specifying RGB color values. This background can't be seen when the plug-in video screen is displayed.

controls

Syntax: controls="true | false"

Specifies whether the player displays the VCR-style controller. The controller consists of:

- a push button that toggles between the Play and Pause icon
- a stop button
- a seek slider indicating the current stream position; you can change the stream position by dragging the slider to another position
- a volume slider indicating the current volume; you can change the volume by dragging the slider to another position

If **controls** is true, the plug-in displays the controls specified by the **controlMask** parameter. The default value is **false**.

Figure A–1 shows the Oracle Video Web Plug-in with both the controller and the status line visible.

Figure A–1 controls=true and controlMask="controller+statusline"

controlMask

Syntax: `controlMask="controller[+statusline]"`

Selects which controls appear in the plug-in. The default is the controller only. This requires you to set **controls** to true, otherwise no controls show up:

- **controls=true** displays the controller only
- **controls=true** and **controlMask="controller+statusline"** displays both the controller and the status line
- **controls=true** and **controlMask="statusline"** displays the status line only

height

Syntax: height=*nnnn*

Required. Specifies the height of the plug-in in pixels. If the **controls** and **border** attributes are not set, then this indicates the actual height of the plug-in. If the **border** attribute is set, then:

1. Multiply the border thickness times two.
2. Add this to the height that you want the video screen.
3. Use this number for **height**.

If **controls** is turned on:

1. If the controller is displayed, add 25 pixels.
2. If the status line is displayed, add 24 pixels.

For example, if you want a video screen that is 200 pixels high with a 2-pixel thick border (the default) and the controller displayed, multiply the border times two to get 4, add this to the desired width of the screen, 200, then add 25 pixels for the controller. This gives you a height of 229 pixels.

hidden

Syntax: hidden="true | **false**"

Specifies whether the plug-in is displayed on the page or not. If true, the plug-in is not displayed, regardless of the value of **height** and **width** attributes. The default value is **true**.

keepAspectRatio

Syntax: keepAspectRatio="**true** | false"

Specifies whether the plug-in will maintain the aspect ratio (the height and width ratio) for the content to be played, even if the Client is resized. If **true**, OVC maintains the correct aspect ratio for the movie when the window is resized. This may result in movies appearing with black rectangles on the sides or on the top and bottom (letterboxing). For commercial deployment, setting this value to **true** eliminates horizontal and vertical distortion of the video image. The default value is **false,** which causes OVC to scale the image to fit the entire window. This may cause the image to become distorted.

leftClick

Syntax: `leftClick="`**`true`** `| false"`

Specifies whether a left click on the video screen toggles Play and Pause. If **leftClick** is **true**, left clicking in the video screen has the same effect as clicking on the Play / Pause button. The default value is **true**.

loop

Syntax: `loop="true |` **`false`**`"`

Specifies whether the player loops the current stream by starting back at the beginning whenever it reaches the end of stream. The default value is **false**.

If you set the **playFrom** and **playTo** attributes or started playback by calling the **play()** method with **to** and **from** parameters, the stream returns to the **playFrom** position when it reaches the **playTo** position.

mediafile

Syntax: `mediafile="`*`vsurl`*`"`

Required. Specifies the protocol, server, and media file to play. You must use the this attribute in conjunction with either the **src** or **type** attribute.

For information on how these attributes work together, see "Specifying the Video Session URL and MIME Type" on page 2-9. For information on video session URLs, see Appendix D, "Oracle Video Session URLs".

name

Syntax: `name="`*`plugin_name`*`"`

Required (if you want to call methods on the plug-in from a JavaScript function or a Java applet). Declares the public name for the plug-in instance.

In the following example, the plug-in instance is named **video1**:

```
<embed type="application/oracle-video" name="video1" width=352
height=240 mediafile="vsi//sun:5000/mds/video/oracle1.mpi"?data_proto=tcp>
```

A JavaScript call to the plug-in would look like this:

```
document.video1.play();
```

playFrom

Syntax: playFrom=**default** | beginning | end | *wallclock* | *hh:mm:ss:cc* |*millisecs*

Specifies a start time for playback. If **loop** is true, then playback loops back to the point specified by this attribute; the default is to play from the beginning.

To specify a start time, use one of these formats:

- The predefined string values **default**, **beginning** or **end**
- A string representing wall clock time and date, formatted as follows:

  wallclock = *time* | *date time* | *date time zone*

  where

  time = *hh*:*mm*[:*ss*]
  date = *dd month yyyy*
  zone = +|-*hhmm* | UTC | GMT
  month = Jan | Feb | Mar | Apr | May | Jun | Jul | Aug | Sep | Oct | Nov | Dec

  Hours are specified as 0-23. If seconds are not set, they are set to zero.

  For example, **00:02:40** starts the stream two minutes, forty seconds from the beginning of the stream (160 seconds).

  For another example, **10 Jul 1999 22:30 UTC** starts the stream on the 10th of July at 10:30 PM Universal Time (which is the same as Greenwich Mean Time).

- The number of milliseconds from the beginning of the stream

  For example, **160000** is the millisecond equivalent of 00:02:40 above.

playTo

Syntax: playTo=beginning | **end** | *wallclock* | *hh:mm:ss:cc* |*millisecs*

Specifies a stop time for playback. If the **loop** attribute is false or not set, the plug-in stops the stream when it reaches this point. The default is to play until the end. If **loop** is true, the stream restarts at the **playFrom** position when the **playTo** position is reached.

To set a specified time use one of these formats:

- The predefined string values **beginning** or **end**
- A string representing wall clock time and date, formatted as shown in "playFrom" on page A-6.
- The number of milliseconds from the beginning of the stream
- No value is specified, which means to play indefinitely.

popupMenu

Syntax: `popupMenu="true | false"`

Specifies whether the pop-up menu appears when the user clicks the right mouse button on the player. This pop-up menu allows basic operations like play and pause, rewind to the beginning of the movie, forward to the end of the movie, and close. The default value is **true**.

skipIncrement

Syntax: `skipIncrement=`*`percentage`*

Specifies the amount the client skips backward or forward when you press the left or right arrow keys when the client is in full-screen mode. The option, **skipIncrement**, can be set as a percentage of the length of the video clip. Valid values are **0** to **100**. So, if the video is a 120-minute movie and the **SkipIncrement** value is set to **1**, seeking once forward will cause the video to advance 1 minute and 12 seconds (1 percent of 7200 seconds = 72 seconds). The default value for **SkipIncrement** is **10**.

sliderRate

Syntax: `sliderRate=`*`milliseconds`*

Specifies how often the seek slider updates, in milliseconds. This setting also specifies how often the time display in the status line updates (when the **OviObserver.onPositionChange()** method is called). The default value for **sliderRate** is **500** milliseconds

src

Syntax: src="*file*.mpi"

Required (if not used, **type** must be used). The **src** attribute is recognized by all browsers. Use the **src** attribute if your browser does not support the **type** attribute (for example, Microsoft Internet Explorer 3.x). The **src** attribute identifies the type of plug-in to load by the extension of the file specified. In the case of the Oracle Video Web Plug-in, this attribute doesn't identify the actual file to load, however: the plug-in actually gets that from the **mediafile** attribute.

Note that your HTTP server must be configured to recognize the **application/oracle-video** MIME type, which is associated with files with an extension of **.mpi**.

toolTips

Syntax: toolTips="true | **false**"

Indicates whether the plug-in displays tool tips when the user moves the mouse pointer over the plug-in area. The default is **false**.

type

Syntax: type="application/oracle-video"

Required (if not used, **src** must be used). Specifies the MIME type. In this case, it invokes the plug-in and prepares it to play an Oracle video file. The plug-in gets the name of the file to play from the **mediafile** attribute.

The only value you should specify for the **type** attribute is **application/oracle-video**, as shown.

> **Note:** The **type** attribute works in Netscape 3.0 or greater and Internet Explorer 4.0 or greater. This attribute is not supported in Internet Explorer 3.0 or other older browsers.

volume

Syntax: volume=*nnn*

Where *n* is a volume level (integer value) in the range 0-100. Normally, allow the user to set the volume with the volume slider. This attribute is useful if there are multiple clips on the same server that have varying base volume levels.

width

Syntax: `width=`*nnn*

Required. Specifies the width of the plug-in in pixels. If the **border** attribute is not set, then this indicates the actual width of the plug-in. If the **border** attribute is set, then:

1. Multiply the border width times two.
2. Add this to the width that you want the video screen.
3. Use this number for **width**.

For example, if you want a video screen that is 320 pixels wide with a 10-pixel thick border, multiply the border times two to get 20, add this to the desired width of the screen, 320, for a width of 340 pixels.

JavaScript Methods

From JavaScript, you can call all of the methods on the plug-in defined in the **OviPlayer** class, as described in "OviPlayer" on page A-11.

If you want to call any of these methods through the JavaScript LiveConnect interface, call the method through the plug-in's name. For example, you have a plug-in embedded into an HTML document with the following code:

```
<embed name="video1" height=320 width=200 src="oracle.mpi" mediafile="spud.mpi">
```

Call the **setLoop()** method for this instance of the plug-in like this:

```
document.video1.setLoop(true);
```

See "OviPlayer" on page A-11 for descriptions of available methods' syntax.

Java Classes

OVC provides two classes to facilitate communications between Java applets, JavaScript, and the Oracle Video Web Plug-in. These classes are:

- **OviPlayer**
- **OviObserver**

OviPlayer

This section describes the public methods in the **OviPlayer** Java class. For information on controlling the Oracle Video Web Plug-in from a Java applet, see "Retrieving the OviPlayer Object" on page 2-26. The following methods are available:

advise()	**pause()**	**statusCodeEOS()**
forward()	**play()**	**statusCodeError()**
getCreateTime()	**resume()**	**statusCodeInit()**
getLength()	**rewind()**	**statusCodePaused()**
getMaxPos()	**setAutoStart()**	**statusCodePlaying()**
getMinPos()	**setFullScreen()**	**statusCodePrepared()**
getObserver()	**setLoop()**	**statusCodeRealized()**
getPos()	**setObserver()**	**statusCodeUnInit()**
getState()	**setPopupMenu()**	**stop()**
getVol()	**setPos()**	**unload()**
load()	**setVol()**	

advise()

Syntax: `boolean advise(OviObserver o)`

Specifies the observer object for plug-in events. Returns true if the observer was successfully registered.

forward()

Syntax: `boolean forward()`

Forwards the stream to the end of the movie or the position specified in the **playTo** property. Returns true if the forward operation was successful.

getCreateTime()

Syntax: `long getCreateTime()`

Returns the date and time the stream was created as an epoch.

getLength()

Syntax: `long getLength()`

Returns the total length of the stream in milliseconds.

getMaxPos()

Syntax: `long getMaxPos()`

Returns the latest seekable stream position in milliseconds.

getMinPos()

Syntax: `long getMinPos()`

Returns the earliest seekable stream position in milliseconds.

getObserver()

Syntax: `OviObserver getObserver()`

Returns the registered observer object.

getPos()

Syntax: `long getPos()`

Gets the current stream position in milliseconds.

getState()

Syntax: `int getState()`

Returns the current state of the player:

Table A–1 State Values for OviPlayer

Symbolic Representation	Value	Description
OviPlayer.ST_UNINIT	0	Indicates the player has been uninitialized; this state is also set after the **unload()** method returns.
OviPlayer.ST_INIT	1	The player has been created and initialized.
OviPlayer.ST_ERROR	2	Indicates a non-recoverable error has occurred.
OviPlayer.ST_PREPARED	3	Indicates the player has been prepared but a stream has not been loaded.
OviPlayer.ST_REALIZED	4	Indicates the player has completed loading a stream.
OviPlayer.ST_PLAYING	5	Indicates the player is playing.
OviPlayer.ST_PAUSED	6	Indicates the player is paused.
OviPlayer.ST_EOS	7	Indicates the end of stream has been reached. If the **playTo** attribute was set or playback was started by calling the **play()** method with **to** and **from** parameters, playback stops at the position indicated.

When calling **getState()** through JavaScript, you can't access the symbolic representation of the return values. Instead you need to use the literal numeric values shown in the Value column.

getVol()

Syntax: `int getVol()`

Gets the play volume (0 to 100).

load()

Syntax: `boolean load(String mediafile)`

Loads a new stream. ***mediafile*** indicates the filename or asset cookie to load. Returns true if the media file was loaded successful.

After loading, if the **autoStart** attribute is false, the plug-in state is set to **OviPlayer.ST_REALIZED**. If **autoStart** is true, the stream starts playing and the plug-in state is set to **OviPlayer.ST_PLAYING**.

pause()

Syntax: `boolean pause()`

Pauses stream playback at the current position. Returns true and sets the plug-in state to **OviPlayer.ST_PAUSED** if the pause was successful.

play()

Syntax: `boolean play(), boolean play(String from, String to)`

Starts stream playback:

- The first version starts from the beginning of the stream or at the position set with **playFrom**. Playback continues until the end of the stream.
- The second version starts at the position indicated by **from** and continues to play until the position indicated by **to**.

Both methods return true if playback was successfully started. The plug-in state is set to **OviPlayer.ST_PLAYING**.

Note that you can't call **play()** while the player is paused (state set to **OviPlayer.ST_PAUSED)**, only when stopped. When the player is paused, call the **resume()** method to restart playback.

resume()

Syntax: `boolean resume()`

Resumes playback of the stream from the pause state. Returns true and sets state to set to **OviPlayer.ST_PLAYING** if playback was successfully resumed.

Note that you can't call **resume()** while the player is stopped (state **OviPlayer.ST_REALIZED)**, only when paused (state **OviPlayer.ST_PAUSED)**. When the player is stopped, call the **play()** method to start playback.

rewind()

Syntax: `boolean rewind()`

Rewinds the stream to the beginning of the movie or the position specified in the **playFrom** property. Returns true if the rewind was successful.

setAutoStart()

Syntax: `void setAutoStart(boolean startflag)`

Turns the autostart feature on and off. If **startflag** is true, the stream starts automatically when loaded.

setFullScreen()

Syntax: `void setFullScreen(boolean mode)`

Sets normal and full-screen mode. Passing **mode** as true sets full-screen mode, while passing **mode** as false sets normal mode.

setLoop()

Syntax: `void setLoop(boolean loop)`

If **loop** is true, the position indicator returns to the beginning of the stream when playback reaches the end of the stream, except:

- If the **playFrom** property specifies a playback starting point other than the beginning of the stream. In that case, the position indicator returns to the **playFrom** position instead of the beginning.
- If the **playTo** property specifies a playback end point other than the end of the stream. In that case, the position indicator returns to the beginning (or **playFrom** position) when it reaches the **playTo** position.

setObserver()

Syntax: `boolean setObserver(OviObserver o)`

Specifies the observer object for plug-in events. Returns true if the observer was successfully registered.

setPopupMenu()

Syntax: `void setPopupMenu(boolean menu)`

Specifies whether the pop-up menu appears when the user clicks the right mouse button on the player. This pop-up menu allows basic operations like play and pause, rewind to the beginning of the movie, forward to the end of the movie, and close.

setPos()

Syntax:
```
boolean setPos(long position); //this method is overloaded
boolean setPos(string position)
```

Goes to the specified position, either the offset from the beginning in milliseconds, or one of the string positions listed on page A-6.

You should only call this method when the loaded stream is active; this means that the stream has been loaded and is currently playing or paused. **setPos()** does not function properly from the initialized or realized states (see **getState()** for information on plug-in states). Call **play()** with the **from** and **to** for initialized or realized streams.

After the call to this method, the stream maintains its current state: if playing, it continues playing, if paused, it remains paused.

setVol()

Syntax: `void setVol(int vol)`

Sets the play volume (range from 0 to 100).

statusCodeEOS()

Syntax: `int statusCodeEOS()`

Returns the corresponding OviPlayer.ST_EOS value to allow Javascript code to access the status codes returned by **getState()** by name. See Table A–1 for more information.

statusCodeError()

Syntax: `int statusCodeError()`

Returns the corresponding OviPlayer.ST_ERROR value to allow Javascript code to access the status codes returned by **getState()** by name. See Table A–1 for more information.

statusCodeInit()

Syntax: `int statusCodeInit()`

Returns the corresponding OviPlayer.ST_INIT value to allow Javascript code to access the status codes returned by **getState()** by name. See Table A–1 for more information.

statusCodePaused()

Syntax: `int statusCodePaused()`

Returns the corresponding OviPlayer.ST_PAUSED value to allow Javascript code to access the status codes returned by **getState()** by name. See Table A–1 for more information.

statusCodePlaying()

Syntax: `int statusCodePlaying()`

Returns the corresponding OviPlayer.ST_PLAYING value to allow Javascript code to access the status codes returned by **getState()** by name. See Table A–1 for more information.

statusCodePrepared()

Syntax: `int statusCodePrepared()`

Returns the corresponding OviPlayer.ST_PREPARED value to allow Javascript code to access the status codes returned by **getState()** by name. See Table A–1 for more information.

statusCodeRealized()

Syntax: `int statusCodeRealized()`

Returns the corresponding OviPlayer.ST_REALIZED value to allow Javascript code to access the status codes returned by **getState()** by name. See Table A–1 for more information.

statusCodeUnInit()

Syntax: `int statusCodeUnInit()`

Returns the corresponding OviPlayer.ST_UNINIT value to allow Javascript code to access the status codes returned by **getState()** by name. See Table A–1 for more information.

stop()

Syntax: `boolean stop()`

Stops playing the stream and rewind to the beginning. If a start position was specified by **playFrom** or the **from** parameter of **play()**, it's erased and the start position returns to the beginning of the stream. Returns true and sets player state to **OviPlayer.ST_REALIZED** if the stop operation was successful.

unload()

Syntax: `boolean unload()`

Unloads the stream. Returns true if the unload operation was successful.

OviObserver

The **OviObserver** interface specifies two methods you must complete when you implement the **OviObserver** interface. Because there are no implementations provided by OVC for these methods, the description of these methods explains when the methods are called. You then need to implement whatever functionality you want for these events. For more information on using the **OviObserver** interface, see "Using OviObserver" in Chapter 2.

OviObserver contains the following methods:

onPositionChange() **onStop()**

onPositionChange()

Syntax: `public void onPositionChange()`

Called when the stream changes position, as measured in regular increments. The default is one second, but this changes to reflect the setting of the **sliderRate** attribute.

onStop()

Syntax: `public void onStop()`

Called when the plug-in reaches the end of the stream or the **playTo** position if set.

B

Oracle Video Java Library Reference

This appendix describes the public Java classes and interfaces in the Oracle Video Java Library. You can use this library to build native Java applications that access the Oracle Video Server. You can find out how to use these classes in your applications by referring to Chapter 3, “Oracle Video Java Library”.

This section contains descriptions of the following classes and interfaces:

- **Content**
- **ContentException**
- **ContentIter**
- **Player**
- **PlayerApplet**
- **PlayerException**
- **PlayerFactory**
- **PlayerListener**
- **StmInfo**
- **StmName**
- **StmOpts**
- **StmPos**
- **StmStats**

Content

> **Note:** The Content query classes and interfaces are deprecated. These query routines return error ovc-46. For information on the supported way to query content, see Appendix C in the *Oracle Video Server Schema and PL/SQL Procedures Guide*.

The **Content** class provides the ability to query an Oracle Video Server for a list of the available content in a directory of the Media Data Store. **Content** provides two public methods, **query()** and **queryNames()**. Since these methods are static, and **Content** has no data members or constants, you don't need to create **Content** objects.

You can find information on using the content classes **Content**, **ContentException**, and **ContentIter** in "Querying Available Content Titles" on page 3-26.

query()

Syntax: `static StmInfo[] query(String srvAddr, String spec, ContentIter iter)`

This method returns an array of **StmInfo** objects containing information about each piece of content that:

- Is on the server indicated by **srvAddr**
- Meets the specification **spec** on the server indicated by **srvAddr**

Use the **ContentIter** to move through the returned objects. See "Querying Available Content Titles" on page 3-26 for more information on how to query for content titles.

queryNames()

Syntax: `static StmName[] queryNames(String srvAddr, String spec, ContentIter iter)`

This method returns an array of **StmName** objects containing information about each piece of content that:

- Is on the server indicated by **srvAddr**
- Meets the specification **spec** on the server indicated by **srvAddr**

Use the **ContentIter** to move through the returned objects. See "Querying Available Content Titles" on page 3-26 for more information on how to query for content titles.

ContentException

Note: The Content query classes and interfaces are deprecated. These query routines return error ovc-46. For information on the supported way to query content, see Appendix C in the *Oracle Video Server Schema and PL/SQL Procedures Guide*.

ContentException objects are thrown when a problem occurs during content query operations. **ContentException** contains the following members.

Constants	
ContentException.EX_BADPARAM	**ContentException.EX_INTERNAL**
ContentException.EX_BADSTATE	**ContentException.EX_NOTIMPL**
ContentException.EX_ERROR	**ContentException.EX_UNTRANS**
Data Members	
m_code	**m_type**
m_msg	
Methods	
toString()	

You can find information on using the content classes **Content**, **ContentException**, and **ContentIter** in "Querying Available Content Titles" on page 3-26.

ContentException Constants

All of the constants defined in the **ContentException** class are of type **int**. You can check the particular type of exception thrown by examining the **ContentException** member **m_type**, which is set to one of these constants.

ContentException.EX_BADPARAM

Indicates that one of the parameters passed to the routine was invalid in the context of that **Content** method. For instance, if you call **Content**.**query()** with an uninitialized **ContentIter** reference, a **ContentException** with this value is thrown.

ContentException.EX_BADSTATE

Indicates that a **Content** method was called during an invalid state.

ContentException.EX_ERROR

Indicates an error has occurred that is not covered by any other **ContentException** constants.

ContentException.EX_INTERNAL

Indicates that an internal (unexpected) error occurred. In this case, **m_code** and **m_msg** are set and should be reported to Oracle Support Services.

ContentException.EX_NOTIMPL

Indicates the caller attempted to use functionality that is not implemented in the current version of the Oracle Video Java Library.

ContentException.EX_UNTRANS

Indicates that OVC received an internal error that could not be translated into one of the other errors.

ContentException Data Members

The **ContentException** class contains the following public data members.

m_code

Type: `int`

Set to an Oracle-specific error code. This code may be useful in debugging and problem solving interactions with Oracle Support Services.

m_msg

Type: `int`

Contains a string containing additional information on the thrown exception.

m_type

Type: `int`

Indicates the type of exception thrown by a **Content** method. This is one of the **EX_*** constants described in "ContentException Constants" on page B-3.

ContentException Methods

The **ContentException** class contains the following public method.

toString()

Syntax: `String toString()`

Returns a single string consisting of all the information contained in each of the member variables.

ContentIter

Note: The Content query classes and interfaces are deprecated. These query routines return error ovc-46. For information on the supported way to query content, see Appendix C in the *Oracle Video Server Schema and PL/SQL Procedures Guide*.

ContentIter works in conjunction with the **Content** class to let you retrieve lists of available content titles from an Oracle Video Server. The process of retrieving these titles is iterative, which means that it is performed in steps, by retrieving only a few titles at a time. You use a **ContentIter** object to manage this process.

ContentIter contains the following members.

Data Members	
m_num	**m_pos**
Methods	
ContentIter()	

You can find information on using the content classes **Content**, **ContentException**, and **ContentIter** in "Querying Available Content Titles" on page 3-26.

ContentIter Data Members

ContentIter contains the following public data members.

m_num

Type: `int`

Indicates the number of titles to retrieve with each query operation. When all available titles have been retrieved, **Content.query()** sets this to the number of titles actually retrieved with the last query. **Content.query()** also sets **m_pos** to -1 when the last titles have been queried.

m_pos

Type: `int`

Indicates the current position in the list of available titles. **Content.query()** sets this to -1 when all available titles have been retrieved. **Content.query()** also sets **m_num** to the number of titles actually retrieved with the last query.

ContentIter Methods

ContentIter contains only one public method, its constructor.

ContentIter()

Syntax: `ContentIter(int pos, int num)`

Sets the iterator values to be used during a query operation:

- **pos** indicates the position in the list of available titles at which you want to start. For example, if your last query retrieved the first 30 titles from a particular server, you may not want to retrieve those again. You could then set **pos** to 30 before your next query.
- **num** indicates the number of titles you want to retrieve with each query operation.

Player

Player is the central object in the Oracle Video Java Library. It provides:

- A set of constants for defining the player state
- Default user interface
- Media control API
- Service methods

Unlike the **PlayerListener** interface, the **Player** interface is already implemented at a lower level in the **oracle.ovc** package and therefore cannot be implemented in your own Java application.

Player contains the following members.

Constants		
Player.ST_EOS	**Player.ST_PAUSED**	**Player.ST_REALIZED**
Player.ST_ERROR	**Player.ST_PLAYING**	**Player.ST_UNINIT**
Player.ST_INIT		
Methods		
addListener()	**getStatusComp()**	**setFullScreen()**
getControlComp()	**getVisualComp()**	**setPos()**
getInfo()	**getVol()**	**setVol()**
getPlayerUI()	**load()**	**stateToString()**
getPos()	**pause()**	**stop()**
getSelRange()	**play()**	**term()**
getState()	**resume()**	**unload()**
getStats()		

Player does not have a public constructor. To create new **Player** objects, use the **PlayerFactory**.**getPlayer()** method. See “PlayerFactory” on page B-20 for more information.

Player Constants

This section details the possible states of the **Player** object. All **Player** states are expressed as constants in the form **Player.ST_***.

Player State Reference

This section describes the **Player** states and the various ways a **Player** object can arrive at these states. Methods that are briefly mentioned in this section are discussed in detail in "Player Methods" on page B-10. For more information on tracking **Player** states, see "Stream and Player State" on page 3-14 and "Handling Player Events" on page 3-18.

Player.ST_EOS

Indicates the end of stream has been reached. **PlayerListener.endOfStream()** provides another method for determining when the player reaches the end of a stream.

Player.ST_ERROR

Indicates a non-recoverable error has occurred. **PlayerListener.error()** provides another method for determining when such errors occur.

Player.ST_INIT

Indicates the **Player** object has been created and initialized, without having a stream loaded. You can also encounter this state after the **unload()** method returns.

Player.ST_PAUSED

Indicates the **Player** object is paused.

Player.ST_PLAYING

Indicates the **Player** object is playing the current stream. This happens after the **play()** or **resume()** methods return. It also happens after **load()** returns if **StmOpts**.**m_autoStart** is **true.** See the **StmOpts** class reference on page B-29 for more information.

Player.ST_REALIZED

Indicates the player has completed loading a stream.

Player.ST_UNINIT

Indicates the player has been terminated and uninitialized. The player is in this state a very brief period of time after the **term()** method is called. Note that the **term()** method should only be called after **unload()** returns and the state is **Player.ST_INIT**. You cannot reuse a **Player** object after calling **term()**.

Player Methods

Player methods break down into three general categories:

- **User Interface Methods**
- **Media Control Methods**
- **Player Service Methods**

There are some things that all of these methods share:

- All of the **Player** methods can throw **PlayerException** objects. You must place **Player** method calls within **try** blocks and catch any **PlayerException** objects in order for your code to compile.
- Each of the methods presented here are declared **public**.

User Interface Methods

The following methods deal with retrieving or configuring the user interface portion of the video client.

getControlComp()

Syntax: `Component getControlComp()`

Retrieves the standard controller interface component to be added to the player interface. The controller includes a Play / Pause button, a Stop button, and a Seek scrollbar.

getPlayerUI()

Syntax:
```
Component getPlayerUI()
Component getPlayerUI(boolean Video, boolean Controls, boolean Status)
Component getPlayerUI(boolean Video, boolean Controls, boolean Status,
                      int width, int height)
```

Retrieves the video screen (visual component), the controller panel, and the status line panel bundled together as a single component to be added to the player interface.

- The first form of this method returns the interface with all of the components.
- The second form lets you select which interface components are displayed.
- The third form, like the second, lets you select which interface components are displayed, but also lets you specify a width and height for the interface. This forces the interface component to keep a fixed width and height, even if the host window is resized.

getSelRange()

Syntax: `void getSelRange(StmPos from, StmPos to)`

Retrieves the current start and end positions of the player and puts them in the **from** and **to** parameters.

getStatusComp()

Syntax: `Component getStatusComp()`

Retrieves the standard status line interface component to be added to the player interface. This status line component includes a stream position counter and a status message text area.

getVisualComp()

Syntax: `Component getVisualComp()`

Retrieves the video screen (visual component) as an interface component to be added to the player interface.

Media Control Methods

The following methods give you control over stream playback.

getPos()

Syntax: `StmPos getPos(int fmt)`

Returns the current position of the stream in the form of a **StmPos** object. This method requires that the position format, **fmt**, be specified by the Java application.

The only two position formats supported by this method are **StmPos.POSFMT_TIME** and **StmPos.POSFMT_FRAMES**. You can call this method from any state in which a stream is present. See the **StmPos** class reference on page B-33 for more information.

getVol()

Syntax: `int getVol()`

Returns the current volume setting of the player as an integer in the range of 0 to 100. You can call this method from any state in which a stream is present.

load()

Syntax: `void load(String mediafile, StmOpts opts)`

Loads the media content file and prepares it for playback. You can call this method from all active states.

mediafile is a **String** object containing a video session URL. This specifier describes the location of the media resource to be loaded. See Appendix D, "Oracle Video Session URLs", for more information on video session URLs.

StmOpts defines a number of stream attributes, such as auto start, start position, end position, and so on. See the **StmOpts** class reference on page B-29 for more information.

Pass a null **StmOpts** object if you want all of the following default media stream options:

- **m_autoStart=false**
- **m_loop=false**

- **m_volume=StmOpts.DEFAULT_VOL**
- **m_playFrom=default**
- **m_playTo=end**

For example:

```
try {
    myapp.m_player.load("/mds/video/oracle1.mpi", null);
}
catch(PlayerException e) {
    System.out.println("Failed to load default stream.");
}
```

pause()

Syntax: `void pause()`

Pauses playback, preserving the current position. The current frame will remain on screen. Can only be called from **Player.ST_PLAYING**.

play()

Syntax: `void play(), void play(StmPos from, StmPos to)`

Starts stream playback:

- The first version starts from the beginning of the stream or the position indicated by the **m_playFrom** member of the **StmOpts** object passed to **load()**. Unless interrupted, playback continues until the stream reaches its end or the position indicated by the **m_playTo** member of the **StmOpts** object passed to **load()**.
- The second version starts playback from the position indicated by the **from** parameter and continues until the stream reaches the position indicated by the **to** parameter. The **from** and **to** parameters override the start and end positions set in the **m_playFrom** and **m_playTo** members of the **StmOpts** object passed to **load()**, but do not replace them: the next call to the first version of **play()** uses the original start and end positions.

You must call this method while the player is stopped. You can't call **play()** to resume play from a paused state; in that case, call **resume()**.

resume()

Syntax: `void resume()`

Resumes playback of the stream from the current position. You must call this method while the player is paused. You can't call **resume()** to resume play from a stopped state; in that case, call **play()**.

setFullScreen()

Syntax: `void setFullScreen(boolean mode)`

Sets player to full-screen mode. Passing **mode** as **true** sets full-screen mode, while passing **mode** as **false** sets normal mode.

setPos()

Syntax: `void setPos(StmPos pos)`

Moves the stream position to the value contained in the **StmPos** object. You can call this method from any state in which a stream is present. See the **StmPos** class reference on page B-33 for more information.

setVol()

Syntax: `void setVol(int vol)`

Sets the current volume of the **Player** as an integer from 0 to 100. You can call this method from any state in which a stream is present.

stateToString()

Syntax: `String stateToString(int state)`

Converts the player state indicated by **state** into a text description contained in the returned **String** value. Use this to convert **Player** constants such as **Player.ST_PLAYING** or **Player.ST_ERROR** returned from **getState()** into text.

stop()

Syntax: `void stop()`

Stops playback and resets the stream position to the beginning of the stream. If a start position was specified in **StmOpts**.**m_playFrom** or the **from** parameter of **play()**, it's erased and the start position returns to the beginning of the stream. The stream rewinds to the first frame, which appears in the video screen. Note that encoding sometimes makes the first frame black. You can call this method while the player state is one of **Player.ST_PLAYING**, **Player.ST_PAUSED**, or **Player.ST_EOS**.

unload()

Syntax: `void unload()`

Releases any resources associated with the current stream. You can call this method from any state in which a stream is present.

Player Service Methods

The following methods give you access to the internals of the **Player** object, such as player state and configuring the listener object.

addListener()

Syntax: `void addListener(PlayerListener listener)`

Registers the listener object as a **PlayerListener**. The listener is notified of specific events occurring in the **Player** object through the **PlayerListener** interface methods.

This example shows a simple class that implements the **PlayerListener** interface, creates a **Player** object, and adds itself as a listener to the **Player** object.

```
public class myClass implements PlayerListener {
   Player m_player = null;
   myClass app = null;

   public static void main(String args[]) {
      app = new myClass();
      m_player = PlayerFactory.getPlayer();
      m_player.addListener(app);
      m_player.load(args[0], null);
      m_player.play();
      }
```

```
// PlayerListener methods
public void stateChange(int newState) {
   System.out.println("Player state change to: " + newState);
   }
public void error(int code, String msg) {
   System.out.println("Error! Code: " + code);
   }
public void endOfStream() {
   System.out.println("End of stream reached!");
   }
}
```

Note: A Java application or object must implement the **PlayerListener** interface to be registered as a listener object. See the **PlayerListener** interface reference on page B-21 for more information.

getInfo()

Syntax: `StmInfo getInfo()`

Returns a **StmInfo** object containing information about the current stream. You can call this method from any state in which a stream is present. See the **StmInfo** class reference on page B-23 for more information.

getState()

Syntax: `int getState()`

Returns the current state of the **Player** object. The return value of this method can be any of the values described in "Player Constants" on page B-9. You can call this method from any **Player** state.

getStats()

Syntax: `StmStats getStats()`

Returns a **StmStats** object containing information concerning the current playback states and statistics of the stream. You can call this method from any state in which a stream is present. See the **StmStats** class reference on page B-37 for more information.

term()

Syntax: `int term()`

Terminates the internal Oracle Video Client processes but does not release the **Player** object in the Java context. If necessary, this method calls **unload()** before terminating the player. This method should be called prior to exiting applications that use the Oracle Video Java Library.

> **Note:** If you want an application to free the **Player** object in the Java context and continue running, call this method, set the **Player** object to **null**, and call the Java garbage collection service.

Setting the **Player** object to **null** allows the Java garbage collector to immediately release all resources associated with the object. For example, this code sample terminates the **Player**, sets the object reference to **null**, then calls the Java garbage collector:

```
try {
   app.m_player.term();
}

catch(PlayerException e){
   System.out.println("term() failed.");
}

app.m_player = null;
System.gc();
```

PlayerApplet

PlayerApplet is a simple Java applet that supports the same syntax and features as the Oracle Video Web Plug-in, which is described in Chapter 2, "Oracle Video Web Plug-in" and Appendix A, "Oracle Video Web Plug-in Reference". Note that the applet is not signed, so it may not function in all browsers. You can find out more about using **PlayerApplet** in "Using the PlayerApplet Class" on page 3-31.

PlayerApplet recognizes the same attributes to the **<applet>** tag that the Oracle Video Web Plug-in recognizes for the **<embed>** tag. For information on these attributes, see "<Embed> Attributes" on page A-1.

PlayerException

The **PlayerException** class represents exceptions thrown specifically by the methods of the **Player** class. *All* public methods of the **Player** class throw **PlayerException** exception objects.

The **PlayerException** object's **m_type** member represents the exception type thrown. This is indicated by a constant of the format **EX_*** (the constants used are described in "PlayerException Constants" on page B-18). The **PlayerException** object's **m_code** property is set to an Oracle-specific error code useful for debugging purposes. The **m_msg** property is a **String**, which may contain additional information about the exception thrown.

PlayerException contains the following members.

Constants	
PlayerException.EX_BADPARAM	**PlayerException.EX_INTERNAL**
PlayerException.EX_BADSTATE	**PlayerException.EX_NOTIMPL**
PlayerException.EX_ERROR	**PlayerException.EX_UNTRANS**
Data Members	
m_code	**m_type**
m_msg	
Methods	
toString()	

PlayerException Constants

All of the constants defined in the **PlayerException** class are of type **int**. You can check the type of exception thrown by examining **m_type**, which is set to one of these constants.

PlayerException.EX_BADPARAM

Indicates that one of the parameters passed to the routine was invalid in the context of that **Player** method. For instance, if you call **Player.setVol()** with an integer value greater than 100 or less than 0, a **PlayerException** with this value is thrown.

PlayerException.EX_BADSTATE

Indicates that a **Player** method was called during an invalid state. For instance, calling **Player.pause()** while loading a stream causes this **PlayerException** to be thrown.

PlayerException.EX_ERROR

Indicates an error has occurred that is not covered by any other **PlayerException** constants.

PlayerException.EX_INTERNAL

Indicates that an internal (unexpected) error occurred. In this case, **m_code** and **m_msg** are set and should be reported to Oracle Support Services.

PlayerException.EX_NOTIMPL

Indicates the caller attempted to use functionality that is not implemented in the current version of the Oracle Video Java Library.

PlayerException.EX_UNTRANS

Indicates that an untranslated error has occurred. To translate the error, call the **toString()** method.

PlayerException Data Members

The **PlayerException** class contains the following public data members.

m_type

Type: `int`

Indicates the type of exception thrown by a **Player** method. This is one of the **EX_*** constants described in “PlayerException Constants” on page B-18.

m_code

Type: `int`

Set to an Oracle-specific error code. This code may be useful when debugging and solving OVC and Oracle Video Java Library problems with Oracle Support Services.

m_msg

Type: `int`

Contains a string containing additional information on the thrown exception.

PlayerException Methods

The **PlayerException** class contains the following public method.

toString()

Syntax: `String toString()`

Returns a single string consisting of all the information contained in each of the member variables.

PlayerFactory

The **PlayerFactory** class provides a means of creating new **Player** objects. You must call the methods of **PlayerFactory** class to create **Player** objects, rather than calling a **Player** constructor method.

PlayerFactory contains the following members.

Methods		
createPlayer()	**getPlayer()**	**getPlayerFactory()**

PlayerFactory Methods

As with the members of the **Player** class, all the methods presented here are declared **public** and can throw a **PlayerException** object.

createPlayer()

Syntax: `Player createPlayer()`

Instantiates a **Player** object for use by the calling Java application. **createPlayer()** throws a **PlayerException** object if a **Player** cannot be instantiated.

getPlayer()

Syntax: `static synchronized Player getPlayer()`

Returns a new **Player** object. This is the easiest and most common method of instantiating new **Player** objects. This method is equivalent to calling **PlayerFactory.getPlayerFactory().createPlayer()**.

Here is an example of calling **getPlayer()**:

```
try {
   myapp.m_player = PlayerFactory.getPlayer();
}
catch(PlayerException e) {
   System.out.println("getPlayer() failed...");
}
```

getPlayerFactory()

Syntax: `static synchronized PlayerFactory getPlayerFactory()`

Returns the default **PlayerFactory** object, which can then be used to instantiate appropriate **Player** objects with the **createPlayer()** method. Throws a **PlayerException** object if a factory is not available.

PlayerListener

The **PlayerListener** interface can be implemented by Java applications or objects that wish to be notified of state changes that occur in the **Player** object.

To receive notification from a **Player** object, the Java application or object must:

- Properly register itself as a **PlayerListener** by calling **Player.addListener()**; see "Player Service Methods" on page B-15 for more details about this method.
- Implement all of the **PlayerListener** methods described in this section

PlayerListener contains the following members.

Methods		
endOfStream()	**error()**	**stateChange()**

PlayerListener Methods

All **PlayerListener** methods must be implemented if you want a **Player** object to notify your Java application or object each time an event occurs. Once properly registered, each of these methods is invoked as the corresponding event occurs. For information on registering an object as a listener, see "Handling Player Events" on page 3-18.

error()

Syntax: `void error(int code, String msg)`

Indicates a known, non-recoverable error has occurred in the **Player** object. **code** is an error code specific to the Oracle Video Client in the format OVC-*xxxx*. **msg** describes the error.

After this method is invoked, the player is in the state **Player.ST_ERROR**. This is a non-recoverable state: the player must be unloaded using the **Player.unload()** method.

endOfStream()

Syntax: `void endOfStream()`

Indicates the player has reached the end of the current stream.

stateChange()

Syntax: `void stateChange(int newState)`

Indicates the **Player** object has changed state.

One of the states of which **stateChange()** is notified is **Player.ST_EOS** (end of the current stream). This performs the same function as the **endOfStream()** method, providing another way of detecting the end of a stream. For a complete list of **Player** states and their descriptions, refer to "Player Constants" on page B-9.

StmInfo

The **StmInfo** class represents static information about the current stream. **StmInfo** objects are returned by the **Player.getInfo()** method.

StmInfo contains the following members.

Constants	
StmInfo.CSTAT_DISK	**StmInfo.CSTAT_ROLLING**
StmInfo.CSTAT_FEED	**StmInfo.CSTAT_TAPE**
StmInfo.CSTAT_LOCALFILE	**StmInfo.CSTAT_TERMINATED**
StmInfo.CSTAT_NETWORK	**StmInfo.CSTAT_UNKNOWN**
Data Members	
m_aspect	**m_fps**
m_asset	**m_msecs**
m_bitrate	**m_name**
m_bytes	**m_proto**
m_contStat	**m_server**
m_contType	**m_size**
m_createTime	**m_url**
m_desc	
Methods	
contStatToString()	**toString()**

StmInfo Constants

All of the constants defined in the **StmInfo** class are of type **int**. You can check the container status of the current stream by examining the **StmInfo** member **m_contStat**, which is set to one of these constants. The container status indicates what type of container the current stream is using.

StmInfo.CSTAT_DISK

Indicates the current stream originates from a file stored on disk on the Oracle Video Server.

StmInfo.CSTAT_FEED

Indicates the current stream comes from a one-step encode feed.

StmInfo.CSTAT_LOCALFILE

Indicates the current stream originates from your local file system.

StmInfo.CSTAT_NETWORK

Indicates the current stream comes from a network feed, such as multicast broadcast or other wide-band network feed.

StmInfo.CSTAT_ROLLING

Indicates the current stream comes from a rolling live feed.

StmInfo.CSTAT_TAPE

Indicates the current stream originates from a file stored in the Hierarchical Storage Manager (HSM) on the Oracle Video Server.

StmInfo.CSTAT_TERMINATED

Indicates the current stream comes from a terminated feed.

StmInfo.CSTAT_UNKNOWN

Indicates the current container status is unknown.

StmInfo Data Members

The **StmInfo** class contains the following public data members.

m_aspect

Type: `int`

Contains the video aspect ratio.

So that the aspect ratio, which usually contains a decimal portion, can be contained in an **int** instead of a **float**, the value of **m_aspect** actually contains the aspect ratio multiplied by 1,000. For example, the aspect ratio 1.6 would be represented as 1600.

m_asset

Type: `String`

Contains the logical content asset cookie name.

m_bitrate

Type: `int`

The total bit rate of the current stream. This includes the audio component, the video component, and the container overhead.

m_bytes

Type: `long`

File size in bytes of the stored content. This is zero if the stream is unbounded, such as with a live video feed.

m_contStat

Type: `int`

Indicates the content status of the stream. This is one of the **StmInfo** constants of the form **CSTAT_***. You can convert the integer value contained in this property into a **String** description by calling the **contStatToString()** method.

m_contType

Type: `String`

Contains the container type of the stream.

m_createTime

Type: `Date`

Contains the date and time the content was created, contained in a Java **Date** object.

m_desc

Type: `String`

Contains a text description of the stream.

m_fps

Type: `int`

Contains the frame rate of the current video component. The format is:

fps x 1,000

where *fps* is frames per second. To find the actual frames per second, divide this by 1,000.

m_msecs

Type: `int`

File size in milliseconds of the stored content. This is zero if the stream is unbounded, such as with a live video feed.

m_name

Type: `String`

Contains the name of the stream.

m_proto

Type: `String`

Contains the transport protocol of the stream.

m_server

Type: `String`

Contains the IP address or DNS name of the video server.

m_size

Type: `Dimension`

Contains the source input size.

m_url

Type: `String`

Contains the video session URL used in the **Player**.**load()** method. See Appendix D, "Oracle Video Session URLs" for a description of video session URLs.

StmInfo Methods

The **StmInfo** class contains the following public methods.

contStatToString()

Syntax: `static String contStatToString(int cstat)`

Converts the stream container status indicated by **cstat** into a text description contained in the returned **String** value. Use this to convert the **m_contStat** property and **StmInfo** constants such as **StmInfo.CSTAT_DISK** or **StmInfo.CSTAT_FEED** into text.

toString()

Syntax: `String toString()`

This routine returns a single string consisting of all the information contained in each of the **StmInfo** member variables.

StmName

The **StmName** class represents static information about the current stream. **StmName** objects are returned by the **Content.queryNames()** method.

StmName contains the following data members.

Data Members		
m_asset	**m_proto**	**m_url**
m_name	**m_server**	
Methods		
toString()		

StmName Data Members

The **StmName** class contains the following public data members.

m_asset

Type: `String`

Contains the logical content asset cookie name.

m_name

Type: `String`

Contains the name of the stream.

m_proto

Type: `String`

Contains the transport protocol of the stream.

m_server

Type: `String`

Contains the IP address or DNS name of the video server.

m_url

Type: `String`

Contains the video session URL used in the **Player**.**load()** method. See Appendix D, "Oracle Video Session URLs" for a description of video session URLs.

StmName Methods

The **StmName** class contains the following public method.

toString()

Syntax: `String toString()`

This routine returns a single string consisting of all the information contained in each of the **StmName** member variables.

StmOpts

The **StmOpts** class consists of various media stream options. Modification of these options allows you to customize attributes of the stream playback.

> **Note:** The **StmOpts** object is passed to the **Player**.**load()** method. Typically, if default values are desired, pass a null **StmOpts** object when loading a stream. Alternatively, create an object of this class that uses some of the default values and customizes other values.

See the code sample provided on page B-10 for an example of passing a null default **StmOpts** object to the **Player**.**load()** method. See the code sample on page B-32 for an example of passing a modified **StmOpts** object to the **Player**.**load()** method.

StmOpts contains the following members.

Constants		
StmOpts.DEFAULT_VOL		
Data Members		
m_autoStart	**m_loop**	**m_popup**
m_img	**m_playFrom**	**m_volume**
m_leftClick	**m_playTo**	
Methods		
StmOpts()		

StmOpts Constants

All constants in the **StmOpts** class are of type **int**.

StmOpts.DEFAULT_VOL

This constant indicates the default player volume. Setting **StmOpts.m_volume** to this when calling **Player**.**load()** leaves the volume unchanged.

StmOpts Data Members

The **StmOpts** class contains the following public data members.

m_autoStart

Type: boolean

Specifies whether playback starts automatically. If **true**, the player starts playback as soon it loads the stream. This is slightly faster than calling **Player**.**load()** then **Player**.**play()**.

The default value is **false**, meaning that you must explicitly call **Player**.**play()** to begin playback.

m_img

Type: `String`

Contains the name of an image file to be displayed when the video screen is blank.

m_leftClick

Type: `boolean`

Specifies whether a left click on the video screen toggles Play and Pause. If **m_leftClick** is **true**, left clicking in the video screen has the same effect as clicking on the Play / Pause button. The default value is **true**.

m_loop

Type: `boolean`

If **true**, the player, upon reaching the end of the stream, returns to the beginning of the stream and continues playback from that point. If start and end points were set, then the player returns from the end point to the start point and continues playback.

The default value is **false** (no looping).

m_playFrom

Type: `StmPos`

Sets the starting position of the stream. The default for this member is the beginning of the stream. Internally this is represented as a **StmPos**(**StmPos.POSFMT_BEGINNING**) object.

m_playTo

Type: `StmPos`

Sets the ending position of the stream. Playback stops when this position is reached. The default value for this member is the end of the stream. Internally this is represented as a **StmPos**(**StmPos.POSFMT_END**) object.

m_popup

Type: boolean

Specifies whether the pop-up menu appears when the user clicks the right mouse button on the player. This pop-up menu allows basic operations like play and pause, rewind to the beginning of the movie, forward to the end of the movie, and close. The default value is **true**.

m_volume

Type: int

Contains the audio volume setting for the stream. This value is an integer in the range of 0 - 100. The default value for this member is **StmOpts.DEFAULT_VOL**, which leaves the volume at its current setting.

StmOpts Methods

The **StmOpts** class contains the following public methods.

StmOpts()

Syntax: StmOpts()

The default constructor for the **StmOpts** class. Sets the **StmOpts** member fields to their default values. To override an option, set the member value explicitly before calling the **Player**.**load()** method.

Here is an example of overriding a **StmOpts** default member value:

```
//instantiate a default StmOpts object and change the m_playFrom position
StmOpts myStmOpts = new StmOpts();
myStmOpts.m_playFrom = 5000;

// Load the stream with the customized StmOpts object
try {
   myapp.m_player.load("/mds/video/oracle1.mpi", myStmOpts);
}
catch(PlayerException e) {
   System.out.println("Failed to load customized stream.");
}
```

StmPos

The **StmPos** class represents stream position information in various formats. This class is used to specify stream positions as passed to the **Player** with the **Player.setPos()** method, or as received from the **Player** with the **Player.getPos()** method. **StmPos** objects are also utilized as member variables in the **StmOpts** class.

StmPos contains the following members.

Constants	
StmPos.POSFMT_BEGINNING	**StmPos.POSFMT_END**
StmPos.POSFMT_CURRENT	**StmPos.POSFMT_FRAMES**
StmPos.POSFMT_DEFAULT	**StmPos.POSFMT_TIME**
Data Members	
m_fmt	**m_val**
Methods	
StmPos()	**toString()**
fromString()	

StmPos Constants

StmPos constants specify the possible formats for the stream position. These constants are used when instantiating **StmPos** objects. There are two forms of the **StmPos()** constructor:

- **StmPos(int fmt)**
- **StmPos(int fmt, long val)**

The constants presented in this section are used for the **int fmt** parameter in both versions of the constructor. Which version of the constructor you use depends on what you specify for **fmt**. This is discussed both in this section and in "StmPos Methods" on page B-35.

StmPos.POSFMT_BEGINNING

Indicates that the desired stream position is the beginning of the stream. Use the **StmPos(int fmt)** version of the constructor to specify the **StmPos.POSFMT_BEGINNING** constant.

StmPos.POSFMT_CURRENT

Indicates that the desired stream position is the current position (that is, don't change the stream position). Use the **StmPos(int fmt)** version of the constructor to specify the **StmPos.POSFMT_CURRENT** constant.

StmPos.POSFMT_DEFAULT

Indicates that the desired stream position is the default position. The default position is the beginning of the stream for all types of streams except for unbounded streams. For unbounded streams, the default position is the end of the stream. Use the **StmPos(int fmt)** version of the constructor to specify the **StmPos.POSFMT_DEFAULT** constant.

StmPos.POSFMT_END

Indicates that the desired stream position is the end of the stream. Use the **StmPos(int fmt)** version of the constructor to specify the **StmPos.POSFMT_END** constant.

StmPos.POSFMT_FRAMES

Indicates that the desired stream position is expressed as the number of frames from the beginning of the stream. Use the **StmPos(int fmt, long val)** version of the constructor to specify the **StmPos.POSFMT_FRAMES** constant.

StmPos.POSFMT_TIME

Indicates that the desired stream position is expressed as a time in milliseconds from the beginning of the stream. Use the **StmPos(int fmt, long val)** version of the constructor to specify the **StmPos.POSFMT_TIME** constant.

StmPos Data Members

The **StmPos** class contains the following public data members.

m_fmt

Type: `int`

The format of the position field. This is limited to one of the **StmPos** constants.

m_val

Type: long

The format-specific stream position value. This field is only used for the time value formats of **StmPos.POSFMT_TIME** and **StmPos.POSFMT_FRAMES**.

StmPos Methods

There are three ways of instantiating objects of the **StmPos** type: two **StmPos** constructors and a **fromString()** method. Each of these ways results in the instantiation of a new **StmPos** object, and each has its appropriate usage.

All of the methods presented here are declared **public** within the **StmPos** class definition. Unlike some other classes in the Oracle Video Java Library, only the constructors, not all of the **StmPos** methods, throw the **PlayerException** object.

StmPos()

Syntax: `StmPos(int fmt), StmPos(int fmt, long val)`

Creates a new **StmPos** object:

- The first constructor specifies only the **m_fmt** format field of the **StmPos** object and can be used when the **m_val** field is not required by the object. **m_val** is not required when you instantiate objects of the following formats:
  - **StmPos.POSFMT_BEGINNING**
  - **StmPos.POSFMT_END**
  - **StmPos.POSFMT_CURRENT**
  - **StmPos.POSFMT_DEFAULT**

  In these cases, the positional information for the object is contained in the format field. A **PlayerException** object is thrown if the format type passed requires **m_val** to be specified.
- The second constructor sets both the format and the value of a stream's positional information. You can only use this constructor when instantiating **StmPos** objects of the formats **StmPos.POSFMT_TIME** or **StmPos.POSFMT_FRAMES**.

Note that this constructor expects **val** to be in milliseconds. For instance, the constructor interprets a long value of 4000 to mean 4 seconds (4000 milliseconds).

See Table B–1 for additional **StmPos** object constructor information.

fromString()

Syntax: `static StmPos fromString(String pos)`

This third method of instantiating **StmPos** objects can be used to convert an input string into a **StmPos** object. Because it is possible to instantiate any of the **StmPos** object formats with this method, this method is the most safe and versatile way of instantiating **StmPos** objects.

See Table B–1 for the available input string formats.

Table B–1 Positional Values and StmPos constructor equivalents

Position Value	Equivalent StmPos Constructor
milliseconds	`StmPos(`**`StmPos.POSFMT_TIME`**`, timeVal)`
hundreds	`StmPos.fromString("hh:mm:ss:cc")`
frame_num	`StmPos(`**`StmPos.POSFMT_FRAMES`**`, frameVal)`
beginning	`StmPos(`**`StmPos.POSFMT_BEGINNING`**`)` `StmPos.fromString("beginning")`
current	`StmPos(`**`StmPos.POSFMT_CURRENT`**`)` `StmPos.fromString("current")`
end	`StmPos(`**`StmPos.POSFMT_END`**`)` `StmPos.fromString("end")`
default	`StmPos(`**`StmPos.POSFMT_DEFAULT`**`)` `StmPos.fromString("default")`

toString()

Syntax: `static String toString()`

This method converts a **StmPos** object into a **String**. See Table B–1 for the possible returned string formats.

StmStats

The **StmStats** class contains dynamic information about both the playback states and statistics of the current media stream. This object is returned from the **Player.getStats()** method.

StmStats contains the following members.

Constants		
StmStats.STM_CONTROL	**StmStats.STM_IDLE**	**StmStats.STM_PLAYING**
StmStats.STM_ENDED	**StmStats.STM_PAUSED**	**StmStats.STM_STALLED**
Data Members		
m_bps	**m_drops**	**m_minTime**
m_cnsState	**m_fBytes**	**m_pkts**
m_curFrame	**m_fps**	**m_prdState**
m_curTime	**m_maxTime**	**m_rBytes**
Methods		
stateToString()	**toString()**	

StmStats Constants

These constants represent the current network stream state, not the state that is returned by the **Player.getState()** method.

StmStats.STM_CONTROL

Indicates that the stream is in the middle of a network transaction.

StmStats.STM_ENDED

Indicates that the end of stream has been reached.

StmStats.STM_IDLE

Indicates that the stream is idle.

StmStats.STM_PAUSED

Indicates that the stream is paused.

StmStats.STM_PLAYING

Indicates that the stream is playing.

StmStats.STM_STALLED

Indicates that the stream has stalled.

StmStats Data Members

The **StmStats** class contains the following public data members.

m_bps

Type: `int`

Indicates the average bit rate in bits per second (bps) over the life of the current stream.

m_cnsState

Type: `int`

Playback state of the current media stream. This is an integer value limited to one of the **StmStats** listed constants, such as **StmStats.STM_CONTROL** or **StmStats.STM_PLAYING**.

m_curFrame

Type: `long`

Current frame that appears on the video screen. The number of frames starts with 0 at the beginning or **playFrom** position and increments with each successive frame shown.

m_curTime

Type: `long`

Indicates the current position in the stream, in milliseconds.

m_drops

Type: `int`

Indicates the number of packet drops since playback started. Unreliable network protocols, such as UDP, tend to drop some packets.

m_fBytes

Type: `int`

Indicates the number of free bytes in the client cache.

m_fps

Type: `int`

Indicates the average frame rate in frames per second (fps) over the life of the current stream.

m_maxTime

Type: `long`

Indicates the latest seekable time in the stream, in milliseconds.

m_minTime

Type: `long`

Indicates the earliest seekable time in the stream, in milliseconds.

m_pkts

Type: `int`

The number of packets received since stream playback began.

m_prdState

Type: `int`

Playback state of the current media stream. This is an integer value limited to one of the **StmStats** listed constants, such as **StmStats.STM_CONTROL** or **StmStats.STM_PLAYING**.

m_rBytes

Type: `int`

Indicates the number of bytes in the client cache ready for consumption.

StmStats Methods

The **StmStats** class contains the following public methods.

stateToString()

Syntax: `static String stateToString(int state)`

Converts the stream state indicated by **state** into a text description contained in the returned **String** value. Use this to convert **StmStats** constants such as **StmStats.STM_PLAYING** or **StmStats.STM_ENDED** into a text description. The **m_cnsState** and **m_prdState** properties both use these constants.

toString()

Syntax: `String toString()`

Returns a single string consisting of all the information contained in each of the member variables.

C

Oracle Video ActiveX Control Reference

This appendix contains the reference information for the Oracle Video ActiveX Control. It can be used to embed the Oracle Video Client in applications that support the Microsoft ActiveX standard. You can find out how to use these classes in your applications by referring to Chapter 4, “Oracle Video ActiveX Control”.

This appendix contains the following sections:

- Methods
- Properties
- Events
- <Object> Attributes and Parameters

Methods

The Oracle Video ActiveX Control provides the following methods:

Forward()	**ImportFileAs()**	**SetPos()**
GetInfo()	**ImportStreamAs()**	**SetVol()**
GetInfoVT()	**Load()**	**ShowInfoDialog()**
GetPos()	**Pause()**	**ShowStatsDialog()**
GetStats()	**Play()**	**Stop()**
GetStatsVT()	**Resume()**	**Unload()**
GetVol()	**Rewind()**	

Note: The descriptions in this section use Visual Basic syntax. For examples that show how Visual C++ can use the Oracle Video ActiveX Control, see the Visual C++ sample code in Chapter 4, "Oracle Video ActiveX Control".

Forward()

Syntax: `Forward`

Forward the video to the end or the position set by the **PlayTo** property.

GetInfo()

Syntax:
```
GetInfo(streamName as string,
        url as string,
        title as string,
        fmtType as string,
        extType as string,
        createTimeStr as string,
        byteLength as long,
        bitrate as long,
        presRate as long,
        capabilities as long,
        milliseconds as long,
        frameHeight as integer,
        frameWidth as integer,
        aspectRatio as long,
        frameRate as long,
```

```
contentStatus as integer,
protocol as integer)
```

This method returns information about the current media file.

Important: Make sure you declare all of the variables that you pass as parameters, and initialize all of the strings to null strings, before calling **GetInfo()**.

If the call to **GetInfo()** succeeds, it sets the variables passed as parameters to the indicated data and returns **1**. Otherwise **GetInfo()** returns **0** and the parameters do not contain valid information about the stream. You should make sure you check the return value before relying on the data contained in the variables.

The parameters contain the following information after a successful **GetInfo()** call:

Parameter	Description
streamName	The unique name of the stream
url	The URL or asset cookie of the stream
title	The stream title or description (not unique)
extType	File extension; for example, **mpi**, **mpg**, or **osf**
fmtType	A readable string version of **extType**
createTimeStr	Time at which the stream was created, in wallclock format.
byteLength	Length of the stream in bytes
bitrate	Average bit rate over the life of the stream, in bits per second (bps)
presRate	Present bit rate
capabilities	Capabilities of the stream; this is a reserved parameter
milliseconds	Total length of video in milliseconds (ms); this is always 0 for an unbounded streams
frameHeight	Default height of the video frame
frameWidth	Default width of the video frame
aspectRatio	Aspect ratio of the video

Parameter	Description
frameRate	Current frame rate of the video, expressed in frames per second
contentStatus	Indicates the status of the content stream. Possible values are: ■ 0: Unknown ■ 1: Stream is from a local file ■ 2: Stream is stored on disk on the server ■ 3: Stream is stored on Hierarchical Storage Manager (HSM) on the server ■ 4: Stream is a server feed ■ 5: Stream is unbounded (that is, a rolling feed with no determinate end) ■ 6: Stream is a wide network feed, such as multicast
protocol	Network protocol by which the stream is being delivered. ■ 0: Protocol is unknown ■ 1: Stream is a local file (that is, there is no network protocol) ■ 2: Delivery is through UDP ■ 3: Delivery is through TCP ■ 4: Delivery is through HTTP ■ 5: Delivery is through ATM

GetInfoVT()

Syntax:
```
GetInfoVT(streamName as variant,
          url as variant,
          title as variant,
          fmtType as variant,
          extType as variant,
          createTimeStr as variant,
          byteLength as variant,
          bitrate as variant,
          presRate as variant,
          capabilities as variant,
          milliseconds as variant,
          frameHeight as variant,
          frameWidth as variant,
          aspectRatio as variant,
          frameRate as variant,
```

```
contentStatus as variant,
protocol as variant)
```

This method returns information about the current media file.

Important: Make sure you declare all of the variables that you pass as parameters, and initialize all of the strings to null strings, before calling **GetInfoVT()**.

If the call to **GetInfoVT()** succeeds, it sets the variables passed as parameters to the indicated data and returns **1**. Otherwise **GetInfoVT()** returns **0** and the parameters do not contain valid information about the stream. You should make sure you check the return value before relying on the data contained in the variables.

The parameters contain the following information after a successful **GetInfoVT()** call:

Parameter	Description
streamName	The unique name of the stream
url	The URL or asset cookie of the stream
title	The stream title or description (not unique)
extType	File extension; for example, **mpi**, **mpg**, or **osf**
fmtType	A readable string version of **extType**
createTimeStr	Time at which the stream was created
byteLength	Length of the stream in bytes
bitrate	Average bit rate over the life of the stream, in bits per second (bps)
presRate	Present bit rate
capabilities	Capabilities of the stream; this is a reserved parameter
milliseconds	Total length of video in milliseconds (ms); this is always 0 for unbounded streams
frameHeight	Default height of the video frame
frameWidth	Default width of the video frame
aspectRatio	Aspect ratio of the video

Parameter (Cont.)	Description (Cont.)
frameRate	Current frame rate of the video, expressed in frames per second
contentStatus	Indicates the status of the content stream. Possible values are: ■ 0: Unknown ■ 1: Stream is from a local file ■ 2: Stream is stored on disk on the server ■ 3: Stream is stored on Hierarchical Storage Manager (HSM) on the server ■ 4: Stream is a server feed ■ 5: Stream is a rolling feed ■ 6: Stream is a broadcast or multicast stream
protocol	Network protocol by which the stream is being delivered. ■ 0: Protocol is unknown ■ 1: Stream is a local file (that is, there is no network protocol) ■ 2: Delivery is through UDP ■ 3: Delivery is through TCP ■ 4: Delivery is through HTTP ■ 5: Delivery is through ATM

GetPos()

Syntax: `GetPos(posfmt as integer) as integer`

Returns the current stream position. The **posfmt** parameter defines the format used to query the position. Possible values are:

Value	Description
1	Returns the current position in milliseconds
2	Returns the current position as the number of frames from the beginning
8	FMT_EPOCH is seconds from epoch.

To return a value in a variable, use the following syntax:

```
retpos = ovcax1.GetPos(posfmt as integer) as integer
```

So

```
retpos = ovcax1.GetPos(1) as integer
```

returns the current position in milliseconds. It returns **0** if:

- the call fails, or
- the video is at the beginning but hadn't been played (such as when first loaded).

GetStats()

Syntax:
```
GetStats(dropPkts as long,
         rcvdPkts as long,
         freeBytes as long,
         readBytes as long,
         streamState as long,
         streamSize as long,
         curPos as long,
         minPos as long,
         maxPos as long)
```

Returns statistics about the current stream.

> **Note:** Make sure you declare all of the variables that you pass as parameters before calling **GetStats()**.

If the call to **GetStats()** succeeds, it sets the variables passed as parameters to the indicated data and returns **1**. Otherwise **GetStats()** returns **0** and the parameters do not contain valid information about the stream. Make sure you check the return value before relying on the data contained in the variables. **GetStats()** returns **0** if no video is loaded.

The parameters contain the following information after a successful **GetStats()** call:

Parameter	Description
dropPkts	Indicates the number of packets dropped since stream creation
rcvdPkts	Indicates the number of packets received since stream creation
freeBytes	Indicates the number of free bytes in the local cache
readBytes	Indicates the number of bytes in the cache ready for decoding

Parameter (Cont.)	Description (Cont.)
streamState	Indicates the current stream state; valid values are described in the **State** property on page C-19
streamSize	Contains the overall size of the stream in milliseconds
curPos	Indicates the current stream position in milliseconds
minPos	Indicates the minimum (earliest) seekable stream position (similar to **PlayFrom**) in milliseconds
maxPos	Indicates the maximum stream position (similar as **PlayTo**) in milliseconds

GetStatsVT()

Syntax:
```
GetStatsVT(dropPkts as variant,
           rcvdPkts as variant,
           freeBytes as variant,
           readBytes as variant,
           streamState as variant,
           streamSize as variant,
           curPos as variant,
           minPos as variant,
           maxPos as variant)
```

Returns statistics about the current stream.

> **Note:** Make sure you declare all of the variables that you pass as parameters before calling **GetStatsVT()**.

If the call to **GetStatsVT()** succeeds, it sets the variables passed as parameters to the indicated data and returns **1**. Otherwise **GetStatsVT()** returns **0** and the parameters do not contain valid information about the stream. Make sure you check the return value before relying on the data contained in the variables. **GetStatsVT()** returns **0** if no video is loaded.

The parameters contain the following information after a successful **GetStatsVT()** call:

Parameter	Description
dropPkts	Indicates the number of packets dropped since stream creation
rcvdPkts	Indicates the number of packets received since stream creation
freeBytes	Indicates the number of free bytes in the local cache
readBytes	Indicates the number of bytes in the cache ready for decoding
streamState	Indicates the current stream state; valid values are described in the **State** property on page C-19
streamSize	Contains the overall size of the stream in milliseconds
curPos	Indicates the current stream position in milliseconds
minPos	Indicates the minimum (earliest) seekable stream position in milliseconds
maxPos	Indicates the maximum (latest) stream position in milliseconds

GetVol()

Syntax: `GetVol() as integer`

Returns the current volume setting of the player as an integer from 0 to 100.

To return a value in a variable, use the following syntax:

```
retvol = ovcax1.GetVol() as integer
```

ImportFileAs()

Note: The **ImportFileAs()** and **ImportStreamAs()** methods are being deprecated.

Syntax: `ImportFileAs() as Boolean`

Presents a standard dialog window from which the user can pick a video stored on the local machine. When the user chooses a video and clicks the OK button, this method:

- Sets the **Mediafile** property to the name of the selected file.
- Calls **Load()** to establish a link between the Oracle Video Client software and the video file; this process does not take as long as preparing video from a server.
- Returns **1**.

If the user clicks the **Cancel** button or if the call fails, **ImportFileAs()** returns 0.

This sample code could be associated with a button that automatically plays the video the user selected.

```
Private Sub Command1_Click()
  ovcax1.ImportFileAs
  ovcax1.Play
End Sub
```

To return a value in a variable, use the following syntax:

```
retfile = ovcax1.ImportFileAs()
```

ImportStreamAs()

Note: The **ImportFileAs()** and **ImportStreamAs()** methods are being deprecated.

ImportStreamAs() returns error 46 unless the OVM.query_proto preference is set to **ovm.** For information about setting OVC preferences, see "OVC Preferences" on page 5-5.

Syntax: `ImportStreamAs(server_addr as string) as Boolean`

Presents a file dialog window from which the user can pick and play a video located on the Oracle Video Server specified by the **server_addr** parameter. You can pass a server name in the **server_addr** parameter or pass a null string (""). The null string indicates that this method should use the default server.

When the user chooses a video and clicks the OK button, this method:

- Sets the **Mediafile** property to the name of the selected file.

- Calls **Load()** to establish a link between the Oracle Video Client software and the selected file.
- Returns **1**.

If the user clicks **Cancel** or if the call fails, **ImportStreamAs()** returns **0**.

To return the value in a variable, use the following syntax:

```
retstrm = ovcax1.ImportStreamAs("server_addr")
```

Load()

Syntax: `Load(media_url as string) as Boolean`

Loads the movie specified by the **media_url** parameter. If the **Load()** method is successful, the **Mediafile** property is set to **media_url**. **Load()** returns **0** if the call fails or if **media_url** is invalid or unspecified. Otherwise **Load()** returns **1**.

To return the value in a variable, use the following syntax:

```
retload = ovcax1.Load("media_url")
```

Pause()

Syntax: `Pause`

Pauses playback, preserving the current position. You can only call this method while the player is in the play state. The current frame remains displayed.

Play()

Syntax: `Play`

Starts stream playback. If the properties **PlayFrom** and **PlayTo** are not specified, the playback starts at the beginning of the stream and continues until the end. Otherwise playback starts at the position specified by **PlayFrom** and finishes at the position set by **PlayTo**. See the descriptions of **PlayFrom** and **PlayTo** for proper format information.

You can only call this method while the player is stopped. To start playing while paused, call the **Resume()** method.

Resume()

Syntax: `Resume`

Resumes playback. This call can only be invoked from the paused state.

Rewind()

Syntax: `Rewind`

Rewinds the movie to the beginning or the position set by the **PlayFrom** property.

SetPos()

Syntax: `SetPos(posfmt as integer, streampos as long)`

Seeks the stream to the position specified by the **streampos** parameter. The **posfmt** parameter specifies the format used to set the position. The possible values for **posfmt** are the same as those used for **GetPos()**.

SetVol()

Syntax: `SetVol(volume as integer)`

Sets the volume of the player as an integer from **0** (inaudible) to **100** (maximum).

ShowInfoDialog()

Syntax: `ShowInfoDialog`

This method displays a dialog box showing information about the current media file. See **GetInfo()** for a description of the various statistics displayed.

ShowStatsDialog()

Syntax: `ShowStatsDialog`

This method displays a dialog box showing the current stream statistics. See **GetStats()** for a description of the various statistics displayed.

Stop()

Syntax: `Stop`

Pauses playback, resetting the current position to the beginning of the stream. If a start position was specified by the **PlayFrom** property, stop resets the start position to "default," and resets **PlayTo** to "end." This call can only be called from the playing state. The current frame remains displayed.

Unload()

Syntax: `Unload`

Frees all resources allocated within the call to the **Load()** method. If you call this method while a stream is playing, the video is stopped and unloaded, and the video rectangle is blanked.

Properties

In addition to the standard Visual Basic or Visual C++ object properties, the Oracle Video ActiveX Control uses these properties to control aspects of the video, its playback, and its external appearance. All properties are writable, unless identified as “read-only”.

AutoStart	**KeepAspectRatio**	**ShowPosition**
BorderStyle	**Loop**	**ShowPositionAndStatus**
EnableLeftClick	**Mediafile**	**ShowStatus**
EnablePopup	**PlayFrom**	**SkipIncrement**
IsLoaded	**PlayTo**	**State**
	ShowControls	**TimerFrequency**

All of these properties except for those that are read-only can be set when embedding the Oracle Video ActiveX Control into an HTML document through the use of **<param>** tags with the **<object>** block (between the **<object>** and **</object>** tags). The **<param>** tags have three elements:

- The **<param>** tag
- The property name, such as **AutoStart** or **Mediafile**:

  name="*propName*"

- The property value; for string or numeric properties, this should just be the string or value, while for Boolean properties, you should specify **1** for true or on and **0** for false or off.

  value="*value*"

For example, to set the video session URL, the **<param>** tag would look something like this:

```
<param name="Mediafile" value="vsi://server:5000/video.mpi?data_proto=tcp">
```

You can also set the values of the writable properties in the Designer component of application development tools like Visual Basic and Visual C++. The procedure for this depends on the tool being used.

AutoStart

Type: Boolean

Specifies whether stream playback begins automatically once the stream is finished loading. If true, video playback starts as soon as the load completes. The default value is **0**.

BorderStyle

Type: Boolean

Specifies whether there is a border around the video screen. The border is two pixels on each edge. Possible values are:

- **0** specifies no border.
- **1** specifies that the border should be displayed; this is the default.

EnableLeftClick

Type: Boolean

Specifies whether a left click on the video screen toggles Play and Pause. If **EnableLeftClick** is **1**, left clicking in the video screen has the same effect as clicking on the Play / Pause button. The default value is **1**.

> **Note:** If **EnableLeftClick** is **1**, the control does not return the **LeftClick** event since it is handled by the video screen.

EnablePopup

Type: Boolean

If **EnablePopup** is **1**, the pop-up menu appears when the user right-clicks the video area. The default value for **EnablePopup** is **1**.

> **Note:** If **EnablePopup** is **1**, the control does not return the **RightClick** event since it is handled by the pop-up.

IsLoaded

Type: Boolean

A read-only property that indicates whether the Oracle Video ActiveX Control has a stream loaded.

KeepAspectRatio

Type: Boolean

A property that, when set to **1** (for true) maintains the aspect ratio of the encoded video in the playback region during normal play and when the Client window is resized. The option, **KeepAspectRatio**, can be set to **0** (for false) or **1** (for true). The default value for **KeepAspectRatio** is **1**.

Loop

Type: Boolean

Specifies whether the movie is to be rewound when the end or the position specified by the **PlayTo** property is reached. If **PlayFrom** is specified, the movie is rewinded at the **PlayFrom** position instead of the beginning. The default value for **Loop** is **0**.

Mediafile

Type: String

Contains a file on the local file system or the video session URL of a file on the Oracle Video Server. See Appendix D, “Oracle Video Session URLs” for the complete syntax for video session URLs.

PlayFrom

Type: `String`

Indicates the position from which to start playback when Play, Rewind, or Loop actions are performed. You can specify the value for this property in one of three formats:

Format	Description
Literal	There are two literal values you can use for **PlayFrom**: ▪ **beginning** indicates that the start of play should be at the beginning of the file or stream. ▪ **default** indicates that play should start at the head of the rolling feed, or the beginning of any other kind of content. ▪ **end** indicates that play should start at the end of the stream.
wallclock	A string representing wall clock time and date, formatted as follows: *wallclock* = *time* | *date time* | *date time zone* where *time* = *hh*:*mm*[:*ss*] *date* = *dd month yyyy* *zone* = +|-*hhmm* | UTC | GMT *month* = Jan | Feb | Mar | Apr | May | Jun | Jul | Aug | Sep | Oct | Nov | Dec Hours are specified as 0-23. If seconds are not set, they are set to zero. For example, **00:02:40** starts the stream two minutes, forty seconds from the beginning of the stream (160 seconds). For another example, **10 Jul 1999 22:30 UTC** starts the stream on the 10th of July at 10:30 PM Universal Time (which is the same as Greenwich Mean Time).
position	A single string value that indicates the position in milliseconds. For example, **10000** indicates that the position from which to begin is 10 seconds (10,000 milliseconds) from the beginning.

If **PlayFrom** is not specified, it defaults to "default" for regular content or the head of a real-time feed.

PlayTo

Type: String

Indicates the position where playback should be stopped when **Play** or **Forward** actions are performed. You can use the same formats for this as for **PlayFrom**.

If **PlayTo** is not specified, it defaults to "end."

ShowControls

Type: Boolean

Specifies whether the playback controls are visible or hidden. This is the master property. When **ShowControls** is **1**, the controls selected by **ShowPosition**, **ShowPositionAndStatus**, and **ShowStatus** are visible; when **0**, all controls are hidden. The default value is **1**.

ShowPosition

Type: Boolean

Specifies whether the controller is visible or hidden when the **ShowControls** property is **1**. The default value is **1**.

ShowPositionAndStatus

Type: Boolean

Specifies whether the controller and status line are visible or hidden when the **ShowControls** property is **1**. The default value is **1**.

ShowPositionAndStatus is included for backward-compatibility only. Use the **ShowPosition** and **ShowStatus** properties instead.

ShowStatus

Type: Boolean

Specifies whether the status line is visible or hidden when the **ShowControls** property is **1**. The default value is **1**.

SkipIncrement

Type: Integer

Specifies the rewind and fast forward skip increment for full-screen mode playback. The skip increment is the amount the client skips backward or forward when you press the left or right arrow keys when the client is in full-screen mode. The property, **SkipIncrement**, can be set as a percentage of the length of the video clip. Valid values are **0** to **100**. So, if the video is a 120-minute movie and the **SkipIncrement** value is set to **1**, seeking once forward will cause the video to advance 1 minute and 12 seconds. The default value for **SkipIncrement** is **10**.

State

Type: Integer

A read-only property that indicates the state of the control. Possible values are:

State	Indicates the player...
0	Has no video loaded.
1	Has loaded a stream, but not locked resources for playback.
2	Has loaded a stream and locked the resources necessary for playback.
3	The player is currently playing a stream.
4	The player is paused.
5	The player has reached the end of the stream.
6	The player has encountered an error.

TimerFrequency

Type: Integer

Specifies the interval in milliseconds at which the position indicator (such as the time counter and seek slider) is updated in the Oracle Video ActiveX Control. Reasonable values are from 100 to 1000 milliseconds.

Note: Ordinarily, do not set the interval to anything less than 100 milliseconds because the multiple screen repaints necessary to update the position informations can seriously degrade video rendering performance.

Events

Events are messages that the Oracle Video ActiveX Control sends to your application. Your application can then perform specific actions in response to these events. For example, the **Stopped** event is sent to your application when the video stops playing.

The following events are defined by the Oracle Video ActiveX Control:

Completed	**PlayStarted**	**RightClick**
LeftClick	**Resumed**	**Stopped**

Completed

Sent when the video stream reaches its end or the position indicated by the **PlayTo** property, if set.

In response to **Completed**, you might want to call **Load()** to prepare the next stream (if you know what it is) so that it's ready to play when the user wants it.

LeftClick

Sent when the user clicks the left mouse button while the cursor is over the video screen portion of the Oracle Video ActiveX Control. This is only sent when a stream is loaded.

> **Note:** This event is not sent when the **EnableLeftClick** property is **1**, since the **LeftClick** event is handled by toggling **Play** and **Pause**.

PlayStarted

Sent when the video is started in response to a call to the **Play()** method.

> **Note:** When a paused video is resumed, the Oracle Video ActiveX Control does not send a corresponding **PlayStarted** event. Instead it sends **Resumed**.

Resumed

Sent when video playback is resumed in response to a call to the **Resume()** method.

RightClick

Sent when the user clicks the right mouse button while the cursor is over the video screen portion of the Oracle Video ActiveX Control. This is only sent when a stream is loaded.

> **Note:** This event is not sent when the **EnablePopup** property is 1, since the **RightClick** event is handled by opening the pop-up menu.

Stopped

Sent when the video is stopped, in response to a call to the **Stop()** method, or in response to a **Pause()** method.

In response to the **Stopped** event, you might want to prepare the next stream (if you know what it is) so that it's ready to play when the user wants it.

<Object> Attributes and Parameters

HTML defines a number of standard attributes that you can use in an **<object>** statement to specify the behavior of the browser and how the browser displays your embedded control. Only a subset of these standard attributes affect the Oracle Video ActiveX Control. These include:

ClassID **ID**

CodeBase **Width**

Height

The Oracle Video ActiveX Control recognizes a number of parameters that affect the behavior of the embedded control. These set the control's properties and use the same names as the properties. See "Properties" on page C-14 for more information on setting the control properties.

ClassID

ActiveX control class identifier. This should *always* be the same value:

```
CLSID:547A04EF-4F00-11D1-9559-0020AFF4F597
```

If you're creating your HTML page by hand, you'll have to type this value in manually. If you're using a composition tool such as Microsoft FrontPage or ActiveX Control Pad, this is usually inserted automatically when you place the control on the page.

You can find this value in the Windows registry by searching under HKEY_CLASSES_ROOT for **ovcax.ovcax**. This contains a single entry named **CurVer**, which points to the entry for the current version of the control. Find that entry, which in turn contains a single entry, the class ID.

CodeBase

ActiveX control code base parameter. This is a *URL* value that points to a location of the latest version of the Oracle Video Client (OVC) installation program, *and* a *version number* to compare to the version of OVC that is currently installed on the user's machine. The format of this parameter is:

```
CodeBase="http://server/directorypath/ovce.exe#version=3,2,0,0"
```

The browser compares the version of the OVC ActiveX Control installed on the local machine with the version listed in the **version** string (in this case, **3,2,0,0**). If the installed version is older, the installation program listed in the CodeBase URL

(in this case, **http://*server/directorypath*/ovce.exe**) installs the newer version of the ActiveX Control automatically.

Also, for the **version** string, Microsoft's Internet Explorer browser supports a four-digit, comma-separated version notation (3,2,0,0), but Oracle uses a five-digit version notation. To convert the five-digit notation to the four-digit notation required by IE, use the following formula.

4th digit of four-digit notation = digit4 x 100 + *digit5*

The fourth digit of the four-digit notation is the sum of the Oracle fourth digit times 100 plus the fifth digit.

Height

Indicates the height in pixels to be reserved to display the control. If the **ShowControls** and **BorderStyle** attributes are not set to true, then this indicates the actual height of the plug-in. If the **BorderStyle** attribute is set, then:

1. The border thickness is 2 pixels, and there are two borders (the top and bottom borders), so, the height of the borders is 4 pixels.
2. Add this to the height that you want the video screen.
3. Use this number for **height**.

If **ShowControls** is turned on, and:

- If **ShowPosition** is set to true, add 26 pixels, and/or
- If **ShowStatus** is set to true, add 24 pixels, or
- If **ShowPositionAndStatus** is set to true, add 50 pixels.

For example, if you have content that is is 200 pixels high plus you want a border and the controls, slider bar, and volume control displayed, you calculate the total height of the OVC ActiveX Control as follows:

1. The border width is 2 on each edge, so add this to the desired height of the screen (200 + 4 = 204).
2. Add 26 pixels for the controller (204 + 26 = 230).

So, you should set the height to 230 for the OVC ActiveX Control.

> **Note:** It is important to set the height and width properly for the OVC ActiveX Control. If the height and width are not set correctly, the video is size has to be recalculated on playback, and that seriously affects performance. A video that normally plays at 30 frames per second will probably only playback at 16 frames per second if the height and width are not set correctly.

ID

A string identifier used to refer to the embedded control. Use this when you have more than one control or object in your document or when you want to script an object.

Width

Indicates the width in pixels to be reserved to display the control. If the control includes a border (if **BorderStyle** is set to 1), add four pixels to the screen width to get the width of the OVC ActiveX Control.

> **Note:** See the important note for Height.

D

Oracle Video Session URLs

This appendix describes how to view media resources with the Oracle Video Client. OVC can access media resources (video and audio data) directly from disk, but usually it receives streams of data from the Oracle Video Server. OVC specifies which media resource to view, and it does this using a Video Session Uniform Resource Locator, or URL, also referred to as the *mediafile*.

This appendix consists of the following sections:

- What Is the Video Session URL, or Mediafile?
- The Video Session URL Syntax
- Shortcuts and Examples

If you are familiar with Oracle Video Session URL and simply want to check syntax, skip ahead to the "Shortcuts and Examples" section on page D-13, which contains a collected Video Session Grammar and examples.

What Is the Video Session URL, or Mediafile?

Definition

The Video Session URL conceptually represents a session between a Video Server and a Video Client. The session describes the **media resource** (video and/or audio data usually streamed from the Server to the Client), the **network connection** between the two and any other **settings necessary to stream the data**. In other words, the Video Session URL represents the content to play on the Client, and other necessary information, if there is any.

To see an example, below is the Video Session URL to play the movie *Gone With the Wind*, stored as logical content on Video Server **ovsserver1** on the default domain running on port **8731** as **GoneWithTheWind** via the **ATM AAL-5** protocol:

Example D–1 Video Session URL for GoneWithTheWind

```
vsi://ovsserver1:8731/vscontsrv:GoneWithTheWind?data_proto=aal5
```

Typically, the Oracle Video Client Web Plug-in, ActiveX Control or Java libraries are used to construct a web page or application that specifies what content the end user wishes to view. Developers are responsible for creating an interface that enables end users to choose content in a simple and straightforward manner, then translates that information into Video Session URLs that the Oracle Video Server or the Web / Application server understand.

Two Types of Video Session URLs

The Oracle Video Client supports two types of sessions with the Oracle Video Server: **Broadcast** (also called *multicast*) and **Interactive** (also called *unicast* or *pointcast*).

Broadcast sessions resemble traditional analog video broadcasts and pay-per-view transmissions, except that they are digital.

Interactive sessions are analogous to “VCR” and Video-on-Demand (VOD) sessions, except in digital format. For Interactive sessions, end users are often able to pause, rewind and seek forward. Additional interactivity can be provided through HTML- or application-based functionality, such as using Javascript or ActiveX Controls on a web page, or provided in a Visual Basic, Visual C++ or Java application.

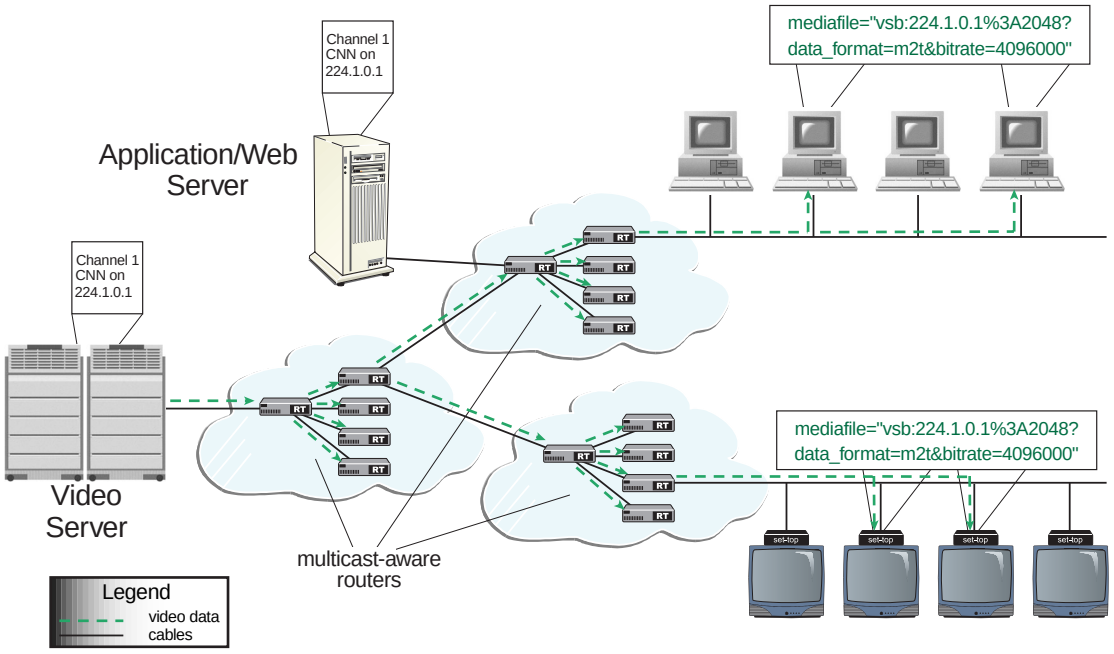
In the following descriptions of Interactive and Broadcast sessions, notice the balloons above the PC and set-top box clients. They contain Video Session URLs.

Broadcast

In Broadcast sessions, one media resource on an Oracle Video Server is simultaneously distributed to many Clients, as shown in Figure D–1, "Broadcast (Multicast) Network View". In this illustration, note how the CNN channel is distributed to all those clients that wish to view it. The Oracle Video Server

provides only one stream of the CNN channel, but it is duplicated and distributed to those machines “tuned in” to the multicast channel by multicast-aware routers.

Figure D–1 Broadcast (Multicast) Network View



The Oracle Video Server streams video and audio data, such as a real-time broadcast of a live event, or a pay-per-view movie, to a series of multicast-aware routers. Those routers duplicate the received signal and send it to other routers that my also duplicate it. This duplication process can convert one data stream to thousands, and conceivably millions. End users access the multicast streams through web pages or applications.

In Multicast transmissions, end users typically do not have interactive control over the video / audio data channel, so there is no or little control data transmitted from the Video Clients to the Video Server. End users will likely watch these channels just as they do analog programming channels or pay-per-view events.

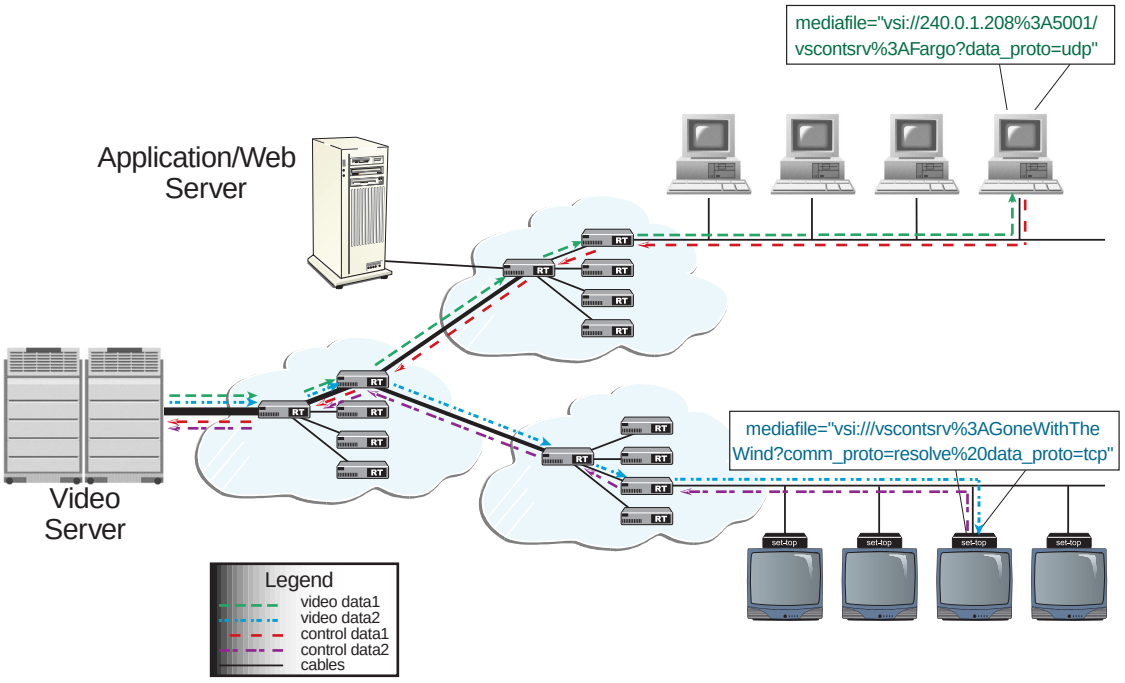
The cost of such transmissions is relatively low at the Video Server because bandwidth requirements are low and one server can support many Broadcast channels. However, the cost of the technology (the multicast-aware routers) required in the network to duplicate the signal should be noted.

Interactive

In Interactive transmissions, for every Client there is one media resource and a unique stream from the Video Server.

In this illustration, two end users have requested movies: *Fargo* and *Gone With the Wind*, as you can see in the Video Session URL balloons above the machines. Note that they have some control over the data stream they receive through the control data that is transmitted back to the server. Users might have the ability to pause, seek forward or rewind, or stop the video at any time.

Figure D–2 Interactive (Unicast) Network View



The bandwidth required for Interactive sessions is relatively high at the Video Server, and relatively high in the network. However, these users can be charged for these special services.

What the Video Session URL Specifies

The Video Session URL can specify many things; not all are required:

- The scheme the Video Client should use for the data stream (Broadcast or Interactive)
- The address and port of the server instance or gateway
- The media resource to be played
- The destination data transport framing, protocol and address
- The communications protocol to use (RTSP, Resolver, OMN) to communicate with the Video Server
- The bit rate of the data transport to be used
- The authorization credentials to be supplied with setting up a request
- Any parameters necessary for multicast sessions to function
- Potential presentation options (for example, **autoStart**, or **playFrom**)

Default values can be used for practically all parts of the URL, except for the media resource to be played.

How Video Session URLs are Used

The different interfaces of the Oracle Video Client all utilize the Video Session URL or mediafile, but in different ways:

- Using the Oracle Video Web Plug-in, the **mediafile** attribute of the **<embed>** tag is a Video Session URL.
- With the Oracle Video Java Library, the **Player**.**load()** method takes a Video Session URL as a parameter.
- In the Oracle Video ActiveX Control, the **Mediafile** property contains the Video Session URL of the currently loaded stream.

The Importance of Video Session URLs

The Video Session URL is undoubtedly the most important attribute, property or method you must specify to view video data using the Oracle Video Client interface. If the Video Session URL is not specified correctly, you will not see any video content, and you will have no idea where the problem is.

The Developer installation of the Oracle Video Client optionally installs several pieces of local content along with the Client. After installation, you can check to make sure the Client is working properly by playing this local content. Fortunately, you don't have to be an expert at specifying the Video Session URL to do this.

Run the Oracle Video Player (**Start** | **Programs** | **Oracle Video Client** | **Oracle Video Player**), then select the **Local** option in the Load Content dialog box (accessed by clicking the **File** menu and selecting **Load Content**) and click the **Browse** button. The content is installed in your **%ORACLE_HOME%\ovc\demo\content** directory. Select a file and the Oracle Video Client Player initializes it. Press the **Play** button to view it. If that content plays properly, you know the Client is installed properly and that it works.

If the Client is working, then the Video Session URL must be correct to play video/audio received from a Video Server. The other attributes, such as height, width, whether or not controls or status information is displayed, etc., can be wrong, but video will still play — not optimally, but it will still play. If the URL is wrong, the Client does not play, because it receives no data. If the Video Session URL is correct and the Client works, and video still does not play, you can be certain the problem is either in the network, at the Web/Application server, or at the Video Server.

The Video Session URL Syntax

The Video Session URLs are just like other URLs by design. If you have ever used a browser, you are familiar with the URL syntax for web pages. For example:

```
http://www.oracle.com/asd/video/video.html
```

For this browser URL, the first part of the URL is **http**, which tells the browser to use the HyperText Transport Protocol to make the connection, followed by a standard delimiter (**://**), followed by the domain (**www.oracle.com**), followed by a location on the Web server for the file the browser should open (**/asd/video/video.html**).

Video Session URLs work the same way as browser URLs. To understand the specifics of the URL syntax, let's first look at the generic syntax.

Generic Syntax

The Video Session URL syntax follows the guidelines for URL schemes defined by the Internet Engineering Task Force (IETF), the standards body of the Internet. According to those guidelines URLs follow the basic syntax (ignore the spaces) of:

scheme : *scheme-specific-part*

In addition, a large class of them follow the generic syntax of

scheme :// *authority path* ? *query*

where one or more of the parts and delimiters may be optional.

Scheme

OVC supports a URL scheme syntax called the Video Session URL, or *vsurl*.

Considering Interactive (unicast) and Broadcast (multicast) URLs, some parameters are required in one case, but make no sense in the other. For example, an asset name (such as the name of the movie to be played) is required in Interactive transmissions, but makes no sense in Broadcast. So, the basic notation (the pipe | character is used as a separator through these explanations, meaning OR) for the Video Session URL consists of three main parts:

vsurl = *interurl* | *bcasturl* | *file*

where

interurl = vsi :// [*host[:port]*] / *asset* [? *params*]

bcasturl = vsb : *address* [? *params*]

file = file :// *asset*

As you can see, OVC supports two schemes for streaming media resources from a Video Server, **vsi:** and **vsb:** for Interactive and Broadcast sessions, respectively — effectively, the *url* of vs*url* is either i for interactive or b for broadcast. OVC also supports a deprecated *interurl* syntax, as described in “Deprecated vsproto Scheme for interurl” on page D-8. Finally, OVC supports the standard “file” specification URL for "receiving" data from the file services.

The **vsi:** scheme follows the generic syntax, where the *scheme* is **vsi://**, the *authority* is the optional server address (an IP address or DNS server name and optional port), the *path* consists of a single required / delimiter followed by the (also required) name of the asset, an optional ? delimiter followed by a series of

name=value pairs or *params* (as described in detail in “Parameters” on page D-10), and / or the & delimiter following standard HTTP CGI notation conventions.

The **vsb:** scheme uses Broadcast *address* in place of the *authority* and *path* fields, and share the ? delimiter and *name=value* pairs or *params* with the **vsi:** scheme.

All unsafe characters in any of the above fields (authority, path, or Broadcast *address*) must be escaped, as discussed in “Escaping” on page D-12.

Deprecated *vsproto* Scheme for *interurl*

Besides the *interurl* syntax described above, OVC supports another syntax for the scheme that it has been used for years. This syntax will be supported until it can safely be abandoned. Oracle recommends that you do not use this syntax and that you update your existing VSURLs to the new syntax.

The basic format of the deprecated syntax is:

```
interurl  =   [vsproto :// ] [ host[:port] ] |  / asset [ ? params ]
```

where the scheme is:

```
vsproto   =   vsudp | vstcp | vsal5
```

This determines the protocol used to transmit media data (not the protocol of an Oracle Media Net connection, which handles control and setup of the data stream). If *vsproto* is specified (it is optional), two forward slashes must appear between protocol and the MDS path and filename or the logical content asset cookie.

OVC supports three protocols, as shown in the *vsproto* syntax. These protocols are:

- **UDP**: This specifies the User Datagram Protocol (UDP), which is the default protocol. UDP does not retransmit dropped packets, so this protocol may not be effective for networks that tend to drop packets because of limited bandwidth.
- **TCP**: This specifies the Transport Communication Protocol (TCP). TCP tracks and retransmits dropped data. TCP is a relatively old but widely used protocol and has been known as the protocol of the Internet.
- **AL5**: This specifies the Asynchronous Transfer Mode (ATM) protocol. ATM is a relatively new and powerful protocol designed for use with high bandwidth networks, but which carries data in 53 byte chunks. The fixed length of chunks or *cells* means that it can contain practically any kind of information, including encoding the TCP or UDP protocols.

Note: Because the protocol is specified as part of the scheme, it should not be specified as a parameter.

Authority (Host and Port/Address)

Interactive For Interactive streams, the *authority* part of the URL consists of [*host[:port]*], which are optional. If they are not specified, the default values are used. If the host (Video Server) name and port number are not specified, the default address is used. If the server and port are defined elsewhere (such as in the default network settings for the Oracle Video Player), the *host*:*port* can be omitted.

Note: To reduce dependencies on the Domain Name Service (DNS) and blocking DNS timeouts on computers, machines should be referred to by IP address, not hostname, where possible.

Broadcast For Broadcast streams, the *authority* and *path* fields are replaced with the Broadcast *address* for the multicast session.

Path (Asset)

Interactive The asset for Interactive streams consists of two parts:

- [/vscontsrv: | /mds:] The service to use to interpret the logical content asset cookie is vscontsrv. This must be included if the media resource is logical content. The mds designation can be used to directly access the media resource file stored on an Media Data Store volume on the server. These are not used with the file:// scheme.
- *media_resource*: The MDS path and file or the logical content asset cookie you want to play from the Oracle Video Server. This is required.

Broadcast As noted previously, for Broadcast streams, the *authority* and *path* fields are replaced with the Broadcast *address* for the multicast session.

Parameters

The final part of the generic Video Session URL syntax is for parameters. The client ignores any parameter it does not recognize. Parameters MUST NOT appear more than once in a given URL. Parameter names are case-insensitive; case does not matter. The following parameters are defined.

authid=*authid*

authid = *pval*

This allows the application to specify a username to be provided to the RTSP authorization mechanism introduced with the 3.2 release of OVS. The value of this field is used for the username field of the RTSP digest authorization challenge.

authpw=*authpw*

authpw = *pval*

This allows the application to specify a password to be provided to the RTSP authorization mechanism introduced with the 3.2 release of OVS. The value of this field is used for the password field of the RTSP digest authorization challenge.

bitrate=1*digit* This is the bit rate to request on the connection and to allocate on the server side, in bits per second (bps). The server is free to ignore this parameter, or to interpret it as it deems most appropriate. This may be the bit rate of the requested asset, but may also be a number higher or lower. This parameter is optional; if not specified, the server determines the bitrate based on its configured bitrate value. This parameter may be used with the Broadcast URL as well, in which case it indicates the incoming bit rate of the multicast stream. This parameter defaults to 4 Mbps if not specified for broadcast URLs. This is only used to determine buffering values. There is no need to change this value unless the bitrate of the broadcast is greater than 4 Mbps. If the broadcast is greater than 4 Mbps, the bitrate field should be specified.

comm_proto=*cproto*

cproto = omn | resolve | rtsp | *pval*

This is the control protocol to use to communicate with the server. This parameter is case-insensitive, and is ignored by broadcast URLs. This parameter is optional; if not specified, a default value will be used instead. See “Customizing OVC using the OVC Configuration and Installation Plug-in” on page 5-2 for information about setting persistent default values.

transport=*trans*

trans = *pval*

This field follows the definitions for the Transport: header in the OVS extended RTSP specification (see the *Oracle Video Custom Client Developer's Guide*). It is included to provide maximum flexibility and application-level control of transport parameters. This parameter is optional; if not specified, application-specific defaults are used.

data_proto=*dproto*

dproto = tcp | udp | aal5 | *pval*

This field maps the existing vsudp, vstcp, and vsal5 protocol fields. If not specified, the default prototype for the client is used. Upon initial installation, the default prototype is UDP, but you can update the default value. See "Customizing OVC using the OVC Configuration and Installation Plug-in" on page 5-2 for information about setting persistent default values.

data_address=*addr*

addr = *family-addr*

addr is a standard OVS-parsible address string (as used by **mtpa**, **mzd**, and **::mzc::netproto**). For Interactive sessions, this represents the destination address of the data stream, and usually is an address on the Client. This parameter is optional for Interactive sessions; if not specified, an application-specific address is used. This parameter is redundant for Broadcast sessions (given that the mcast `family-addr` is required), and is ignored in a Broadcast session.

data_format=*dfmt*

dfmt = osf | m1s | m2t | m2p | wav | avi | mov | *pval*

This field represents the data format of the stream (often referred to as the "container" format). This field is case insensitive. It is required for Broadcast sessions, where the content type must be known up front. This field is optional for Interactive streams, where the server typically will inform the client of the data format of the streams. If specified for Interactive streams, the semantics are undefined. If this field is not specified for a broadcast URL, it defaults to **mls**.

data_framing=*dframe*

dframe = mp2t | m2t | ogf | *pval*

This indicates the transport-layer framing present over the underlying network protocol. This field is case insensitive. For Interactive streams, if specified then the framing type is requested from the server; if not specified, an application-specific default will be used. For Broadcast streams, if specified this field tells the client how the inbound stream is framed; if not specified, the client may assume a default framing type, or may figure it out on the fly from the data stream.

Escaping

The Video Session URL syntax follows current best practices for handling character sets and control characters. US ASCII characters in the 00 - 7F range (the first 128 ASCII characters) are translated without change. Characters outside that range are escaped using the standard %<*hex*><*hex*> mechanism.

The “safe range” or supported characters (urics) are:

uric = *reserved* | *unreserved* | *escaped*

where

reserved = ; | / | ? | : | @ | & | = | + | $ | ,

unreserved = *alphanum* | *mark*

escaped = % *hexdigit hexdigit*

and where

mark = - | _ | . | ! | ~ | * | ‘ | (|)

Data MUST be escaped if it does not have a representation using an unreserved character.

Fully escaped URLs can become awkward in some instances. For example, the colon character is required as part of the asset cookie field, and needs to be escaped. Whitespace is allowed in asset cookies, and needs to be escaped. This makes for a cumbersome and difficult to read (by humans) URL.

For example, to specify the asset "vscontsrv:Gone With The Wind":

vsi:///vscontsrv%3cGone%20With%20The%20Wind

So, the following extensions are implemented by a client who is handed a VSURL:

- If the client encounters the literal ':' in either the *asset* or *param* values, it handles it correctly (in other words, in that context colons are not reserved).
- If the client encounters whitespace in either the *asset*, parameter names, or parameter values, it handles it correctly. Note however that trailing whitespace (either at the end of the URL as a whole or in between fields is still not allowed. In other words, you can say

  ```
  "vsi://Gone With The Wind&param one=yadayada"
  ```

  but not

  ```
  "vsi://Sharks &param = value "
  ```

  Note: This syntax is a non-standard and is discouraged. To avoid the problems caused by these optional clauses, if an application is using automated generation of URLs, it should use the escaped-syntax version instead of this relaxed version.

Shortcuts and Examples

Shortcuts

The following are "shortcuts" supported as part of Video Session URL standard:

- The media asset is the only part of the URL required for Interactive streams. If only the asset is listed, default values or the current client settings values are used for the other parts of the URL.
- The default communication protocol is OMN (Oracle Media Net), and does not have to be overridden unless you are using some other data protocol, such as RTSP.
- The default data protocol as installed is UDP.
- Files on Media Data Store volumes can be named directly, via the **vsi://** scheme and **/mds:** prefix designation, but the file to play must include the suffix, such as **vsi:///mds:/mds/video1/GoneWithTheWind.mpi**. Files on a local disk can be named directly, via the **file://** scheme, but the file must also include the suffix, such as **file://ntpc2/content/GoneWithTheWind.mpg**.

Examples

The following examples assume that the OVC defaults are: **UDP** as the data protocol, **OMN** as the control protocol, Oracle Generic Framing (**OGF**) as the framing type, and that no data address is set (equivalently, the data address could be **0.0.0.0**:**0**).

Example D–2 An Interactive URL

```
vsi:///GoneWithTheWind
```

This is the Video Session URL for the logical content "GoneWithTheWind." Note that this format uses default values for most of the Video Session URL.

Example D–3 An Interactive URL, Including the Server and Data Protocol

```
vsi://vssrvr2/GoneWithTheWind?data_proto=tcp
```

This is the Video Session URL for the logical content "GoneWithTheWind" on **vssrvr2** via **TCP**.

Example D–4 An Interactive URL, Specifying a Logical Content Asset Cookie Including Spaces

```
vsi:///vscontsrv%3AGreat%20Caesar's%20Ghost
```

This is the Video Session URL syntax for logical content "Great Caesar's Ghost."

Example D–5 A Broadcast URL, Specifying the IP Address, MPEG2 Transport Stream and Bit Rate

```
vsb:224.1.0.1%3A2048?data_format=m2t&bitrate=2048000
```

This URL indicates an MPEG-2 transport stream @ **2 Mbps** being multicast to the IP class D address **224.1.0.1**, port **2048**.

Example D–6 An Interactive URL, Specifying a Logical Content Asset Cookie and the Control Communications Protocol

```
vsi:///vscontsrv%3AGoneWithTheWind?comm_proto=resolve
```

This is a resolver URL for the asset `vscontsrv:GoneWithTheWind`.

Example D–7 A File URL

```
file:///ORANT/demo/content/GoneWithTheWind.mpg
```

This is a URL for the sample file **GoneWithTheWind.mpg** installed in the location specified on the local file system.

Deprecated Syntax Examples

Below are some examples of possible Interactive Video Session URL values using the deprecated syntax. The first listing in each example uses a Media Data Store (MDS) file, while the second listing uses a logical content asset cookie.

Example D–8 Including the Protocol, Server, Port, and Media Resource

```
vstcp://sunserver:5000/mds/video/oracle1.mpi
vstcp://130.4.11.208:5000/vscontsrv:oracle1
```

In this example, **vstcp** specifies the TCP protocol, **sunserver:5000** specifies the server and port (temporarily overriding the **OMN_ADDR** environment variable), **/mds/video/oracle1.mpi** specifies the MDS file, and **vscontsrv:oracle1** specifies the logical content asset cookie. The second listing uses the IP address of the server, which is also valid.

Example D–9 Including the Protocol, Server and Media Resource

```
vstcp://sunserver/mds/video/oracle1.mpi
vstcp://sunserver/vscontsrv:oracle1
```

In this example, **vstcp** specifies the TCP protocol, **sunserver** specifies the server and the default port 5000 (again, temporarily overriding the **OMN_ADDR** environment variable), **/mds/video/oracle1.mpi** specifies the MDS file, and **vscontsrv:oracle1** specifies the logical content asset cookie.

Example D–10 Specifying the Protocol and Media Resource

```
vstcp:///mds/video/oracle1.mpi
vstcp:///vscontsrv:oracle1
```

In this example, **vstcp** specifies the TCP protocol, **/mds/video/oracle1.mpi** specifies the MDS file, and **vscontsrv:oracle1** specifies the logical content asset cookie. The default server and port are used. Note that **OMN_ADDR** must be set for this to work. Since the protocol is specified, three forward slashes are required.

Example D–11 Specifying the Media Resource

```
/mds/video/oracle1.mpi
```

In this example, the default UDP protocol is used as well as the default server and port. Since the protocol is not specified, only one forward slash is required. Note that **OMN_ADDR** must be set for this to work.

Collected Video Session URL Grammar

This is the collected notation for Oracle Video Session URLs. Spaces should not be included, and pipes are separators which logically mean OR.

```
vsurl       =   interurl | bcasturl | file
interurl    =   vsi :// [ host[:port] ] / asset [ ? params ]
bcasturl    =   vsb : addr [ ? params ]
file        =   file :// asset
host[:port] =   server_address : OVS_port
asset       =   [ path ]filename
addr        =   pval
params      =   param *optparam
optparam    =   & param
param       =   pname = pval
pname       =   authid | authpw | bitrate | comm_proto | transport | data_proto |
                data_address | data_format | data_framing | pval
pval        =   1*pchar
pchar       =   unreserved | escape
unreserved  =   alphanum | mark
mark        =   - | _ | . | ! | ~ | * | ‘ | ( | )
escaped     =   % hexdigit hexdigit
```

Index

Symbols

A

B

C

D

E

F

G

H

I

J

L

M

N

O

P

Q

R

S

T

U

V

W